



Unități de învățare

Noțiuni de bază

Inserarea scripturilor

Afișarea datelor în JavaScript

Comentarii JavaScript

Tipuri de date

Variabile JavaScript

Operatori JavaScript

Caractere speciale

Casete Popup

Instrucțiuni JavaScript

Instrucțiuni condiționale

Instrucțiuni repetitive

Break și continue

Instrucțiunea with

Funcții

Evenimente JavaScript

Obiectele JavaScript

Obiectul String

Obiectul Date

Obiectul Array

Obiectul Boolean

Obiectul Math

Obiectul RegExp

Obiectul Number

Obiectul Navigator

[Obiectele browserului](#)[Cookies](#)[Obiectul Form](#)[Validarea formularelor](#)[Animație](#)[Imagini mapate](#)[Programarea evenimentelor](#)[Depanarea aplicațiilor](#)[Crearea obiectelor proprii](#)[Proprietăți și metode globale](#)

JavaScript - Noțiuni de bază

JavaScript a fost dezvoltat prima dată de către firma **Netscape**, cu numele de **Live Script**, un limbaj de script care extindea capacitățile HTML, oferă o alternativă parțială la utilizarea unui număr mare de scripturi CGI pentru prelucrarea informațiilor din formulare și care adaugă dinamism în paginile web. După lansarea limbajului **Java**, **Netscape** a început să lucreze cu firma **Sun**, cu scopul de a crea un limbaj de script cu o sintaxă și semantică asemănătoare cu a limbajului **Java**, și din motive de marketing numele noului limbaj de script a fost schimbat în "**JavaScript**". **JavaScript** a apărut din nevoia ca logica și inteligența să fie și pe partea de client, nu doar pe partea de server. Dacă toată logica este pe partea de server, întreaga prelucrare este făcută la server, chiar și pentru lucruri simple, așa cum este validarea datelor. Astfel, **JavaScript** îl înzestrează pe client și face ca relația să fie un adevărat sistem client-server. Limbajul **HTML** oferă autorilor de pagini Web o anumită flexibilitate, dar statică. Documentele **HTML** nu pot interacționa cu utilizatorul în alt mod mai dinamic, decât pune la dispoziția acestuia legături la alte resurse (URL-uri). Crearea de CGI-uri (Common Graphics Interface) - [programe care rulează pe serverul Web și care accepta informații primite din pagina de web și returnează cod HTML] - a dus la îmbogățirea posibilităților de lucru. Astfel, un pas important spre interactivizare a fost realizat **JavaScript**, care permite inserarea în paginile web a script-urilor care se execută în cadrul paginii web, mai exact în cadrul browser-ului utilizatorului, ușurând astfel și traficul dintre server și client. De exemplu, într-o pagină pentru colectarea de date de la utilizator se pot adăuga scripturi **JavaScript** pentru a valida corectitudinea introducerii și apoi pentru a trimite serverului doar date corecte spre procesare. **JavaScript** conține o listă destul de amplă de funcții și comenzi menite să ajute la operații matematice, manipulări de șiruri, sunete, imagini, obiecte și ferestre ale browser-ului, link-urile URL și verificări de introduceri ale datelor în formulare. Codul necesar acestor acțiuni poate fi inserat în pagina web și executat pe calculatorul vizitatorului. După lansarea sa, în decembrie 1995, **JavaScript** și-a atras sprijinul principalilor distribuitori din domeniu, cum sunt **Apple**, **Borland**, **Informix**, **Oracle**, **Sybase**, **HP** sau **IBM**. S-a dezvoltat în continuare, obținând recunoaștere majorității browserelor. Înțelegând importanța scripting-ului web, **Microsoft** s-a dorit să ofere suport și pentru **JavaScript**. **Netscape** a preferat să acorde licența de tehnologie companiei **Microsoft** în loc să o vândă, astfel **Microsoft** a analizat **JavaScript**, și bazându-se pe documentația publică a creat propria sa implementare, "**Jscript**", care este recunoscută de **Microsoft Internet Explorer**. **Jscript 1.0** este aproximativ compatibil cu **JavaScript 1.1**, care este recunoscut de **Netscape Navigator**. Totuși, versiunile ulterioare de **JavaScript** și diversele diferențe specifice platformelor de operare au început să dea destule probleme programatorilor web și astfel **Netscape**, **Microsoft** și

alti distribuitori au fost de acord sa predea limbaajul unei organizații internaționale de standardizare - ECMA ; aceasta a finalizat o specificatie de limbaaj, cunoscuta ca ECMAScript, recunoscuta de toti distribuitorii. Deși standardul ECMA este util, atit **Netscape** cat și **Microsoft** au propriile lor implementari ale limbaajului și continua sa extinda limbaajul dincolo de standardul de baza. Pe lângă **Jscript**, **Microsoft** a introdus și un concurent pentru **JavaScript**, numit **VBScript**, realizat pentru a usura patrunderea pe web a programatorilor **VB**. **VBScript** este un subset al limbaajului **Visual Basic**. Cu toate acestea **JavaScript** a devenit cunoscut ca limbaajul de scripting standard pentru web. În general se consideră că există zece aspecte fundamentale ale limbaajului **JavaScript** pe care orice programator în acest limbaaj ar trebui să le cunoască :

1. **JavaScript poate fi introdus în HTML** - De obicei codul JavaScript este gazduit în documentele HTML și executat în interiorul lor. Majoritatea obiectelor JavaScript au etichete HTML pe care le reprezintă, astfel încât programul este inclus pe partea de client a limbaajului. JavaScript foloseste HTML pentru a intra în cadrul de lucru al aplicațiilor pentru web.
2. **JavaScript este dependent de mediu** - JavaScript este un limbaaj de scriptare; software-ul care ruleaza de fapt programul este browser-ul web (Firefox, Opera, Netscape Navigator, Internet Explorer, Safari, etc.) Este important să luăm în considerare această dependență de browser atunci când utilizăm aplicații JavaScript.
3. **JavaScript este un limbaaj în totalitate interpretat** - codul scriptului va fi interpretat de browser înainte de a fi executat. JavaScript nu necesită compilări sau preprocesări, ci rămâne parte integranta a documentului HTML. Dezavantajul acestui limbaaj este că rularea durează ceva mai mult deoarece comenzile JavaScript vor fi citite de navigatorul Web și procesate atunci când user-ul apelează la acele funcții (prin completare de formulare, apăsare de butoane, etc). Avantajul principal este faptul că putem mult mai ușor să actualizăm codul sursă.
4. **JavaScript este un limbaaj flexibil** - în aceasta privință limbaajul diferă radical de C++ sau Java. În JavaScript putem declara o variabilă de un anumit tip, sau putem lucra cu o variabilă deși nu-i cunoaștem tipul specificat înainte de rulare.
5. **JavaScript este bazat pe obiecte** - JavaScript nu este un limbaaj de programare orientat obiect, ca Java, ci mai corect, este "bazat pe obiecte"; modelul de obiect JavaScript este bazat pe instanță și nu pe moștenire.
6. **JavaScript este condus de evenimente** - mare parte a codului JavaScript răspunde la evenimente generate de utilizator sau de sistem. Obiectele HTML, cum ar fi butoanele, sunt îmbunătățite pentru a accepta handlers de evenimente.
7. **JavaScript nu este Java** - Cele doua limbaje au fost create de companii diferite, motivul denumirii asemănătoare este legat doar de marketing.
8. **JavaScript este multifuncțional** - limbaajul poate fi folosit într-o multitudine de contexte pentru a rezolva diferite probleme: grafice, matematice, și altele.
9. **JavaScript evoluează** - limbaajul evoluează, fapt pozitiv care însă poate genera și probleme, programatorii trebuind să verifice permanent ce versiune să folosească pentru ca aplicațiile să poată fi disponibile unui numar cât mai mare de utilizatori de browsere diferite.
10. **JavaScript acoperă contexte diverse** - programarea cu acest limbaaj este îndreptată mai ales către partea de client, dar putem folosi JavaScript și pentru partea de Server. JavaScript este limbaajul nativ pentru unele instrumente de dezvoltare web, ca Borland IntraBuilder sau Macromedia Dreamweaver.

Inserarea scripturilor JavaScript

Pentru a insera cod JavaScript într-un document HTML, folosim tagul **<script>** și respectiv **</script>**. Deci tagurile **<script>** și **</script>** marchează locul în care începe, respectiv se sfârșește scriptul.

Obs: În exemplele vechi se utiliza atributul **type** pentru a defini limbaajul în care este scris scriptul. Atributul **type**

nu este necesar, JavaScript este limbajul de scripting implicit în HTML.

Pentru inserarea script-urilor în documente HTML pot fi folosite cele **4 metode** prezentate mai jos:

Metoda 1 – plasarea script-ului în antet-ul paginii (între **<head>** și **</head>**);

Metoda 2 – plasarea script-ului în corpul paginii (între **<body>** și **</body>**);

Metoda 3 – utilizarea fișierelor sursă externe;

Metoda 4 – crearea unui gestionar de evenimente.

Scripturi în antetul paginii

Scripturile care trebuie executate când sunt apelate sau când are loc un eveniment, trebuie scrise în secțiunea **head**. În acest fel, scriptul va fi sigur încărcat înainte de a fi utilizat.

```

1  <html>
2  <head>
3  <script>
4  function message()
5  {
6  alert("Aceasta caseta de alertare este apelata și afisata cind are loc evenimentul onload");
7  }
8  </script>
9  </head>
10
11 <body onload="message()">
12 <h3>Casetele de alertare</h3> <hr/>
13 </body>
14 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function message()
5  {
6  alert("Aceasta caseta de alertare este apelata și afisata cind are loc evenimentul onload");
7  }
8  </script>
9  </head>
10
11 <body onload="message()">
12 <h3>Casetele de alertare</h3> <hr/>
13 </body>
14 </html>

```

Scripturi în corpul paginii

Scripturile care trebuie executate când pagina se încarcă trebuie scrise în secțiunea **body** și vor genera conținutul paginii.

Exemplu

```

1  <html>
2  <head>
3  </head>
4  <body>
5  <h3>Afisarea textului cu JavaScript</h3> <hr/>
6  <script>
7  document.write("Acest mesaj este scris cu JavaScript");
8  </script>
9  </body>
10 </html>

```

```

1  <html>
2

```

```

3 | <head>
4 | </head>
5 | <body>
6 | <h3>Afisarea textului cu JavaScript</h3> <hr/>
7 | <script>
8 | document.write("Acest mesaj este scris cu JavaScript");
9 | </script>
10| </body>
   | </html>

```

Scripturi în fișier extern

Dacă dorim să utilizăm același script în mai multe pagini web fără a rescrie codul, trebuie să scriem scriptul JavaScript într-un fișier extern. Fișierul trebuie să aibă extensia **.js** și nu poate conține tagul **<script>**. Pentru a utiliza fișierul extern, indicăm numele fișierului în atributul **src** al tagului **<script>**.

Exemplu

```

1 |
2 | <html>
3 | <head>
4 | <script src="functie.js"></script>
5 | </head>
6 | <body>
7 | <p onclick="afisare()"> Apasa aici!</p>
8 | </body>
   | </html>

```

```

1 |
2 | <html>
3 | <head>
4 | <script src="functie.js"></script>
5 | </head>
6 | <body>
7 | <p onclick="afisare()"> Apasa aici!</p>
8 | </body>
   | </html>

```

Conținutul fișierului **functie.js**

```

1 |
2 | function afisare() {
3 | document.write("<h2>Imi place sa fiu asa cum sunt</h2>");
   | }

```

```

1 |
2 | function afisare() {
3 | document.write("<h2>Imi place sa fiu asa cum sunt</h2>");
   | }

```

Crearea unui gestionar de evenimente

Nu este obligatoriu ca toate script-urile JavaScript să se găsească în interiorul tag-urilor **<script> ... </script>**. Puteți apela, de asemenea la script-uri sub forma gestionarilor de evenimente, care indică navigatorului cum să reacționeze atunci când se produc anumite evenimente.

În exemplu de mai jos s-a creat un script care afișează un mesaj cu ajutorul gestionarului de evenimente **onClick**.

Exemplu

```

1 |
2 | <html>

```

```

3 <head>
4 </head>
5 <body>
6 <p onclick="alert('Imi place sa fiu asa cum sunt')"> Apasa aici!</p>
7 </body>
  </html>

1
2 <html>
3 <head>
4 </head>
5 <body>
6 <p onclick="alert('Imi place sa fiu asa cum sunt')"> Apasa aici!</p>
7 </body>
  </html>

```

Browsers care nu recunosc JavaScript

Dacă browserul nu recunoaște JS, liniile de cod vor fi afișate ca atare în pagină. Pentru a evita acest lucru, scriptul ar trebui „ascuns” în taguri de comentariu. În exemplul următor, scriptul este scris între tagurile de comentariu

```

1
2 <html>
3 <body>
4 <script>
5 <!--
6 document.write("Bine ati venit!");
7 //-->
8 </script>
9 </body>
  </html>

1
2 <html>
3 <body>
4 <script>
5 <!--
6 document.write("Bine ati venit!");
7 //-->
8 </script>
9 </body>
  </html>

```

Ultimele două caractere // reprezintă simbolul JS pentru comentariu și sunt scrise pentru a împiedica JS să execute tagul -->.

Afișarea datelor în JavaScript

În limbajul JavaScript datele pot fi afișate în **4 moduri** diferite:

Afișarea în fereastra de alertare, folosind **window.alert ()**.

Afișarea în documentul HTML folosind **document.write ()**.

Afișarea într-un element HTML, folosind **innerHTML**.

Afișarea în consola browser-ului, folosind **console.log ()**.

Utilizarea window.alert ()

Putem folosi o fereastră de alertare pentru a afișa date.

Exemplu

```

1

```

```

2  <!DOCTYPE html>
3  <html>
4  <body>
5  <h1>Prima mea pagina WEB.</h1>
6  <p>Primul meu paragraf.</p>
7  <script>
8      window.alert(5 + 6);
9  </script>
10 </body>
    </html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <script>
7      window.alert(5 + 6);
8  </script>
9  </body>
10 </html>

```

Utilizarea document.write ()

Pentru testare se recomandă de folosit metoda **document.write ()**.

Exemplu

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <script>
7      document.write(5 + 6);
8  </script>
9  </body>
10 </html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <script>
7      document.write(5 + 6);
8  </script>
9  </body>
10 </html>

```

Utilizarea metodei **document.write ()** după ce este complet încărcat un document HTML, **șterge toate elementele HTML existente:**

Exemplu

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <button onclick="document.write(5 + 6)">Click aici!</button>
7  </body>
8  </html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <button onclick="document.write(5 + 6)">Click aici!</button>
7
8  </body>
   </html>

```

Obs: document.write () este o metodă ce ar trebui să fie folosită numai pentru testare.

Utilizarea innerHTML

Pentru a accesa un element HTML, JavaScript poate utiliza metoda **document.getElementById (id)**. Atributul **id** definește elementul HTML. Proprietatea **innerHTML** definește conținutul HTML:

Exemplu

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <p id="demo"></p>
7  <script>
8      document.getElementById("demo").innerHTML = 5 + 6;
9  </script>
10 </body>
11 </html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <p id="demo"></p>
7  <script>
8      document.getElementById("demo").innerHTML = 5 + 6;
9  </script>
10 </body>
11 </html>

```

Utilizarea console.log ()

În browser-ul dvs., puteți utiliza metoda **console.log ()** pentru a afișa date. Activați consola browserului cu **F12**, și selectați **"Console"** din meniu.

Exemplu

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <h1>Prima mea pagina WEB.</h1>
5  <p>Primul meu paragraf.</p>
6  <script>
7      console.log(5 + 6);
8  </script>
9  </body>
10 </html>

```

```

1  <!DOCTYPE html>
2  <html>
3

```



```

4   <body>
5   <h1>Prima mea pagina WEB.</h1>
6   <p>Primul meu paragraf.</p>
7   <script>
8       console.log(5 + 6);
9   </script>
10  </body>
    </html>

```

Comentarii JavaScript

Comentariile pot fi adăugate pentru a explica codul sau a-l face mai ușor de citit. Comentariile care se scriu pe o singură linie încep cu //.

În exemplul următor, comentariile tip linie sunt folosite pentru a explica codul:

```

1   <html>
2   <body>
3   <script>
4   // Scrie un titlu:
5   document.write("<h1>Acesta este un titlu</h1>");
6   // Scrie doua paragrafe:
7   document.write("<p>Acesta este un paragraf.</p>");
8   document.write("<p>Acesta este un alt paragraf.</p>");
9   </script>
10  </body>
11  </html>

```

```

1   <html>
2   <body>
3   <script>
4   // Scrie un titlu:
5   document.write("<h1>Acesta este un titlu</h1>");
6   // Scrie doua paragrafe:
7   document.write("<p>Acesta este un paragraf.</p>");
8   document.write("<p>Acesta este un alt paragraf.</p>");
9   </script>
10  </body>
11  </html>

```

Comentarii multi-linie

Aceste comentarii încep cu /* și se încheie cu */, ca în exemplul următor:

```

1   <html>
2   <body>
3   <script>
4   /*
5   Codul urmator va scrie in pagina
6   un titlu și doua paragrafe
7   */
8   document.write("<h1>Acesta este un titlu</h1>");
9   document.write("<p>Acesta este un paragraf.</p>");
10  document.write("<p>Acesta este un alt paragraf.</p>");
11  </script>
12  </body>
13  </html>

```

```

1   <html>
2   <body>
3   <script>
4   /*

```

```

5
6   Codul urmator va scrie in pagina
7   un titlu și doua paragrafe
8   */
9   document.write("<h1>Acesta este un titlu</h1>");
10  document.write("<p>Acesta este un paragraf.</p>");
11  document.write("<p>Acesta este un alt paragraf.</p>");
12  </script>
13  </body>
    </html>

```

Folosirea comentariilor pentru a preveni execuția

În exemplul următor, comentariul este utilizat pentru a impiedica executarea unei linii de cod (metoda poate fi utilizată pentru a depăna codul):

```

1
2  <html>
3  <body>
4  <script>
5  //document.write("<h1>Acesta este un titlu</h1>");
6  document.write("<p>Acesta este un paragraf.</p>");
7  document.write("<p>Acesta este un alt paragraf.</p>");
8  </script>
9  </body>
    </html>

```

```

1
2  <html>
3  <body>
4  <script>
5  //document.write("<h1>Acesta este un titlu</h1>");
6  document.write("<p>Acesta este un paragraf.</p>");
7  document.write("<p>Acesta este un alt paragraf.</p>");
8  </script>
9  </body>
    </html>

```

În exemplul următor, comentariul este utilizat pentru a impiedica execuția unui bloc de cod (util pentru depănarea codului):

```

1
2  <html>
3  <body>
4  <script>
5  /*
6  document.write("<h1>Acesta este un titlu</h1>");
7  document.write("<p>Acesta este un paragraf.</p>");
8  document.write("<p>Acesta este un alt paragraf.</p>");
9  */
10 </script>
11 </body>
    </html>

```

```

1
2  <html>
3  <body>
4  <script>
5  /*
6  document.write("<h1>Acesta este un titlu</h1>");
7  document.write("<p>Acesta este un paragraf.</p>");
8  document.write("<p>Acesta este un alt paragraf.</p>");
9  */
10 </script>
11 </body>
    </html>

```

În exemplul următor, comentariul este plasat la sfârșitul liniei de cod:

```

1  <html>
2  <body>
3  <script>
4  document.write("Salutare"); // scrie in pagina textul "Salutare"
5  document.write(" prieteni!"); // scrie in pagina textul " prieteni!"
6  </script>
7  </body>
8  </html>

1  <html>
2  <body>
3  <script>
4  document.write("Salutare"); // scrie in pagina textul "Salutare"
5  document.write(" prieteni!"); // scrie in pagina textul " prieteni!"
6  </script>
7  </body>
8  </html>

```

Tipuri de date JavaScript

În JavaScript există 5 tipuri de date diferite care pot conține valori:

- Number
- String
- Boolean
- Object
- Function

Există 3 tipuri de obiecte:

- Object
- Date
- Array

Și 2 tipuri de date care nu pot conține valori:

- Null
- Undefined

Date numerice

Limbaajul JavaScript permite specificarea datelor numerice în patru formate diferite: **întreg**, **virgulă flotantă**, **octal** și **hexazecimal**.

Prin definiție, numerele în octal și în hexazecimal sunt numere întregi care sunt exprimate într-un sistem de numerație cu baza **8**, respectiv baza **16**. În JavaScript un număr întreg octal este precedat de zero iar un număr hexazecimal este precedat de caracterele **"0x"** sau **"0X"**.

Observații:

JavaScript recunoaște numerele întregi (în baza **10**, pozitive sau negative) cuprinse între: **+/-**

1.7976931348623157 E 308 și **+/-5 E - 324**.

Un număr întreg hexazecimal (hexadecimal, în limba engleză) începe în mod obligatoriu cu **0X** sau **0x** și este compus din următoarele simboluri: **0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F**. Putem utiliza majuscule sau minuscule.

Un număr întreg octal începe obligatoriu cu **0** (zero) și este compus din următoarele simboluri: **0, 1, 2, 3, 4, 5, 6, 7**

Un număr în virgulă flotantă este în baza **10**. El poate fi pozitiv sau negativ și poate conține zecimale. El poate de asemenea include un exponent pozitiv sau negativ, prin **E**. Separatorul zecimal este întotdeauna punctul.

JavaScript recunoaște numerele cuprinse între: **+/-1.7976931348623157 E 308** și **+/-5 E -324**.

Șir de caractere

Un șir de caractere (**string**, în limba engleză) este compus din litere, cifre, simboluri, caractere speciale și secvențe de ieșire. Conținutul unui șir de caractere este considerat tot timpul ca fiind text, chiar dacă el constă din cifre și simboluri numerice. Un șir de caractere este încadrat de ghilimele simple sau duble. Un șir de caractere încadrat de ghilimele duble poate fi inclus într-un șir încadrat de ghilimele simple și vice versa.

Valori logice sau booleene

Ele sunt în număr de două: **true (adevărat)** și **false (fals)**. Cele două valori se folosesc pentru a indica dacă rezultatul evaluării unei condiții este adevărat sau nu.

Variable JavaScript

De obicei, limbajele de programare cer să definiți tipul de date pe care-l va reprezenta o variabilă, în plus se generează o eroare atunci când încercați să-i atribuiți variabilei un alt tip de date. Din fericire, așa ceva nu se întâmplă în JavaScript, care este un limbaj flexibil. Variabilele JavaScript pot accepta oricând un nou tip de date, fapt care duce la modificarea tipului variabilei.

Reguli pentru numele variabilelor JavaScript:

Numele este case-sensitive (**y** și **Y** sunt două variabile diferite)

Numele trebuie să înceapă cu o literă sau cu liniuța de subliniere (underscore)

Exemplu

Valoarea unei variabile se poate modifica în timpul execuției scriptului. Putem referi variabila prin nume pentru a-i afișa sau modifica conținutul, ca în exemplul următor:

```

1  <html>
2  <body>
3  <h3>Declararea, initializarea, atribuirea și afisarea unei variabile</h3> <hr/>
4  <script>
5  var prenume;
6  prenume="Mihai";
7  document.write("<b>Numele variabilei</b>: prenume");
8  document.write("<br/>");
9  document.write("<b>Valoare initiala</b>: "+ prenume);
10 document.write("<br/>");
11 prenume="Adrian";
12 document.write("<b>Valoare dupa atribuire</b>: "+ prenume);
13 </script>
14 </body>
15 </html>
```

```

1  <html>
2  <body>
3  <h3>Declararea, initializarea, atribuirea și afisarea unei variabile</h3> <hr/>
4  <script>
5  var prenume;
6  prenume="Mihai";
7  document.write("<b>Numele variabilei</b>: prenume");
8  document.write("<br/>");
9  document.write("<b>Valoare initiala</b>: "+ prenume);
10 document.write("<br/>");
11 prenume="Adrian";
12 document.write("<b>Valoare dupa atribuire</b>: "+ prenume);
```

```
13
14 | </script>
15 | </body>
    | </html>
```

Declararea variabilelor JavaScript

Putem crea variabile cu sintaxa:

```
var nume_variabila;
```

După declarare, variabila nu conține valori (este vidă). Puteți să inițializați o variabilă chiar în momentul declarării:

```
var x=8;
var prenume="Matei";
```

Obs: cînd atribuiți unei variabile o valoare de tip text, textul trebuie scris între ghilimele.
Dacă atribuiți valori unei variabile care nu a fost încă declarată, ea va fi declarată automat.

Declarațiile:

```
x=8;
prenume="Matei";
```

au același efect cu declarațiile:

```
var x=8;
var prenume="Matei";
```

Dacă redeclarați o variabilă JavaScript, ea va păstra valoarea inițială:

```
var x=7;
var x;
```

După execuția instrucțiunilor de mai sus, variabila **x** are valoarea **7** care nu a fost resetată la redeclarare.

Operatori JavaScript

Operatorii aritmetici

Sunt folosiți pentru a efectua operații aritmetice cu variabile și/sau valori.
Dacă y=5, tabelul următor prezintă operatorii aritmetici:

Operator	Descriere	Exemplu	Rezultat
+	adunare	x=y+2	x=7
-	scădere	x=y-2	x=3
*	înmulțire	x=y*2	x=10
/	împărțire	x=y/2	x=2.5
%	modulo (restul împărțirii întregi)	x=y%2	x=1

Operatorii de atribuire

Sunt folosiți pentru a atribui valori variabilelor JavaScript. Dacă x=10 și y=5, tabelul următor prezintă operatorii de atribuire:

Operator	Exemplu	Echivalent cu	Rezultat

=	x=y		x=5
+=	x+=y	x=x+y	x=15
-=	x-=y	x=x-y	x=5
=	x=y	x=x*y	x=50
/=	x/=y	x=x/y	x=2
%=	x%=y	x=x%y	x=0

Operatorul + utilizat pentru șiruri de caractere

Acest operator poate fi utilizat și pentru a concatena variabile tip șir de caractere (string sau text).

Exemplu:

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <p>Operatorul + utilizat pentru șiruri de caractere.</p>
5  <script>
6  var txt1 = "Ce mai";
7  var txt2 = "faci azi?";
8  document.write(txt1 + txt2);
9  </script>
10 </body>
11 </html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4  <p>Operatorul + utilizat pentru șiruri de caractere.</p>
5  <script>
6  var txt1 = "Ce mai";
7  var txt2 = "faci azi?";
8  document.write(txt1 + txt2);
9  </script>
10 </body>
11 </html>

```

Adunarea șirurilor și a numerelor

Regulă: Dacă adunați un număr cu un șir de caractere, veți obține un șir de caractere.

Exemplu:

```

1  <html>
2  <body>
3  <h3>Adunarea șirurilor de caractere și a numerelor cu șiruri de caractere</h3> <br/>
4  <script>
5  x=6+7;
6  document.write("6+7="+x);
7  document.write("<br />");
8  x="6"+"7";
9  document.write("'6"+"7"='+x);
10 document.write("<br />");
11 x=6+"7";
12 document.write('6+"7"='+x);
13 document.write("<br />");
14 x="6"+7;
15 document.write("'6"+7='+'+x);
16 document.write("<br />");
17

```

```

18 </script>
19 </body>
    </html>

1 <html>
2 <body>
3 <h3>Adunarea sirurilor de caractere și a numerelor cu siruri de caractere</h3> <hr/>
4 <script>
5 x=6+7;
6 document.write("6+7="+x);
7 document.write("<br />");
8 x="6"+"7";
9 document.write('"6"+"7"='+x);
10 document.write("<br />");
11 x=6+"7";
12 document.write('6+"7"='+x);
13 document.write("<br />");
14 x="6"+7;
15 document.write('"6"+7=' +x);
16 document.write("<br />");
17 </script>
18 </body>
19 </html>

```

Operatori de incrementare / decrementare

Operatorii se mai clasifică și în funcție de operația pe care o efectuează și anume în:

operatori de incrementare - crește valoarea variabilei cu o unitate

operatori de decrementare - scade valoarea variabilei cu o unitate;

Acestea la rândul lor sunt de două feluri:

prefixată - operatorul se scrie înaintea operandului. Operația de incrementare (sau decrementare) se efectuează înainte de a se efectua operațiile expresiei în care se află.

De exemplu, dacă inițial a=1 și x=++a; vom avea la final x=2 și a=2, adică se incrementează variabila a cu o unitate înainte de a i se atribui variabilei x valoarea variabilei a.

postfixată - operatorul se scrie după operand. Operația de incrementare (sau decrementare) se efectuează după ce se efectuează operațiile expresiei în care se află.

De exemplu, dacă inițial a=1 și x=a++; vom avea la final x=1 și a=2, adică se incrementează variabila a cu o unitate după ce i se va atribui variabilei x valoarea variabilei a.

Operator	Tip de operator	Descriere	Exemplu
++	incrementare	incrementare prefixată	++x sau x=x+1
		incrementare postfixată	x++ sau x=x+1
--	decrementare	decrementare prefixată	--x sau x=x-1
		decrementare postfixată	x-- sau x=x-1

Exemplu

```

1 <html>
2 <head><title>Operatori de incrementare/decrementare </title>
3 </head>
4 <body>
5 <script >
6 var x=1, y=2;
7 document.write("x=1 y=2");
8

```

```
9 | document.write("<br />");
10 | document.write("z=++x-y--=",++x-y--);
11 | </script>
12 | </body>
    | </html>

1 |
2 | <html>
3 | <head><title>Operatori de incrementare/decrementare </title>
4 | </head>
5 | <body>
6 | <script >
7 | var x=1, y=2;
8 | document.write("x=1 y=2");
9 | document.write("<br />");
10 | document.write("z=++x-y--=",++x-y--);
11 | </script>
12 | </body>
    | </html>
```

Operatorii de comparare

Operatorii de comparare sunt utilizați în construcții logice pentru a verifica egalitatea sau diferența dintre două variabile sau valori. Dacă x=5, tabelul următor prezintă operatorii de comparare

Operator	Descriere	Exemple
==	Egal cu	x==5 este fals
===	Este egal exact (valoare și tip)	x===5 este adevărat x===5" este fals
!=	Diferit	x!=8 este adevăra
>	Mai mare decit	x>8 este fals
<	Mai mic decit	x<8 este adevărat
>=	Mai mare sau egal cu	x>=8 este fals
<=	Mai mic sau egal cu	x<=8 este adevarat

Operatorii logici

Operatorii logici sunt utilizați pentru a determina relația logică dintre variabile sau valori. Dacă x=6 și y=3, tabelul următor prezintă operatorii logici:

Operator	Descriere	Exemple
&&	și	(x < 10 && y > 1) este adevărat
	sau	(x==5 y==5) este fals
!	not	!(x==y) este adevărat

Operatorul condițional(ternar)

Acest operator atribuie o valoare unei variabile în funcție de o anumită condiție.

Sintaxă:

variabila=(conditie)?valoare1:valoare2

Exemplul 1:

```

1  <!doctype html>
2  <html>
3  <head>
4  <title>Operatorul ternar ?</title>
5  </head>
6  <body>
7  <script>
8      var a=4;
9      document.write(
10         a <= 5 ?
11         "a este mai mic și egal ca 5" :
12         "a este mai mare ca 5"
13     )
14 </script>
15 </body>
16 </html>

```

```

1  <!doctype html>
2  <html>
3  <head>
4  <title>Operatorul ternar ?</title>
5  </head>
6  <body>
7  <script>
8      var a=4;
9      document.write(
10         a <= 5 ?
11         "a este mai mic și egal ca 5" :
12         "a este mai mare ca 5"
13     )
14 </script>
15 </body>
16 </html>

```

Exemplul 2:

```

1  <html>
2  <body>
3  <script>
4      var x=5; y=7;
5      document.write("x=2 y=3");
6      document.write("<br />");
7      document.write("Calculati elementul maxim.");
8      document.write("<br />");
9      document.write("Expresia conditionala este:");
10     document.write(" (x>y)?max=x:max=y");
11     document.write("<br />");
12     document.write("Rezultatul este: ",(x>y)?"max=x":"max=y");
13 </script>
14 </body>
15 </html>

```

```

1  <html>
2  <body>
3  <script>
4      var x=5; y=7;
5      document.write("x=2 y=3");
6      document.write("<br />");
7      document.write("Calculati elementul maxim.");
8      document.write("<br />");
9      document.write("Expresia conditionala este:");
10     document.write(" (x>y)?max=x:max=y");
11     document.write("<br />");
12     document.write("Rezultatul este: ",(x>y)?"max=x":"max=y");

```

```

13 document.write("Rezultatul este: ",(x>y)?"max=x":"max=y");;
14 </script>
15 </body>
    </html>

```

Operatorul typeof

Operatorul **typeof** returnează tipul de date continut de operandul său. Tipurile de date pe care le poate returna sunt:

string - pentru șiruri de caractere

number - pentru numere

function - pentru funcțiile JavaScript

object - pentru obiectele JavaScript

Exemplu:

```

1 <!DOCTYPE html>
2 <html>
3 <body>
4 <p>Operatorul typeof utilizat pentru a returna tipul de date.</p>
5 <script>
6 document.write("variabila: tipul de date <br>");
7 var x1=-33.4;
8 document.write(x1+ ": ", typeof x1);
9 document.write("<br>");
10 x2=234;
11 document.write(x2+ ": ", typeof x2);
12 document.write("<br>");
13 y="Lucian Blaga";
14 document.write(y+ ": ", typeof y);
15 document.write("<br>");
16 z=escape;
17 document.write(z+ ": ", typeof z);
18 document.write("<br>");
19 t=Image;
20 document.write(t+ ": ", typeof t);
21 </script>
22 </body>
23 </html>

```

```

1 <!DOCTYPE html>
2 <html>
3 <body>
4 <p>Operatorul typeof utilizat pentru a returna tipul de date.</p>
5 <script>
6 document.write("variabila: tipul de date <br>");
7 var x1=-33.4;
8 document.write(x1+ ": ", typeof x1);
9 document.write("<br>");
10 x2=234;
11 document.write(x2+ ": ", typeof x2);
12 document.write("<br>");
13 y="Lucian Blaga";
14 document.write(y+ ": ", typeof y);
15 document.write("<br>");
16 z=escape;
17 document.write(z+ ": ", typeof z);
18 document.write("<br>");
19 t=Image;
20 document.write(t+ ": ", typeof t);
21 </script>
22 </body>
23 </html>

```

Inserarea caracterelor speciale

Pentru a insera într-un șir caractere speciale, cum ar fi apostrof, ghilimele, întrerupere de linie etc., se folosește caracterul backslash (\).

Fie următorul cod JavaScript:

```
1 |  
2 | var txt="Noi suntem echipa "Dinamo" din Bucuresti.";  
   | document.write(txt);  
  
1 |  
2 | var txt="Noi suntem echipa "Dinamo" din Bucuresti.";  
   | document.write(txt);
```

În JavaScript, un șir de caractere începe și termină cu apostrof sau cu ghilimele. Asta înseamnă că șirul de mai sus va fi trunchiat la: Noi suntem echipa

Pentru a rezolva această problemă, trebuie să inserați caracterul backslash (\) inaintea fiecărui caracter special care trebuie afișat, ca în exemplul următor:

```
1 |  
2 | var txt="Noi suntem echipa \"Dinamo\" din Bucuresti.";  
   | document.write(txt);  
  
1 |  
2 | var txt="Noi suntem echipa \"Dinamo\" din Bucuresti.";  
   | document.write(txt);
```

Tabelul următor prezintă caracterele speciale ce pot fi inserate într-un text cu ajutorul caracterului backslash:

Cod	leșire
\'	apostrof
\"	ghilimele
\&	ampersand
\\	backslash
\n	linie nouă
\r	retur de car
\t	tab
\b	backspace
\f	form feed

Casete Popup

JavaScript are trei tipuri de casete popup: **caseta Alert**, **caseta Confirm** și **caseta Prompt**.

Caseta Alert

O casetă de alertă se utilizează atunci doriți să fiți siguri că o anumită informație ajunge în atenția utilizatorului.

cînd o casetă de alertă este afișată, utilizatorul va trebui să acționeze butonul **"OK"** pentru a putea continua.

Sintaxă:

```
1 alert("...un text...");
```

```
1 alert("...un text...");
```

Exemplu:

```
1 <html>
2 <head>
3 <script>
4 function afiseaza_alert()
5 {
6   alert("Sunt o caseta de alertare!");
7 }
8 </script>
9 </head>
10 <body>
11 <h3>La apasarea butonului va fi apelata o functie care afiseaza caseta alert</h3> <hr/>
12 <input type="button" onclick="afiseaza_alert()" value="Apasa" />
13 </body>
14 </html>
```

```
1 <html>
2 <head>
3 <script>
4 function afiseaza_alert()
5 {
6   alert("Sunt o caseta de alertare!");
7 }
8 </script>
9 </head>
10 <body>
11 <h3>La apasarea butonului va fi apelata o functie care afiseaza caseta alert</h3> <hr/>
12 <input type="button" onclick="afiseaza_alert()" value="Apasa" />
13 </body>
14 </html>
```

Caseta Confirm

O casetă de confirmare se utilizează atunci cînd doriți ca utilizatorul să verifice sau să accepte ceva. Cînd caseta de confirmare este afișată, utilizatorul va trebui să acționeze butonul **"OK"** sau butonul **"Cancel"** pentru a putea continua. Dacă utilizatorul acționează butonul **"OK"**, caseta returnează valoarea **true**, dacă acționează butonul **"Cancel"**, caseta returnează valoarea **false**.

Sintaxă:

```
1 confirm("...un text...");
```

```
1 confirm("...un text...");
```

Exemplu:

```
1
```

```

1  <html>
2  <head>
3  <script>
4  function afiseaza_confirm()
5  {
6  var r=confirm("Apasati un buton");
7  if (r==true)
8  {
9  document.write("Ati apasat butonul OK!");
10 }
11 else
12 {
13 document.write("Ati apasat butonul Cancel!");
14 }
15 }
16 }
17 </script>
18 </head>
19 <body>
20 <h3>La apasarea butonului va fi apelata o functie care afiseaza caseta confirm și verifica ce
21 buton ati apasat</h3>
22 <hr/>
23 <input type="button" onclick="afiseaza_confirm()" value="Apasa" />
    </body>
    </html>

```

```

1  <html>
2  <head>
3  <script>
4  function afiseaza_confirm()
5  {
6  var r=confirm("Apasati un buton");
7  if (r==true)
8  {
9  document.write("Ati apasat butonul OK!");
10 }
11 else
12 {
13 document.write("Ati apasat butonul Cancel!");
14 }
15 }
16 }
17 </script>
18 </head>
19 <body>
20 <h3>La apasarea butonului va fi apelata o functie care afiseaza caseta confirm și verifica ce
21 buton ati apasat</h3>
22 <hr/>
23 <input type="button" onclick="afiseaza_confirm()" value="Apasa" />
    </body>
    </html>

```

Caseta Prompt

Această casetă se utilizează atunci când doriți ca utilizatorul să introducă o anumită valoare înainte de a accesa pagina. Când caseta prompt este afișată, utilizatorul va trebui să acționeze butonul **"OK"** sau butonul **"Cancel"** pentru a putea continua după ce introduce valoarea solicitată. Dacă utilizatorul acționează butonul **"OK"**, caseta returnează valoarea true, dacă acționează butonul **"Cancel"**, caseta returnează valoarea false.

Sintaxă:

```

1  prompt("....un text....","valoare_implicita");

1  prompt("....un text....","valoare_implicita");

```

Exemplu:

```

1  <html>
2  <head>
3  <script>
4  function afiseaza_prompt()
5  {
6  var name=prompt("Va rog sa va introduceti numele","");
7  if (name!=null && name!="")
8  {
9  document.write("Buna ziua " + name + "! Ce mai faci?");
10 }
11 }
12 }
13 </script>
14 </head>
15 <body>
16 <h3>La apasarea butonului va fi apelata o functie care afiseaza caseta prompt</h3> <hr/>
17 <input type="button" onclick="afiseaza_prompt()" value="Apasa" />
18 </body>
19 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function afiseaza_prompt()
5  {
6  {
7  var name=prompt("Va rog sa va introduceti numele","");
8  if (name!=null && name!="")
9  {
10 document.write("Buna ziua " + name + "! Ce mai faci?");
11 }
12 }
13 }
14 </script>
15 </head>
16 <body>
17 <h3>La apasarea butonului va fi apelata o functie care afiseaza caseta prompt</h3> <hr/>
18 <input type="button" onclick="afiseaza_prompt()" value="Apasa" />
19 </body>
20 </html>

```

Obs. Dacă doriți ca textul dintr-o casetă să fie afișat pe mai multe linii, procedați ca în exemplul următor:

```

1  <html>
2  <head>
3  <script>
4  function afiseaza_alert()
5  {
6  {
7  alert("Buna! Asa se adauga" + '\n' + "o întrerupere de linie" + '\n' + "intr-o caseta de
8  alertare!");
9  }
10 }
11 </script>
12 </head>
13 <body>
14 <h3>Caseta alert cu textul scris pe mai multe linii</h3> <hr/>
15 <input type="button" onclick="afiseaza_alert()" value="Apasa" />
16 </body>
17 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function afiseaza_alert()
5  {
6  {
7  alert("Buna! Asa se adauga" + '\n' + "o întrerupere de linie" + '\n' + "intr-o caseta de
8  alertare!");

```

```

9   }
10  </script>
11  </head>
12  <body>
13  <h3>Caseta alert cu textul scris pe mai multe linii</h3> <hr/>
14  <input type="button" onclick="afiseaza_alert()" value="Apasa" />
    </body>
    </html>

```

Instrucțiuni JavaScript

JavaScript este o secvență de declarații, instrucțiuni și comenzi care vor fi executate de către browser. Spre deosebire de HTML, JavaScript este case-sensitive, deci aveți grijă când scrieți instrucțiuni, declarați variabile sau apelați funcții. O instrucțiune JavaScript este o comandă către browser și are rolul de a spune browserului ce trebuie să facă. Următoarea instrucțiune JavaScript spune browser-ului să scrie în pagină textul "Buna ziua":

document.write("Buna ziua");

Fiecare instrucțiune se încheie cu punct și virgulă (;).

Codul JavaScript este o secvență de instrucțiuni JavaScript. Fiecare instrucțiune este executată de browser în ordinea în care a fost scrisă.

Exemplul următor va scrie un titlu și două paragrafe într-o pagină web:

```

1   <html>
2   <body>
3   <h3>Utilizarea tagurilor HTML in mesajele afisate cu JavaScript</h3> <hr/>
4   <script>
5   document.write("<h1>Acesta este un titlu</h1>");
6   document.write("<p>Acesta este un paragraf.</p>");
7   document.write("<p>Acesta este un alt paragraf.</p>");
8   </script>
9   </body>
10  </html>

```

```

1   <html>
2   <body>
3   <h3>Utilizarea tagurilor HTML in mesajele afisate cu JavaScript</h3> <hr/>
4   <script>
5   document.write("<h1>Acesta este un titlu</h1>");
6   document.write("<p>Acesta este un paragraf.</p>");
7   document.write("<p>Acesta este un alt paragraf.</p>");
8   </script>
9   </body>
10  </html>

```

Blocuri JavaScript

Instrucțiunile JavaScript pot fi grupate în blocuri care se scriu între acolade. Instrucțiunile dintr-un bloc vor fi executate împreună.

În acest exemplu, instrucțiunile care scriu un titlu și două paragrafe, au fost grupate împreună într-un bloc.

```

1   <html>
2   <body>
3   <script>
4   {
5   document.write("<h1>Acesta este un titlu</h1>");
6   document.write("<p>Acesta este un paragraf.</p>");
7   document.write("<p>Acesta este un alt paragraf.</p>");
8   }

```

```

9
10 </script>
11 </body>
    </html>

1
2 <html>
3 <body>
4 <script>
5 {
6 document.write("<h1>Acesta este un titlu</h1>");
7 document.write("<p>Acesta este un paragraf.</p>");
8 document.write("<p>Acesta este un alt paragraf.</p>");
9 }
10 </script>
11 </body>
    </html>

```

În mod normal, un bloc este folosit pentru a grupa un grup de instrucțiuni într-o funcție sau într-o condiție (blocul va fi executat dacă o anumită condiție este satisfăcută).

Instrucțiunile condiționale

Adesea, când scrieți cod JavaScript, trebuie să realizați operații diferite în funcție de decizii diferite. Pentru a realiza acest lucru, folosiți în cod instrucțiunile condiționale.

În JavaScript există următoarele instrucțiuni condiționale:

if - folosiți această instrucțiune dacă un cod trebuie executat când o condiție este adevărată

if...else - folosiți această instrucțiune pentru a executa un cod când o condiție este adevărată și alt cod dacă condiția este falsă

if...else if....else - folosiți această instrucțiune pentru a selecta unul din mai multe blocuri de cod pentru a fi executat

switch - folosiți această instrucțiune pentru a selecta unul din mai multe blocuri de cod pentru a fi executat

Instrucțiunea if

Sintaxă:

```

1
2 if (conditie) {
3   cod ce trebuie executat daca conditia este adevarata
4 }

1
2 if (conditie) {
3   cod ce trebuie executat daca conditia este adevarata
4 }

```

Exemplu:

```

1
2 <html>
3 <body>
4 <h3>Scriptul va afisa un mesaj daca ora<10 folosind instructiunea if</h3> <hr/>
5 <script>
6 var d = new Date();
7 var time = d.getHours();
8   if (time < 10)
9   {
10 document.write("<b>Buna dimineata</b>");
11 }
12 </script>

```



```

13 | </body>
    | </html>

1 |
2 | <html>
3 | <body>
4 | <h3>Scriptul va afisa un mesaj daca ora<10 folosind instructiunea if</h3> <hr/>
5 | <script>
6 |   var d = new Date();
7 |   var time = d.getHours();
8 |   if (time < 10)
9 |   {
10 | document.write("<b>Buna dimineata</b>");
11 |   }
12 | </script>
13 | </body>
    | </html>

```

Instructiunea if...else

Sintaxă:

```

1 |
2 | if (conditie) {
3 |   cod executat daca conditia este adevarata
4 | }
5 | else {
6 |   cod executat daca conditia este falsa
  | }

1 |
2 | if (conditie) {
3 |   cod executat daca conditia este adevarata
4 | }
5 | else {
6 |   cod executat daca conditia este falsa
  | }

```

Exemplu:

```

1 |
2 | <html>
3 | <body>
4 | <h3>Scriptul va afisa un mesaj sau altul in functie de ora, cu instructiunea if..else</h3>
5 | <hr/>
6 | <script>
7 |   var d = new Date();
8 |   var time = d.getHours();
9 |   if (time < 10)
10 |   {
11 | document.write("<b>Buna dimineata</b>");
12 |   }
13 |   else
14 |   {
15 | document.write("<b>Buna ziua</b>");
16 |   }
17 | </script>
18 |
    | </body>
    | </html>

1 |
2 | <html>
3 | <body>
4 | <h3>Scriptul va afisa un mesaj sau altul in functie de ora, cu instructiunea if..else</h3>
5 | <hr/>
6 | <script>

```

```

7   var d = new Date();
8   var time = d.getHours();
9   if (time < 10)
10  {
11    document.write("<b>Buna dimineata</b>");
12  }
13  else
14  {
15    document.write("<b>Buna ziua</b>");
16  }
17  </script>
18  </body>
    </html>

```

Instrucțiunea if...else if...else

Sintaxă:

```

1
2   if (conditia1) {
3     cod executat daca conditia1 este adevarata
4   }
5   else if (conditia2) {
6     cod executat daca conditia2 este adevarata
7   }
8   else {
9     cod executat daca nici conditia1, nici conditia2 nu sunt adevarate
10  }

```

```

1
2   if (conditia1) {
3     cod executat daca conditia1 este adevarata
4   }
5   else if (conditia2) {
6     cod executat daca conditia2 este adevarata
7   }
8   else {
9     cod executat daca nici conditia1, nici conditia2 nu sunt adevarate
10  }

```

Exemplu:

```

1
2   <html>
3   <body>
4   <h3>Scriptul va afisa unul din trei mesaje in functie de ora, cu instructiunea if-else-if-
5   else</h3> <hr/>
6   <script>
7     var d = new Date();
8     var time = d.getHours();
9     if (time<10)
10    {
11      document.write("<b>Buna dimineata</b>");
12    }
13    else if (time>=10 && time<17)
14    {
15      document.write("<b>Buna ziua</b>");
16    }
17    else
18    {
19      document.write("<b>Buna seara</b>");
20    }
21  </script>
22  </body>
    </html>

```

```

1  <html>
2  <body>
3  <h3>Scriptul va afisa unul din trei mesaje in functie de ora, cu instructiunea if-else-if-
4  else</h3> <hr/>
5  <script>
6  var d = new Date();
7  var time = d.getHours();
8  if (time<10)
9  {
10 {
11 document.write("<b>Buna dimineata</b>");
12 }
13 else if (time>=10 && time<17)
14 {
15 document.write("<b>Buna ziua</b>");
16 }
17 else
18 {
19 document.write("<b>Buna seara</b>");
20 }
21 </script>
22 </body>
    </html>

```

Exemplu:

Link-ul din exemplul următor va deschide Google sau Yahoo

```

1  <html>
2  <body>
3  <h3>Scriptul afiseaza in mod aleator unul din doua link-uri, folosind if..else</h3> <hr/>
4  <script>
5  var r=Math.random();
6  if (r>0.5)
7  {
8  {
9  document.write("<a href='http://www.google.com'>Google!</a>");
10 }
11 else
12 {
13 document.write("<a href='http://www.yahoo.com'>Yahoo!</a>");
14 }
15 </script>
16 </body>
    </html>

```

```

1  <html>
2  <body>
3  <h3>Scriptul afiseaza in mod aleator unul din doua link-uri, folosind if..else</h3> <hr/>
4  <script>
5  var r=Math.random();
6  if (r>0.5)
7  {
8  {
9  document.write("<a href='http://www.google.com'>Google!</a>");
10 }
11 else
12 {
13 document.write("<a href='http://www.yahoo.com'>Yahoo!</a>");
14 }
15 </script>
16 </body>
    </html>

```

Instrucțiunea switch**Sintaxă:**

```

1  switch(exp) {
2
3  case val-1: executa bloc-1; break;
4  case val-2: executa bloc-2; break;
5  .....
6  default: cod executat daca exp este diferit de val-1, val-2,...
   }

```

```

1  switch(exp) {
2
3  case val-1: executa bloc-1; break;
4  case val-2: executa bloc-2; break;
5  .....
6  default: cod executat daca exp este diferit de val-1, val-2,...
   }

```

Exemplu:

```

1  <html>
2  <body>
3  <h3>Scriptul utilizeaza instructiunea switch</h3>
4  <hr/>
5  <script>
6  var d = new Date();
7  theDay=d.getDay();
8  switch (theDay)
9  {
10 {
11 case 5: document.write("<b>Vineri</b>"); break;
12 case 6: document.write("<b>Sambata</b>"); break;
13 case 0: document.write("<b>Duminica</b>"); break;
14 default: document.write("<b>Astept weekend-ul!</b>");
15 }
16 </script>
17 </body>
   </html>

```

```

1  <html>
2  <body>
3  <h3>Scriptul utilizeaza instructiunea switch</h3>
4  <hr/>
5  <script>
6  var d = new Date();
7  theDay=d.getDay();
8  switch (theDay)
9  {
10 {
11 case 5: document.write("<b>Vineri</b>"); break;
12 case 6: document.write("<b>Sambata</b>"); break;
13 case 0: document.write("<b>Duminica</b>"); break;
14 default: document.write("<b>Astept weekend-ul!</b>");
15 }
16 </script>
17 </body>
   </html>

```

Exerciții:

1. Să se afle minimum și maximum din trei numere date **a**, **b**, **c**, cu ajutorul instrucțiunii if else.
2. Sunt date trei numere **a**, **b**, **c**. De afișat numerele în ordine crescătoare(descrescătoare).

Instrucțiuni repetitive

Instrucțiunile repetitive sunt utilizate pentru a executa o secvență de cod în mod repetat. În JavaScript putem folosi următoarele instrucțiuni repetitive:

for - repetă o secvență de cod de un număr precizat de ori

for...in - parcurge elementele unui tablou sau a enumera proprietățile unui obiect

while - repetă o secvență de cod cît timp o condiție este adevărată

do...while - întâi execută o dată secvență de cod apoi o repetă cît timp o condiție este adevărată

Instrucțiunea for

Instrucțiunea **for** se utilizează cînd se cunoaște dinainte numărul de iterații dorit pentru secvența de cod.

Sintaxă:

```
1
2  for (var=val_iniciala; var<=val_finala; var=var+increment)
3  {
4  codul ce trebuie executat
5  }
```

```
1
2  for (var=val_iniciala; var<=val_finala; var=var+increment)
3  {
4  codul ce trebuie executat
5  }
```

Exemplu:

În exemplul următor, instrucțiunea **for** începe cu **i=0**, corpul instrucțiunii se repetă cît timp **i≤50** și contorul **i** este incrementat cu **2** la fiecare iterație. Vor fi afișate numerele pare ≤50.

Obs: Valoarea **increment** poate fi și negativă și comparația se poate realiza și cu orice alt operator de comparare.

```
1
2  <html>
3  <body>
4  <h3>Utilizarea instructiunii for</h3> <hr/>
5  <script>
6  document.write("Numerele pare mai mici sau egale cu 50: "+"<br/>");
7  var i=0;
8  for (i=0;i<=50;i+=2)
9  {
10 document.write(i + " ");
11 }
12 </script>
13 </body>
14 </html>
```

```
1
2  <html>
3  <body>
4  <h3>Utilizarea instructiunii for</h3> <hr/>
5  <script>
6  document.write("Numerele pare mai mici sau egale cu 50: "+"<br/>");
7  var i=0;
8  for (i=0;i<=50;i+=2)
9  {
10 document.write(i + " ");
11 }
12 </script>
13 </body>
14 </html>
```

Exemplu: În acest exemplu, instrucțiunea **for** este utilizată pentru a afișa cele **6** titluri HTML.

```
1
2  <html>
3  <body>
4  <h3>Afisarea titlurilor HTML cu instructiunea
```

```

4   for</h3> <hr />
5   <script>
6   for (i = 1; i <= 6; i++)
7   {
8   document.write("<h" + i + ">Aceste este un titlu " + i);
9   document.write("</h" + i + ">");
10  }
11  }
12  </script>
13  </body>
    </html>

```

```

1   <html>
2   <body>
3   <h3>Afisarea titlurilor HTML cu instructiunea
4   for</h3> <hr />
5   <script>
6   for (i = 1; i <= 6; i++)
7   {
8   document.write("<h" + i + ">Aceste este un titlu " + i);
9   document.write("</h" + i + ">");
10  }
11  }
12  </script>
13  </body>
    </html>

```

Instrucțiunea for...in

Această instrucțiune este utilizată pentru a parcurge elementele unui tablou sau a enumera proprietățile unui obiect.

Sintaxă:

```

1   for (variabila in obiect) {
2   cod ce trebuie executat
3   }

```

```

1   for (variabila in obiect) {
2   cod ce trebuie executat
3   }

```

Obs: Codul din corpul instrucțiunii este executat cîte o dată pentru fiecare element din tablou sau proprietate.

Obs: Argumentul variabila poate fi o variabilă, un element de tablou sau o proprietate a unui obiect.

Exemplu:

Instrucțiunea **for..in** este utilizată pentru a parcurge elementele unui tablou:

```

1   <html>
2   <body>
3   <h3>Parcurea elementelor unui tablou cu instructiunea for..in</h3> <hr />
4   <script>
5   var x;
6   var pets = new Array();
7   pets[0] = "Pisica";
8   pets[1] = "Caine";
9   pets[2] = "Papagal";
10  pets[3] = "Hamster";
11  document.write("Valorile memorate in tablou sunt:"+"<br />");
12  for (x in pets)
13  {
14  document.write(pets[x] + "<br />");
15  }
16  }

```

```

17 </script>
18 </body>
    </html>

1 <html>
2 <body>
3 <h3>Parcurea elementelor unui tablou cu instructiunea for..in</h3> <br/>
4 <script>
5 var x;
6 var pets = new Array();
7 pets[0] = "Pisica";
8 pets[1] = "Caine";
9 pets[2] = "Papagal";
10 pets[3] = "Hamster";
11 document.write("Valorile memorate in tablou sunt:"+"<br/>");
12 for (x in pets)
13 {
14     document.write(pets[x] + "<br />");
15 }
16 </script>
17 </body>
18 </html>

```

Exemplu:

Instrucțiunea **for..in** este utilizată pentru a parcurge proprietățile unui obiect:

```

1 <!DOCTYPE html>
2 <html>
3
4 <body>
5 <script>
6 var txt = "";
7 var persoana = {nume:"Ion", prenume:"Cheni", virsta:25};
8 var x;
9 for (x in persoana) {
10     txt += persoana[x] + " ";
11 }
12 document.write(txt);
13 </script>
14 </body>
15 </html>

1 <!DOCTYPE html>
2 <html>
3
4 <body>
5 <script>
6 var txt = "";
7 var persoana = {nume:"Ion", prenume:"Cheni", virsta:25};
8 var x;
9 for (x in persoana) {
10     txt += persoana[x] + " ";
11 }
12 document.write(txt);
13 </script>
14 </body>
15 </html>

```

Instrucțiunea while

Instrucțiunea execută în mod repetat o secvență de cod, cît timp o condiție este adevărată.

Sintaxă:

```

■

```

```

1
2 while (var<=val_finala) {
3   codul ce trebuie executat
4 }

1
2 while (var<=val_finala) {
3   codul ce trebuie executat
4 }

```

Obs: Operatorul `<=` poate fi înlocuit cu orice alt operator de comparare.

Exemplu:

```

1
2 <html>
3 <body>
4 <h3>Utilizarea instructiunii while</h3> <hr/>
5 <script>
6 document.write("Numerele naturale mai mici egale cu 5:"+"<br/>");
7 var i=0;
8 while (i<=5)
9 {
10  document.write(i);
11  document.write("<br/>");
12  i++;
13 }
14 </script>
15 </body>
16 </html>

```

```

1
2 <html>
3 <body>
4 <h3>Utilizarea instructiunii while</h3> <hr/>
5 <script>
6 document.write("Numerele naturale mai mici egale cu 5:"+"<br/>");
7 var i=0;
8 while (i<=5)
9 {
10  document.write(i);
11  document.write("<br/>");
12  i++;
13 }
14 </script>
15 </body>
16 </html>

```

Instrucțiunea do...while

Instrucțiunea **do...while** este o variantă a instrucțiunii **while**. Secvența de instrucțiuni va fi executată în mod sigur o dată, apoi în mod repetat, cît timp condiția specificată este adevărată.

Sintaxă:

```

1
2 do {
3   codul ce trebuie executat
4 }
5 while (var<=val_finala);

1
2 do {
3   codul ce trebuie executat
4 }

```



```
while (var<=val_finala);
```

Exemplu:

În acest exemplu vor fi afișate numerele impare ≤ 50 .

```

1  <html>
2  <body>
3  <h3>Utilizarea instructiunii do-while</h3> <hr/>
4  <script>
5  document.write("Numerele impare mai mici sau egale cu 50:"+"<br/>");
6  var i=1;
7  do
8  {
9  document.write(i + " ");
10 i+=2;
11 }
12 while (i<=50);
13 </script>
14 </body>
15 </html>

```

```

1  <html>
2  <body>
3  <h3>Utilizarea instructiunii do-while</h3> <hr/>
4  <script>
5  document.write("Numerele impare mai mici sau egale cu 50:"+"<br/>");
6  var i=1;
7  do
8  {
9  document.write(i + " ");
10 i+=2;
11 }
12 while (i<=50);
13 </script>
14 </body>
15 </html>

```

Exerciții:

1. Sunt date numerele **K** și **N** ($N > 0$). De afișat de **N** ori numărul **K**;
2. De elaborat un script ce va afișa tabla înmulțirii;
3. De elaborat un script ce va afișa tabla de șah;
4. De elaborat un script ce va calcula $n!$;
5. De elaborat un script ce va calcula numărul exact de ani, luni și zile de la ziua de naștere.
6. Să se scrie un program care să afișeze următoarele valori:

$$1 \cdot 9 + 2 = ?$$

$$12 \cdot 9 + 3 = ?$$

$$123 \cdot 9 + 4 = ?$$

.....

$$123456789 \cdot 9 + 10 = ?$$

7. Să se scrie un program care să afișeze următoarea "piramidă" de numere:

1

1 2 3

1 2 3 4 5

.....

1 $(2n-1)$

Instrucțiunile break și continue

Instrucțiunea break

Este utilizată pentru a întrerupe în mod forțat execuția unei bucle. În exemplul următor, bucla va fi întreruptă când **i=3**.

```

1  <html>
2  <body>
3  <h3>Utilizarea instrucțiunii break</h3> <br/>
4  <script>
5  document.write("Numerele naturale mai mici egale cu 5:"+"<br/>");
6  var i=0;
7  for (i=0;i<=10;i++)
8  {
9    if (i==3)
10   {
11     document.write("Ciclu oprit cu break"); break;
12   }
13   document.write(i);
14   document.write("<br />");
15 }
16 </script>
17 </body>
18 </html>

```

```

1  <html>
2  <body>
3  <h3>Utilizarea instrucțiunii break</h3> <br/>
4  <script>
5  document.write("Numerele naturale mai mici egale cu 5:"+"<br/>");
6  var i=0;
7  for (i=0;i<=10;i++)
8  {
9    if (i==3)
10   {
11     document.write("Ciclu oprit cu break"); break;
12   }
13   document.write(i);
14   document.write("<br />");
15 }
16 </script>
17 </body>
18 </html>

```

Instrucțiunea continue

Instrucțiunea întrerupe execuția iterației curente și sare la următoarea iterație. În exemplul următor, valoarea **3** nu va fi afișată (este sărită cu instrucțiunea **continue**).

```

1  <html>
2  <body>
3  <h3>Utilizarea instrucțiunii continue</h3> <br/>
4  <script>
5  document.write("Numerele naturale mai mici sau egale cu 10:"+"<br/>");
6  var i=0;
7  for (i=0;i<=10;i++)
8  {
9    if (i==3)
10   {
11     document.write("valoare sarita cu continue"+"<br/>");continue;
12   }
13   document.write(i);
14 }

```

```

15 document.write("<br />");
16 }
17 </script>
18 </body>
    </html>

1
2 <html>
3 <body>
4 <h3>Utilizarea instructiunii continue</h3> <hr/>
5 <script>
6 document.write("Numerele naturale mai mici sau egale cu 10:"+"<br/>");
7 var i=0
8 for (i=0;i<=10;i++)
9 {
10 if (i==3)
11 {
12 document.write("valoare sarita cu continue"+"<br/>");continue;
13 }
14 document.write(i);
15 document.write("<br />");
16 }
17 </script>
18 </body>
    </html>

```

Instrucțiunea with

Instrucțiunea **with** simplifică scrierea programelor JavaScript sau reduce pe cât posibil cantitatea de cod JavaScript. Instrucțiunea **with** permite specificarea unui obiect și este urmat de un bloc de instrucțiuni plasat între acolade. Pentru fiecare din instrucțiunile blocului, proprietățile menționate fără ca obiectul corespunzător să fie indicat se referă la obiectul specificat prin **with**.

Sintaxă:

```

with (obiect) {
  cod JavaScript
}

```

Exemplu 1:

În următorul exemplu este apelată metoda **document.write()** de patru ori

```

1
2 <html>
3 <head></head>
4 <body>
5 <script>
6 document.write("Pe trotuar, alături saltă"+"<br />");
7 document.write("Două fete vesele,"+"<br />");
8 document.write("Zău că-mi vine să las baltă,"+"<br />");
9 document.write("Toate interesele"+"<br />");
10 </script>
11 </body>
    </html>

1
2 <html>
3 <head></head>
4 <body>
5 <script>
6 document.write("Pe trotuar, alături saltă"+"<br />");
7 document.write("Două fete vesele,"+"<br />");
8 document.write("Zău că-mi vine să las baltă,"+"<br />");
9 document.write("Toate interesele"+"<br />");
10 </script>
    </body>

```

```
11      </html>
```

Exemplu 2:

Acelaș exemplu, numai că cuvântul cheie **with** elimină referințele multiple la obiectul **document**

```
1  <html>
2  <head></head>
3  <body>
4  <script>
5    with(document){
6      write("Pe trotuar, alături saltă"<br />");
7      write("Două fete vesele,"<br />");
8      write("Zău că-mi vine să las baltă,"<br />");
9      write("Toate interesele"<br />");
10   }
11 }
12 </script>
13 </body>
14 </html>
```

```
1  <html>
2  <head></head>
3  <body>
4  <script>
5    with(document){
6      write("Pe trotuar, alături saltă"<br />");
7      write("Două fete vesele,"<br />");
8      write("Zău că-mi vine să las baltă,"<br />");
9      write("Toate interesele"<br />");
10   }
11 }
12 </script>
13 </body>
14 </html>
```

Obs: Pentru un cod scurt interesul utilizării instrucțiunii **with** nu este evident, dar când trebuie să accesăm același obiect în cadrul unei proceduri sau când utilizăm un obiect predefinit precum **Math**, **with** vă ajută să câștigați foarte mult timp.

Funcții

O funcție va fi executată când are loc un eveniment sau când este apelată. Dacă doriți ca browserul să nu execute un script atunci când pagina se încarcă, puteți scrie scriptul într-o funcție. O funcție poate fi apelată din orice punct al paginii, sau chiar din alte pagini, dacă funcția este scrisă într-un fișier JS extern. Funcțiile JS pot fi scrise în secțiunea **<head>** sau în secțiunea **<body>** a documentului. Totuși, pentru a fi siguri că funcția este încărcată în browser înainte de a fi apelată, este recomandat să o scrieți în secțiunea **<head>**.

Definirea unei funcții

Sintaxă:

```
1
2  function nume_funcție(var1,var2,...,varX)
3  {
4    codul funcției
5  }
```

```
1
2  function nume_funcție(var1,var2,...,varX)
3  {
4    codul funcției
```

}

Parametrii **var1**, **var2**, etc. sunt variabile sau valori transmise funcției. Acoladele marchează începutul și sfârșitul corpului funcției.

Obs: Chiar dacă funcția nu are parametri, parantezele rotunde de după numele funcției trebuie să fie prezente.

Exemplu:

```

1  |
2  | <html>
3  | <head>
4  | <script>
5  | function afiseaza_mesaj()
6  | {
7  |   alert("Bine ati venit!");
8  | }
9  | </script>
10 | </head>
11 | <body>
12 | <h3>La apasarea butonului este apelata o functie JS care afiseaza caseta alert</h3> <hr/>
13 | <form>
14 | <input type="button" value="Apasati!" onclick="afiseaza_mesaj()" />
15 | </form>
16 | </body>
   | </html>

```

```

1  |
2  | <html>
3  | <head>
4  | <script>
5  | function afiseaza_mesaj()
6  | {
7  |   alert("Bine ati venit!");
8  | }
9  | </script>
10 | </head>
11 | <body>
12 | <h3>La apasarea butonului este apelata o functie JS care afiseaza caseta alert</h3> <hr/>
13 | <form>
14 | <input type="button" value="Apasati!" onclick="afiseaza_mesaj()" />
15 | </form>
16 | </body>
   | </html>

```

Dacă linia de cod **alert("Bine ati venit!")** din exemplul anterior nu ar fi fost scrisă în corpul unei funcții, codul ar fi fost executat imediat ce linia respectivă ar fi fost încărcată în browser. Deoarece codul a fost inclus într-o funcție, el nu va fi executat decit atunci cînd utilizatorul acționează butonul și este apelată funcția **afiseaza_mesaj()**.

Instrucțiunea return

Instrucțiunea **return** este folosită pentru a specifica valoarea returnată de o funcție și trebuie inclusă în orice funcție care returnează o valoare.

În exemplul următor, funcția **suma** returnează suma celor doi parametri de intrare:

```

1  |
2  | <html>
3  | <head>
4  | <script>
5  | function suma(a,b)
6  | {
7  |   return a+b;
8  | }
9  | </script>

```

```

10 </head>
11 <body>
12 <h3>Suma urmatoare este calculata și returnata de o functie</h3> <hr/>
13 <script>
14 document.write("7+9="+suma(7,9));
15 </script>
16 </body>
   </html>

```

```

1 <html>
2 <head>
3 <script>
4 function suma(a,b)
5 {
6   return a+b;
7 }
8 </script>
9 </head>
10 <body>
11 <h3>Suma urmatoare este calculata și returnata de o functie</h3> <hr/>
12 <script>
13 document.write("7+9="+suma(7,9));
14 </script>
15 </body>
16 </html>

```

Durata de viață a variabilelor JavaScript

Dacă declarați o variabilă în interiorul unei funcții, ea poate fi accesată numai din interiorul funcției. Când funcția se încheie, variabila este distrusă. Variabilele declarate în corpul unei funcții se numesc **variabile locale**. Puteți avea variabile locale cu același nume în funcții diferite, deoarece fiecare variabilă locală este recunoscută numai în interiorul funcției în care este declarată. Dacă declarați o variabilă în afara tuturor funcțiilor (**variabilă globală**), ea poate fi accesată de toate funcțiile din pagină. O **variabilă globală** este distrusă numai atunci când pagina este închisă.

Example:

Exemplul 1

Ilustrează cum se poate transmite o variabilă unei funcții și cum poate fi folosită respectiva variabilă în corpul funcției.

```

1 <html>
2 <head>
3 <script>
4 function functia_1(text)
5 {
6   alert(text);
7 }
8 </script>
9 </head>
10 <body>
11 <h3>Functii JavaScript</h3> <hr/>
12 <form>
13 <input type="button" onclick="functia_1('Bune ati venit!')" value="Apasati">
14 </form>
15 <p>Cind apasati butonul, va fi apelata o functie cu textul "Bine ati venit!" drept parametru.
16 Functia va afisa parametrul cu o caseta de alertare.</p>
17 </body>
   </html>

```

```

1 <html>
2 <head>
3 <script>

```

```

5   function functie_1(text)
6   {
7   alert(text);
8   }
9   </script>
10  </head>
11  <body>
12  <h3>Functii JavaScript</h3> <hr/>
13  <form>
14  <input type="button" onclick="functie_1('Bune ati venit!')" value="Apasati">
15  </form>
16  <p>Cind apasati butonul, va fi apelata o functie cu textul "Bine ati venit!" drept parametru.
17  Functia va afisa parametrul cu o caseta de alertare.</p>
    </body>
    </html>

```

Exemplul 2

Ilustrează cum poate fi folosit rezultatul returnat de o funcție.

```

1   <html>
2   <head>
3   <script>
4   function functie_2()
5   {
6   {
7   return ("Bine ati venit!");
8   }
9   </script>
10  </head>
11  <body>
12  <h3>Textul urmator este returnat de o functie apelata direct din document.write()</h3> <hr/>
13  <script>
14  document.write(functie_2())
15  </script>
16  </body>
    </html>

```

```

1   <html>
2   <head>
3   <script>
4   function functie_2()
5   {
6   {
7   return ("Bine ati venit!");
8   }
9   </script>
10  </head>
11  <body>
12  <h3>Textul urmator este returnat de o functie apelata direct din document.write()</h3> <hr/>
13  <script>
14  document.write(functie_2())
15  </script>
16  </body>
    </html>

```

Exemplul 3

```

1   <html>
2   <head>
3   <script>
4   function salut(txt)
5   {
6   {
7   alert(txt);
8   }
9   </script>
10  </head>

```

```

11 <body>
12 <h3>Utilizarea functiilor JavaScript</h3> <hr/>
13 <form>
14 <input type="button" onclick="salut('Buna dimineata!')" value="Dimineata">
15 <input type="button" onclick="salut('Buna seara!')" value="Seara">
16 </form>
17 <p>Cind apasati unul dintre butoane, va fi apelata o functie care afiseaza mesajul primit ca
18 parametru.</p>
</body>
</html>

1 <html>
2 <head>
3 <script>
4 function salut(txt)
5 {
6   alert(txt);
7 }
8 </script>
9 </head>
10 <body>
11 <h3>Utilizarea functiilor JavaScript</h3> <hr/>
12 <form>
13 <input type="button" onclick="salut('Buna dimineata!')" value="Dimineata">
14 <input type="button" onclick="salut('Buna seara!')" value="Seara">
15 </form>
16 <p>Cind apasati unul dintre butoane, va fi apelata o functie care afiseaza mesajul primit ca
17 parametru.</p>
18 </body>
</html>

```

Evenimentele JavaScript. Conceptul de eveniment și gestionar de evenimente

Un eveniment este o acțiune care se produce în raport cu un element (fereastră, document, buton, etc.) el poate fi detectat și prelucrat de către un script care va declanșa o acțiune. Script-ul este executat dacă evenimentul se produce pe obiectul căruia îi este asociat.

Reacția la un eveniment este cunoscută sub numele de prelucrarea evenimentului, iar codul JavaScript corespunzător este cunoscut sub numele de gestionar de evenimente.

Un gestionar de evenimente este o metodă puțin specială care va fi apelată în mod automat de către browser ori de câte ori va surveni un eveniment particular.

Gestionarii de evenimente sunt funcții JavaScript. Aceștia sunt ușor de programat, de multe ori este suficientă o singură instrucțiune pentru a putea fi creați.

Acțiunile utilizatorilor sunt cele mai frecvente evenimente. Însă acestea nu sunt singurele. De exemplu evenimentul **load** este declanșat automat de către browser atunci când este încheiată încărcarea unui document.

De obicei evenimentele sunt provocate de utilizator prin efectuarea unui click sau introducerea unor date.

Fiecare eveniment are un nume prestabilit care în același timp este și parametrul unui tag. Acest parametru stabilește care eveniment anume trebuie procesat - glisarea mouse-ului, introducerea textului, încărcarea paginii, etc. Valoarea parametrului este acțiunea care trebuie îndeplinită:

<tag gestionar_evenimente="cod JavaScript (descrierea acțiunii)"> conținut </tag>

Petru a defini un gestionar de evenimente se adaugă prefixul **on** la numele evenimentului.

Toate evenimentele procesate de browser pot fi împărțite în patru grupuri în dependență de elementul ce le provoacă: evenimente provocate de mouse, de tastatură, de elementul formularului, și de ferestre.

Evenimente provocate de mouse

onclick - are loc cand se da click pe un element

ondblclick - are loc cand se da dublu click pe un element

onmousedown - are loc cand este apasat click-ul

onmousemove - are loc cand se misca cursorul în interiorul unui element

onmouseover - are loc cand se pune cursorul deasupra unui element

onmouseout - are loc cand cursorul iese din cadrul unui element

onmouseup - are loc cand se elibereaza click-ul

Example:

Exemplul 1

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4    <input type="button" value="onclick" onclick="alert('eveniment la click')" />
5    <p>
6    <input type="button" value="ondblclick" ondblclick="alert('eveniment la dublu click')" />
7    <p>
8    <input type="button" value="onmouseout" onmouseout="alert('eveniment la mouseout')" />
9    <p>
10   <input type="button" value="onmouseup" onmouseup="alert('eveniment la mouseup')" />
11   <p>
12   <input type="button" value="onmouseover" onmouseover="alert('eveniment la mouseover')" />
13 </body>
14 </html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <body>
4    <input type="button" value="onclick" onclick="alert('eveniment la click')" />
5    <p>
6    <input type="button" value="ondblclick" ondblclick="alert('eveniment la dublu click')" />
7    <p>
8    <input type="button" value="onmouseout" onmouseout="alert('eveniment la mouseout')" />
9    <p>
10   <input type="button" value="onmouseup" onmouseup="alert('eveniment la mouseup')" />
11   <p>
12   <input type="button" value="onmouseover" onmouseover="alert('eveniment la mouseover')" />
13 </body>
14 </html>

```

Exemplul 2

În exemplul de mai jos, am utilizat o functie care permite afișarea coordonatelor cursorului unui mouse în mișcare.

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4    <title>Coordonatele cursorului</title>
5  </head>
6  <body>
7    <form name=formular> Coordonatele cursorului<br /><br />
8      X <input type="text" name=x value=""><br />
9      Y <input type="text" name=y value="">
10    </form>
11    <script>
12      var InternetExplorer = document.all?true:false;
13      if (!InternetExplorer)
14        document.captureEvents(Event.mousemove);
15        document.onmousemove = pozitie;
16        function pozitie(poz)
17        {
18          var x;

```

```

20     var y;
21     if (!InternetExplorer)
22     {
23     x = poz.pageX;
24     y = poz.pageY;
25     }
26     if (InternetExplorer)
27     {
28     x = event.clientX + document.body.scrollLeft;
29     y = event.clientY + document.body.scrollTop;
30     }
31     document.formular.x.value=x;
32     document.formular.y.value=y;
33     return true; }
34 </script>
35 </body>
</html>

```

```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>Coordonatele cursorului</title>
5  </head>
6  <body>
7  <form name=formular> Coordonatele cursorului<br /><br />
8  X <input type=text name=x value=""><br />
9  Y <input type=text name=y value="">
10 </form>
11 <script>
12 var InternetExplorer = document.all?true:false;
13 if (!InternetExplorer)
14 document.captureEvents(Event.mousemove);
15 document.onmousemove = pozitie;
16 function pozitie(poz)
17 {
18     var x;
19     var y;
20     if (!InternetExplorer)
21     {
22     x = poz.pageX;
23     y = poz.pageY;
24     }
25     if (InternetExplorer)
26     {
27     x = event.clientX + document.body.scrollLeft;
28     y = event.clientY + document.body.scrollTop;
29     }
30     document.formular.x.value=x;
31     document.formular.y.value=y;
32     return true; }
33 </script>
34 </body>
35 </html>

```

Evenimente provocate de tastatură

onkeydown - are loc cand se apasa o tasta

onkeypress - are loc cand este apasata o tasta

onkeyup - are loc cand este eliberata o tasta

Example:

Exemplul 1

```

1  <!DOCTYPE html>
2  <html><body>
3  <input type="text" onkeydown="alert('eveniment la onkeydown')" />
4  <p>

```

```

6  <input type="text" onkeyup="alert('eveniment la onkeyup')" />
7  <p>
8  <input type="text" onkeypress="alert('eveniment la onkeypress')" />
9  </body>
   </html>

1  <!DOCTYPE html>
2  <html><body>
3  <input type="text" onkeydown="alert('eveniment la onkeydown')" />
4  <p>
5  <input type="text" onkeyup="alert('eveniment la onkeyup')" />
6  <p>
7  <input type="text" onkeypress="alert('eveniment la onkeypress')" />
8  </body>
9  </html>

```

Exemplul 2

```

1  <!DOCTYPE html>
2  <html>
3  <head><title>Evenimente</title>
4  </head>
5  <body>
6  <form>
7  <input type="button" value="apasati orice tasta" onKeyPress="window.alert('Codul tastei apasate
8  este: ' +window.event.keyCode)">
9  </form>
10 </body>
   </html>

1  <!DOCTYPE html>
2  <html>
3  <head><title>Evenimente</title>
4  </head>
5  <body>
6  <form>
7  <input type="button" value="apasati orice tasta" onKeyPress="window.alert('Codul tastei apasate
8  este: ' +window.event.keyCode)">
9  </form>
10 </body>
   </html>

```

Exemplul 3

```

1  <!DOCTYPE html>
2  <html>
3  <head><title>Evenimente</title>
4  </head>
5  <script>
6  function urmatorul(element,text) {
7  if (text.length==element.maxLength) {
8  next=element.tabIndex;
9  if (next<document.form3.elements.length) {
10 document.form3.elements[next].focus();
11 }
12 }
13 }
14 }
15 </script>
16 </head>
17 <body>
18 - Cursorul se muta automat in urmatoarea caseta atunci cand s-a completat tot campul
19 precedent (cand se ajunge la lungimea stabilita a campului, aici 4 caractere) <br>
20 <form name="form3">

```

```

21 <input size="4" tabindex="1" name="field" maxlength="4" onkeyup="urmatorul(this,this.value)">
22 <input size="4" tabindex="2" maxlength="4" onkeyup="urmatorul(this,this.value)">
23 <input size="4" tabindex="4" maxlength="4" onkeyup="urmatorul(this,this.value)">
24 </form>
25 </body>
    </html>

1
2 <!DOCTYPE html>
3 <html>
4 <head><title>Evenimente</title>
5 </head>
6 <script>
7 function urmatorul(element,text) {
8   if (text.length==element.maxLength) {
9     next=element.tabIndex;
10    if (next<document.form3.elements.length) {
11      document.form3.elements[next].focus();
12    }
13  }
14 }
15 </script>
16 </head>
17 <body>
18 - Cursorul se muta automat in urmatoarea caseta atunci cand s-a completat tot campul
19 precedent (cand se ajunge la lungimea stabilita a campului, aici 4 caractere) <br>
20 <form name="form3">
21 <input size="4" tabindex="1" name="field" maxlength="4" onkeyup="urmatorul(this,this.value)">
22 <input size="4" tabindex="2" maxlength="4" onkeyup="urmatorul(this,this.value)">
23 <input size="4" tabindex="4" maxlength="4" onkeyup="urmatorul(this,this.value)">
24 </form>
25 </body>
    </html>

```

Funcția **urmatorul(element,text)**, are doua argumente: **element** (care, prin apel, se referă la caseta **input**) și **text** (se referă la valoarea casetei). În apelul funcției: **urmatorul(this,this.value)**, **this** se referă la obiectul curent (reprezintă primul argument) iar al doilea argument **this.value** reprezintă valoarea obiectului curent (valoarea din câmp). Observați folosirea evenimentului **onKeyUp**, pentru a se trece la urmatorul câmp atunci când s-a completat ultimul caracter din caseta precedentă.

Pentru a înțelege mai bine puteți să înlocuiți **onkeyup** cu **onkeydown** pentru a vedea diferența.

Evenimente provocate de ferestre

onabort - are loc în momentul în care se se întrerupe încărcarea ferestrei

onerror - are loc în momentul în care o imagine nu se încarcă corect (pentru **object**, **body** și **frameset**)

onload - are loc în momentul în care se termină încărcarea ferestrei

onresize - are loc când este modificată dimensiunea ferestrei

onscroll - are loc în momentul în care se derulează bara de scroll

onunload - are loc în momentul în care nu se încarcă fereastra

Exemplu

```

1
2 <!DOCTYPE html>
3 <html>
4 <body>
5 <script>
6   function g()
7   {
8     alert("Salut!");
9   }
10  </script>
11  </head>
12  <body onLoad="g()">
13 </script>

```

```

14 | </body>
    | </html>

1 |
2 | <!DOCTYPE html>
3 | <html>
4 | <body>
5 | <script>
6 |     function g()
7 |     {
8 |         alert("Salut!");
9 |     }
10 | </script>
11 | </head>
12 | <body onLoad="g()">
13 | </script>
14 | </body>
    | </html>

```

Evenimente provocate de formular

onblur - are loc în momentul în care cursorul este scos din casuta respectiva (se dă click în alta parte)

onchange - are loc în momentul în care se schimbă conținutul din casuța respectivă

onfocus - are loc în momentul în care se pune cursorul în casuța respectivă

onreset - are loc când este apasat butonul reset din formular

onselect - are loc în momentul în care se selectează un text din câmpul respectiv

onsubmit - are loc în momentul în care se apasă butonul de **submit** din formular

Example:

Exemplul 1

În exemplul de mai jos, am utilizat o funcție care preia o valoare introdusă în formular, o verifică și afișează un mesaj corespunzător. Dacă nu a fost introdus un număr, funcția **isNaN()** va returna valoarea **true**, iar pe monitor va apărea mesajul **Introduceți un număr!**. Mesajele afișate vor apare pe ecran numai dacă introducem altceva decât un număr pozitiv. Evenimentul **onBlur** își face efectul, adică transmite valoarea introdusă de către utilizator, funcției **verificare**, dacă dăm click în afara zonei de text a formularului.

```

1 |
2 | <!DOCTYPE html>
3 | <html>
4 | <head>
5 | <title> onBlur </title>
6 | <script>
7 | function verificare(form)
8 | {
9 |     var x=0;
10 |     x=document.formular.date.value;
11 |     y=isNaN(x);
12 |     if (y==true)
13 |         alert("Introduceți un numar!");
14 |     else
15 |         if (x<0)
16 |         {
17 |             alert("Va rog sa introduceți un numar pozitiv!");
18 |         }
19 |     }
20 | </script>
21 | </head>
22 | <body>
23 | <h3>Introduceți un numar pozitiv:</h3>
24 | <p>Pentru verificare, puteti introduce altceva
25 | <br />
26 | <form name="formular">
27 | <input type="text" name="date" value="" size="20" onBlur='verificare(this.form)'+
28 | </form>

```

```

29 | </body>
    | </html>

1 |
2 | <!DOCTYPE html>
3 | <html>
4 | <head>
5 | <title> onBlur </title>
6 | <script>
7 | function verificare(form)
8 | {
9 |   var x=0;
10 | x=document.formular.date.value;
11 | y=isNaN(x);
12 | if (y==true)
13 | alert("Introduceti un numar!");
14 | else
15 |   if (x<0)
16 |   {
17 |     alert("Va rog sa introduceti un numar pozitiv!");
18 |   }
19 | }
20 | </script>
21 | </head>
22 | <body>
23 | <h3>Introduceti un numar pozitiv:</h3>
24 | <p>Pentru verificare, puteti introduce altceva
25 | <br />
26 | <form name="formular">
27 | <input type="text" name="date" value="" size="20" onBlur='verificare(this.form)'+
28 | </form>
29 | </body>
    | </html>

```

Exemplul 2

În exemplul de mai jos, am utilizat o functie care preia șirul de caractere introdus în formular și il transformă în șir scris cu majuscule.

```

1 |
2 | <!DOCTYPE html>
3 | <html>
4 | <head><title>onChange</title>
5 | <script>
6 | function majuscule(a)
7 | {
8 |   a.value = a.value.toUpperCase();
9 | }
10 | </script>
11 | </head>
12 | <body>
13 | <h3>Introduceti numele și prenumele:</h3>
14 | <form name="formular">
15 | <table>
16 | <tr>
17 | <td>Numele:</td>
18 | <td><input type="text" name="x" value="" size="20" onChange="majuscule(this)"></td>
19 | </tr>
20 | <tr>
21 | <td>Prenumele:</td>
22 | <td><input type="text" name="y" value="" size="20" onChange="majuscule(this)"></td>
23 | </tr>
24 | </table>
25 | </form>
26 | </body>
    | </html>

```

```

1 |
2 | <!DOCTYPE html>
3 | <html>

```

```

4   <head><title>onChange</title>
5   <script>
6   function majuscule(a)
7   {
8   a.value = a.value.toUpperCase();
9   }
10  </script>
11  </head>
12  <body>
13  <h3>Introduceti numele și prenumele:</h3>
14  <form name="formular">
15  <table>
16  <tr>
17  <td>Numele:</td>
18  <td><input type="text" name="x" value="" size="20" onChange="majuscule(this)"></td>
19  </tr>
20  <tr>
21  <td>Prenumele:</td>
22  <td><input type="text" name="y" value="" size="20" onChange="majuscule(this)"></td>
23  </tr>
24  </table>
25  </form>
26  </body>
   </html>

```

Exemplul 3

În următorul exemplu, am utilizat o funcție care afișează un mesaj, în cazul în care selectăm caracterele scrise într-un câmp de tip text.

```

1   <!DOCTYPE html>
2
3
4
5   <html>
6   <head>
7   <title>onSelect</title>
8   </head>
9   <script>
10  function f()
11  {
12  alert("Felicitari! \n Ati selectat textul.");
13  }
14  </script>
15  <body>
16  <h3>Selectati textul de mai jos!</h3>
17  <form name="formular">
18  <input type="text" name="data" value="Textul de mai jos :)" size=20 onSelect='f()>
19  </form>
20  </body>
   </html>

```

```

1   <!DOCTYPE html>
2
3
4
5   <html>
6   <head>
7   <title>onSelect</title>
8   </head>
9   <script>
10  function f()
11  {
12  alert("Felicitari! \n Ati selectat textul.");
13  }
14  </script>
15  <body>
16  <h3>Selectati textul de mai jos!</h3>
17  <form name="formular">
18  <input type="text" name="data" value="Textul de mai jos :)" size=20 onSelect='f()>

```

```

19 </form>
20 </body>
    </html>

```

Exemplul 4

În exemplul de mai jos am creat un formular ce permite calcularea unei sume, pe măsură ce sunt introduse cifrele în câmpurile formularului.

```

1 <!DOCTYPE html>
2 <html>
3 <title>Exemplu de formular</title>
4 <style>
5 body {
6 background-color: #B3FFB3;
7 }
8 .titlul { font-family: "Aldine721 Lt BT", Adolescence, "Alexei Copperplate";
9 font-size: 18px;
10 font-weight: bolder;
11 color: #006600;
12 }
13 </style>
14 <script>
15 function PornireCalculare()
16 {
17 interval = setInterval("Calculare()",1);
18 }
19 function Calculare()
20 {
21 primul = document.adunare.x1.value;
22 alDoilea = document.adunare.x2.value;
23 document.adunare.x3.value = (primul * 1) + (alDoilea * 1);
24 }
25 function Oprire()
26 {
27 clearInterval(interval);
28 }
29 </script>
30 </head>
31 <body>
32 <p class="titlul">Introduceti numere, pentru a le aduna
33 </p><br />
34 <form name="adunare">
35 <input type="text" name="x1" value="" onFocus="PornireCalculare();" onBlur="Oprire();" size="10">
36 +
37 <input type="text" name="x2" value="" onFocus="PornireCalculare();" onBlur="Oprire();" size="10">
38 =
39 <input type="text" name="x3" size="10">
40 </form>
    </body>
</html>

```

```

1 <!DOCTYPE html>
2 <html>
3 <title>Exemplu de formular</title>
4 <style>
5 body {
6 background-color: #B3FFB3;
7 }
8 .titlul { font-family: "Aldine721 Lt BT", Adolescence, "Alexei Copperplate";
9 font-size: 18px;
10 font-weight: bolder;
11 color: #006600;
12 }
13 </style>
14 <script>
15 function PornireCalculare()
16 {
17

```



```

18 interval = setInterval("Calculare()",1);
19 }
20 function Calculare()
21 {
22     primul = document.adunare.x1.value;
23     alDoilea = document.adunare.x2.value;
24     document.adunare.x3.value = (primul * 1) + (alDoilea * 1);
25 }
26 function Oprire()
27 {
28     clearInterval(interval);
29 }
30 </script>
31 </head>
32 <body>
33 <p class="titlul">Introduceti numere, pentru a le aduna
34 </p><br />
35 <form name="adunare">
36 <input type="text" name="x1" value="" onFocus="PornireCalculare();" onBlur="Oprire();" size="10">
37 +
38 <input type="text" name="x2" value="" onFocus="PornireCalculare();" onBlur="Oprire();" size="10">
39 =
40 <input type="text" name="x3" size="10">
    </form>
    </body>
    </html>

```

Exemplul 5

În exemplul de mai jos am creat un formular ce permite transmiterea unui comentariu. La activarea butonului de expediere a datelor apare o caseta de alertare.

```

1
2 <!DOCTYPE html>
3 <html>
4 <title>Evenimente</title>
5 <body>
6 <form onSubmit="window.alert('Multumim! Comentariul Dvs va fi analizat!')">
7 Comentarii:
8 <textarea cols=25 rows=5>
9 </textarea>
10 <input type="submit" value="ok">
11 </form>
12 </body>
    </html>

1
2 <!DOCTYPE html>
3 <html>
4 <title>Evenimente</title>
5 <body>
6 <form onSubmit="window.alert('Multumim! Comentariul Dvs va fi analizat!')">
7 Comentarii:
8 <textarea cols=25 rows=5>
9 </textarea>
10 <input type="submit" value="ok">
11 </form>
12 </body>
    </html>

```

Obiectele JavaScript

JavaScript este un limbaj de programare orientat pe obiecte (POO). Un limbaj POO vă permite să vă definiți propriile obiecte și propriile tipuri de variabile.

Obiectele JavaScript sunt de **trei** tipuri:

- **Obiecte interne(integrate)** - fac parte din limbajul JavaScript și anume **Array**, **Boolean**, **Date**, **Function**, **Math**, **Number**, **Object**, **RegExp**, **String** și **Arguments**.
- **Obiectele navigatorului** - nu fac parte din limbajul JavaScript, dar sunt recunoscute de navigatoare.
- **Obiecte personalizate** - obiecte create de utilizator.

Crearea propriilor obiecte va fi explicată mai târziu. Vom începe prin a examina obiectele încorporate în JavaScript și cum sunt ele utilizate.

Trebuie de reținut că un **obiect** este de fapt, un tip special de date care are **proprietăți** și **metode**.

Proprietăți

Proprietățile sunt valori asociate cu un obiect.

În exemplul următor, utilizăm proprietatea **length** a obiectului **String** (șir de caractere) pentru a determina numărul de caractere memorate într-un șir:

```
1
2 <html>
3 <head></head>
4 <body>
5 <script>
6 var txt="Bine ati venit!";
7 document.write(txt.length);
8 </body>
   </html>
```

```
1
2 <html>
3 <head></head>
4 <body>
5 <script>
6 var txt="Bine ati venit!";
7 document.write(txt.length);
8 </body>
   </html>
```

Codul de mai sus va afișa valoarea: **15**

Metode

Metodele sunt acțiuni ce pot fi realizate cu un obiect.

În exemplul următor, utilizăm metoda **UpperCase()** a obiectului **String** pentru a afișa un text cu litere mari:

```
1
2 <html>
3 <head></head>
4 <body>
5 <script>
6 var txt="Bine ati venit!";
7 document.write(txt.toUpperCase());
8 </body>
   </html>
```

```
1
2 <html>
3 <head></head>
4 <body>
5 <script>
6 var txt="Bine ati venit!";
7 document.write(txt.toUpperCase());
8 </body>
   </html>
```

Codul de mai sus va afișa șirul: BINE ATI VENIT!

Obiectul String

Obiectul **String** este folosit pentru a manipula secvențe de caractere (text). Un obiect **String** este creat cu instrucțiunea **new String()**.

Sintaxa:

var txt = new String(string);

sau mai simplu:

var txt = string;

Proprietățile obiectului String

Proprietate	Descriere
constructor	Returnează funcția care a creat prototipul obiectului String
length	Returnează lungimea șirului
prototype	Permite adăugarea de proprietăți și metode unui obiec

Metodele obiectului String

Metodă	Descriere
charAt()	Returnează caracterul cu indexul specificat
charCodeAt()	Returnează codul Unicode al caracterului cu indexul specificat
concat()	Concatenează două sau mai multe șiruri și returnează șirul obținut
fromCharCode()	Convertește valori Unicode în caractere
indexOf()	Returnează poziția primei apariții a unui subșir într-un și
lastIndexOf()	Returnează poziția ultimei apariții a unui subșir ontr-un șir
match()	Caută potrivirile dintre un subșir și un string și returnează subșirul sau null (dacă subșirul nu este găsit)
replace()	Caută toate aparițiile unui subșir într-un șir și le înlocuiește cu un nou subșir
search()	Caută potrivirea dintre un subșir și un șir și returnează poziția în care apare potrivirea
slice()	Elimină o porțiune din șir și returnează șirul extras
split()	Imparte un șir în subșiruri pe baza unui caracter separator
substr()	Extrage dintr-un șir secvența de caractere care începe într-o anumită poziție și are o anumită lungime
substring()	Extrage dintr-un șir caracterele situate între două poziții
toLowerCase()	Convertește un șir în litere mici

toUpperCase()	Convertește un șir în litere mari
valueOf()	Returnează valoarea primară a unui obiect String

Metode împachetate în taguri HTML

Aceste metode returnează șirul împachetat în tagurile HTML potrivite.

Metodă	Descriere
anchor()	Creează o ancoră
big()	Afișează șirul cu font mare
blink()	Afișează un șir care clipește
bold()	Afișează șirul cu font bold
fixed()	Afișează șirul cu un font cu pas fix
fontcolor()	Afișează șirul folosind o anumită culoare
fontsize()	Afișează șirul cu o anumită dimensiune a fontului
italics()	Afișează șirul cu font italic
link()	Afișează șirul ca hiperlegătură
small()	Afișează șirul cu font mic
strike()	Afișează șirul ca tăiat
sub()	Afișează șirul ca subscript (indice)
sup()	Afișează șirul ca superscript (exponent)

Exemple:

Exemplul 1

Ilustrează utilizarea proprietății **length** pentru a determina lungimea unui șir.

```
1  <html>
2  <body>
3  <h3>Obiectul String. Determinarea lungimii unui sir</h3> <hr/>
4  <script>
5  var txt="Bine ati venit!";
6  document.write("Sirul este: "+txt+"<br/>");
7  document.write("Are lungimea "+txt.length);
8  </script>
9  <p><b>Obs.</b>Sirul nu se modifica.</p>
10 </body>
11 </html>
```

```
1  <html>
2  <body>
3  <h3>Obiectul String. Determinarea lungimii unui sir</h3> <hr/>
4  <script>
5  var txt="Bine ati venit!";
6  document.write("Sirul este: "+txt+"<br/>");
7  document.write("Are lungimea "+txt.length);
8  </script>
```

```

10 <p><b>Obs.</b>Sirul nu se modifica.</p>
11 </body>
    </html>

```

Exemplul 2

Ilustrează cum se utilizează tagurile HTML pentru a stiliza un șir.

```

1  <html>
2  <body>
3  <h3>Obiectul String. Utilizarea tagurilor HTML pentru stilizarea unui sir.</h3> <hr/>
4  <script>
5  var txt="Bine ati venit!";
6
7  document.write("<p>Big: " + txt.big() + "</p>");
8  document.write("<p>Small: " + txt.small() + "</p>");
9
10 document.write("<p>Bold: " + txt.bold() + "</p>");
11 document.write("<p>Italic: " + txt.italics() + "</p>");
12
13 document.write("<p>Blink: " + txt.blink() + " (nu functioneaza in IE, Chrome, Safari)</p>");
14 document.write("<p>Fixed: " + txt.fixed() + "</p>");
15 document.write("<p>Strike: " + txt.strike() + "</p>");
16
17 document.write("<p>Fontcolor: " + txt.fontcolor("Blue") + "</p>");
18 document.write("<p>Fontsize: " + txt.fontsize(14) + "</p>");
19
20 document.write("<p>Subscript: " + txt.sub() + "</p>");
21 document.write("<p>Superscript: " + txt.sup() + "</p>");
22
23 document.write("<p>Link: " + txt.link("http://www.google.com") + "</p>");
24 </script>
25 <br/> <br/>
26 <p><b>Obs.</b>Sirul stilizat nu se modifica!</p>
27 </body>
28 </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul String. Utilizarea tagurilor HTML pentru stilizarea unui sir.</h3> <hr/>
4  <script>
5  var txt="Bine ati venit!";
6
7  document.write("<p>Big: " + txt.big() + "</p>");
8  document.write("<p>Small: " + txt.small() + "</p>");
9
10 document.write("<p>Bold: " + txt.bold() + "</p>");
11 document.write("<p>Italic: " + txt.italics() + "</p>");
12
13 document.write("<p>Blink: " + txt.blink() + " (nu functioneaza in IE, Chrome, Safari)</p>");
14 document.write("<p>Fixed: " + txt.fixed() + "</p>");
15 document.write("<p>Strike: " + txt.strike() + "</p>");
16
17 document.write("<p>Fontcolor: " + txt.fontcolor("Blue") + "</p>");
18 document.write("<p>Fontsize: " + txt.fontsize(14) + "</p>");
19
20 document.write("<p>Subscript: " + txt.sub() + "</p>");
21 document.write("<p>Superscript: " + txt.sup() + "</p>");
22
23 document.write("<p>Link: " + txt.link("http://www.google.com") + "</p>");
24 </script>
25 <br/> <br/>
26 <p><b>Obs.</b>Sirul stilizat nu se modifica!</p>
27 </body>
28 </html>

```

Exemplul 3

Ilustrează cum se utilizează metoda **concat()** pentru a concatena șiruri.

Concatenarea a două șiruri:

```

1  <html>
2  <body>
3  <h3>Obiectul String. Concatenarea a doua siruri.</h3>
4  <hr/>
5  <script>
6
7  var txt1 = "Buna ";
8  var txt2 = "ziua!";
9  document.write("Primul sir este: "+txt1+"<br/>");
10 document.write("Al doilea sir este: "+txt2+"<br/>");
11 document.write("Sirul concatenat este: "+txt1.concat(txt2)+"<br/>");
12
13 </script>
14 <p><b>Obs.</b>Sirurile concatenate nu se modifica. Rezultatul concatenarii poate fi pastrat
15 intr-un nou sir.</p>
16 </body>
   </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul String. Concatenarea a doua siruri.</h3>
4  <hr/>
5  <script>
6
7  var txt1 = "Buna ";
8  var txt2 = "ziua!";
9  document.write("Primul sir este: "+txt1+"<br/>");
10 document.write("Al doilea sir este: "+txt2+"<br/>");
11 document.write("Sirul concatenat este: "+txt1.concat(txt2)+"<br/>");
12
13 </script>
14 <p><b>Obs.</b>Sirurile concatenate nu se modifica. Rezultatul concatenarii poate fi pastrat
15 intr-un nou sir.</p>
16 </body>
   </html>

```

Concatenarea a trei șiruri:

```

1  <html>
2  <body>
3  <h3>Obiectul String. Concatenarea a trei siruri.</h3>
4  <hr/>
5  <script>
6
7  var txt1="Buna ";
8  var txt2="ziua!";
9  var txt3=" Bine ati venit!";
10 document.write("Primul sir este: "+txt1+"<br/>");
11 document.write("Al doilea sir este: "+txt2+"<br/>");
12 document.write("Al treilea sir este: "+txt3+"<br/>");
13 document.write("Sirul concatenat este: "+txt1.concat(txt2,txt3)+"<br/>");
14 </script>
15 <p><b>Obs.</b>Sirurile concatenate nu se modifica. Rezultatul concatenarii poate fi pastrat
16 intr-un nou sir.</p>
   </body>
   </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul String. Concatenarea a trei siruri.</h3>
4  <hr/>
5  <script>
6  var txt1="Buna ";

```

```

7
8 var txt2="ziua!";
9 var txt3=" Bine ati venit!";
10 document.write("Primul sir este: "+txt1+"<br/>");
11 document.write("Al doilea sir este: "+txt2+"<br/>");
12 document.write("Al treilea sir este: "+txt3+"<br/>");
13 document.write("Sirul concatenat este: "+txt1.concat(txt2,txt3)+"<br/>");
14 </script>
15 <p><b>Obs.</b>Sirurile concatenate nu se modifica. Rezultatul concatenarii poate fi pastrat
16 intr-un nou sir.</p>
  </body>
</html>

```

Exemplul 4

Ilustrează cum se utilizează metoda **indexOf()** pentru a determina poziția primei apariții a unei valori într-un șir.

```

1
2 <html>
3 <body>
4 <h3>Obiectul String. Cautarea primei aparitii a unei valori in sir cu indexOf().</h3> <hr/>
5 <script>
6 var str="Buna ziua!";
7 document.write("Sirul in care se cauta este: "+str+"<br/>");
8 document.write("Sirul \"Buna\" apare in sir in pozitia "+str.indexOf("Buna") + "<br />");
9 document.write("Sirul \"ZIUA\" apare in sir in pozitia "+str.indexOf("ZIUA") + "<br />");
10 document.write("Sirul \"ziua\" apare in sir in pozitia "+str.indexOf("ziua"));
11 </script>
12 <p><b>Obs.</b>Sirul nu se modifica in urma cautarii!</p>
13 </body>
  </html>

```

```

1
2 <html>
3 <body>
4 <h3>Obiectul String. Cautarea primei aparitii a unei valori in sir cu indexOf().</h3> <hr/>
5 <script>
6 var str="Buna ziua!";
7 document.write("Sirul in care se cauta este: "+str+"<br/>");
8 document.write("Sirul \"Buna\" apare in sir in pozitia "+str.indexOf("Buna") + "<br />");
9 document.write("Sirul \"ZIUA\" apare in sir in pozitia "+str.indexOf("ZIUA") + "<br />");
10 document.write("Sirul \"ziua\" apare in sir in pozitia "+str.indexOf("ziua"));
11 </script>
12 <p><b>Obs.</b>Sirul nu se modifica in urma cautarii!</p>
13 </body>
  </html>

```

Valorile afișate sunt: **0 -1 5**.

Obs. Dacă valoarea nu apare în șir, valoarea returnată este -1. Șirurile sunt indexate de la **0**.

Exemplul 5

Ilustrează cum se utilizează metoda **match()** pentru a căuta un subșir într-un șir. Metoda returnează subșirul, dacă este găsit, sau valoarea null, dacă subșirul nu este găsit în șir.

```

1
2 <html>
3 <body>
4 <h3>Obiectul String. Cauta unui subsir intr-un sir cu match().</h3> <hr/>
5 <script>
6 var str="Hello world!";
7 document.write("Sirul in care se cauta este: "+str+"<br/>");
8 document.write("Sirul cautat: "+"world"+" ");
9 document.write("Valoarea returnata: "+str.match("world") + "<br />");
10 document.write("Sirul cautat: "+"World"+" ");
11 document.write("Valoarea returnata: "+str.match("World") + "<br />");
12 document.write("Sirul cautat: "+"worlld"+" ");
13 document.write("Valoarea returnata: "+str.match("worlld") + "<br />");

```

```

14 </script>
15 <p><b>Obs.</b>Sirul nu se modifica in urma cautarii. Rezultatul poate fi memorat intr-o
16 variabila.</p>
</body>
</html>

1
2 <html>
3 <body>
4 <h3>Obiectul String. Cauta unui subsir intr-un sir cu match().</h3> <hr/>
5 <script>
6 var str="Hello world!";
7 document.write("Sirul in care se cauta este: "+str+"<br/>");
8 document.write("Sirul cautat: "+"world"+" ". ");
9 document.write("Valoarea returnata: "+str.match("world") + "<br />");
10 document.write("Sirul cautat: "+"World"+" ". ");
11 document.write("Valoarea returnata: "+str.match("World") + "<br />");
12 document.write("Sirul cautat: "+"worldld"+" ". ");
13 document.write("Valoarea returnata: "+str.match("worldld") + "<br />");
14 </script>
15 <p><b>Obs.</b>Sirul nu se modifica in urma cautarii. Rezultatul poate fi memorat intr-o
16 variabila.</p>
</body>
</html>

```

Exemplul 6

Ilustrează cum se utilizează metoda **replace()** pentru a înlocui o secvență din șir cu altă secvență.

```

1
2 <html>
3 <body>
4 <h3>Obiectul String. Inlocuirea unei secvente din sir cu replace().</h3> <hr/>
5 <script>
6
7 var str="Vizitati Microsoft!";
8 document.write("Sirul initial este: "+str+"<br/>");
9 document.write("Subsirul care se modifica este: "+"Microsoft"+"<br/>");
10 document.write("Subsirul cu care se inlocuieste este: "+"Google"+"<br/>");
11 str.replace("Microsoft","Google");
12 document.write("Sirul obtinut este: "+str);
13
14 </script>
15 <p><b>Obs.</b>Sirul se modifica!</p>
16 </body>
</html>

1
2 <html>
3 <body>
4 <h3>Obiectul String. Inlocuirea unei secvente din sir cu replace().</h3> <hr/>
5 <script>
6
7 var str="Vizitati Microsoft!";
8 document.write("Sirul initial este: "+str+"<br/>");
9 document.write("Subsirul care se modifica este: "+"Microsoft"+"<br/>");
10 document.write("Subsirul cu care se inlocuieste este: "+"Google"+"<br/>");
11 str.replace("Microsoft","Google");
12 document.write("Sirul obtinut este: "+str);
13
14 </script>
15 <p><b>Obs.</b>Sirul se modifica!</p>
16 </body>
</html>

```

Exemplul 7

Ilustrează cum se folosește metoda **slice()** pentru a extrage dintr-un șir o secvență. Metoda returnează șirul extras

sau valoarea **-1**. În mod normal are două argumente: poziția din care începe extragerea (primul caracter are indexul **0**) și, opțional, poziția în care se încheie extragerea. Dacă al doilea argument lipsește, se vor extrage toate caracterele dintre poziția de început și sfârșitul șirului. Dacă se folosesc valori negative, extragerea se va face numărând înapoi de la sfârșitul șirului.

```

1  <html>
2  <body>
3  <h3>Obiectul String. Extragerea unui subsir dintr-un sir cu slice().</h3> <hr/>
4  <script>
5
6  var str="Bine ati venit!";
7  document.write("Sirul initial este: "+str+"<br/>");
8  // extrage toate caracterele incepand cu pozitia 0:
9  var txt=str.slice(0);
10 document.write("Subsirul extras cu slice(0): "+txt+"<br/>");
11
12 //extrage toate caracterele incepand cu pozitia 5:
13 txt=str.slice(5);
14 document.write("Subsirul extras cu slice(5): "+txt+"<br />");
15
16 //extrage ultimele 6 caractere de la sfarsitul sirului:
17 txt=str.slice(-6);
18 document.write("Subsirul extras cu slice(-6): "+txt+"<br />");
19
20 //extrage primul caracter din sir:
21 txt=str.slice(0,1);
22 document.write("Subsirul extras cu slice(0,1): "+txt+"<br />");
23
24 //extrage caracterele dintre pozitiile 5 și 10
25 txt=str.slice(5,10);
26 document.write("Subsirul extras cu slice(5,10): "+txt+"<br />");
27
28 </script>
29 <p><b>Obs.</b> Sirul nu se modifica in urma extragerii. Subsirul extras poate fi memorat intr-
30 o variabila.</p>
31 </body>
32 </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul String. Extragerea unui subsir dintr-un sir cu slice().</h3> <hr/>
4  <script>
5
6  var str="Bine ati venit!";
7  document.write("Sirul initial este: "+str+"<br/>");
8  // extrage toate caracterele incepand cu pozitia 0:
9  var txt=str.slice(0);
10 document.write("Subsirul extras cu slice(0): "+txt+"<br/>");
11
12 //extrage toate caracterele incepand cu pozitia 5:
13 txt=str.slice(5);
14 document.write("Subsirul extras cu slice(5): "+txt+"<br />");
15
16 //extrage ultimele 6 caractere de la sfarsitul sirului:
17 txt=str.slice(-6);
18 document.write("Subsirul extras cu slice(-6): "+txt+"<br />");
19
20 //extrage primul caracter din sir:
21 txt=str.slice(0,1);
22 document.write("Subsirul extras cu slice(0,1): "+txt+"<br />");
23
24 //extrage caracterele dintre pozitiile 5 și 10
25 txt=str.slice(5,10);
26 document.write("Subsirul extras cu slice(5,10): "+txt+"<br />");
27
28 </script>
29 <p><b>Obs.</b> Sirul nu se modifica in urma extragerii. Subsirul extras poate fi memorat intr-
30 o variabila.</p>
31 </body>
32 </html>

```

</html>

Exemplul 8

Ilustrează utilizarea metodei **split()** pentru a împărți un șir în subsiruri pe baza unui caracter separator. Dacă caracterul separator este omis, se va returna întreg șirul. Dacă caracterul este șirul vid, șirul va fi împărțit caracter cu caracter. Opțional, se poate preciza și numărul maxim de împărțiri.

```

1  <html>
2  <body>
3  <h3>Obiectul String. Impartirea unui sir in subsiruri cu split().</h3> <hr/>
4  <script>
5
6  var str="Bine ati venit";
7
8  document.write("Sirul initial: "+str+"<br/>");
9
10 //este returnat întreg sirul
11 document.write("Sirurile returnate cu split(): "+str.split() + "<br />");
12
13 //sunt returnate cele trei cuvinte din sir
14 document.write("Sirurile returnate cu split(\" \"): "+str.split(" ") + "<br />");
15
16 //sunt returnate caracterele din sir
17 document.write("Sirurile returnate cu split(\"\\"): "+str.split("") + "<br />");
18
19 //sunt returnate numai primele doua cuvinte din sir
20 document.write("Subsirurile returnate cu split(\" \",2): "+str.split(" ",2)+"<br/>");
21
22 </script>
23 <p><b>Obs.</b> Sirul initial nu se modifica. Rezultatul poate fi memorat intr-o variabila.
24 </body>
25 </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul String. Impartirea unui sir in subsiruri cu split().</h3> <hr/>
4  <script>
5
6  var str="Bine ati venit";
7
8  document.write("Sirul initial: "+str+"<br/>");
9
10 //este returnat întreg sirul
11 document.write("Sirurile returnate cu split(): "+str.split() + "<br />");
12
13 //sunt returnate cele trei cuvinte din sir
14 document.write("Sirurile returnate cu split(\" \"): "+str.split(" ") + "<br />");
15
16 //sunt returnate caracterele din sir
17 document.write("Sirurile returnate cu split(\"\\"): "+str.split("") + "<br />");
18
19 //sunt returnate numai primele doua cuvinte din sir
20 document.write("Subsirurile returnate cu split(\" \",2): "+str.split(" ",2)+"<br/>");
21
22 </script>
23 <p><b>Obs.</b> Sirul initial nu se modifica. Rezultatul poate fi memorat intr-o variabila.
24 </body>
25 </html>

```

Obiectul Date

Obiectul **Date** este utilizat pentru a lucra cu date calendaristice și ore. Un obiect de tip **Date** este creat cu instrucțiunea **new Date()**. Sunt patru metode de a instanția un obiect **Date**:

- 1) `var d = new Date();`
- 2) `var d = new Date(milisecunde);`
- 3) `var d = new Date(dataString);`
- 4) `var d = new Date(an, luna, zi, ore, minute, secunde, milisecunde);`

Setarea datei

Putem manevra ușor datele calendaristice folosind metodele obiectului **Date**.

În exemplul următor, data este setată la 19 februarie 2010:

```

1
2  var myDate=new Date();
    myDate.setFullYear(2010,1,19);

1
2  var myDate=new Date();
    myDate.setFullYear(2010,1,19);

```

În exemplul următor, data este setată la șapte zile în viitor:

```

1
2  var myDate=new Date();
    myDate.setDate(myDate.getDate()+7);

1
2  var myDate=new Date();
    myDate.setDate(myDate.getDate()+7);

```

Compararea a două date calendaristice

Exemplul următor compară data curentă cu 19 februarie 2010:

```

1
2  var myDate=new Date();
3  myDate.setFullYear(2010,1,19);
4  var today = new Date();
5
6  if (myDate>today)
7  {
8    alert("Astazi este inainte de 19 Februarie 2010");
9  }
10 else
11 {
12   alert("Astazi este dupa 19 Februarie 2010");
13 }

1
2  var myDate=new Date();
3  myDate.setFullYear(2010,1,19);
4  var today = new Date();
5
6  if (myDate>today)
7  {
8    alert("Astazi este inainte de 19 Februarie 2010");
9  }
10 else
11 {
12   alert("Astazi este dupa 19 Februarie 2010");
13 }

```

Metodele obiectului Date

Metodă	Descriere
getDate()	Returnează ziua din lună (între 1 și 31)
getDay()	Returnează ziua din săptămână (0-6)
getFullYear()	Returnează anul (patru cifre)
getHours()	Returnează ora (0-23)
getMilliseconds()	Returnează milisecundele (0-999)
getMinutes()	Returnează minutele (0-59)
getMonth()	Returnează luna (0-11)
getSeconds()	Returnează secunde (0-59)
getTime()	Returnează numărul de milisecunde scurse de la 1.01.1970
getTimezoneOffset()	Returnează diferența dintre GMT și timpul local, în minute
parse()	Analizează(parsează) o dată ca șir de caractere și returnează numărul de milisecunde scurse de la 1.01.1970
setDate()	Setează data din lună (1-31)
setFullYear()	Setează anul (patru cifre)
setHours()	Setează ora (0-23)
setMilliseconds()	Setează milisecundele (0-999)
setMinutes()	Setează minutele (0-59)
setMonth()	Setează lunile (0-11)
setSeconds()	Setează secunde (0-59)
setTime()	Setează o dată și o oră adunînd sau scăzînd un anumit număr de milisecunde la/din 1.01.1970
toString()	Convertește porțiunea corespunzătoare datei calendaristice dintr-un obiect Date într-un șir de caractere
toDateString()	Convertește un obiect Date într-un șir de caractere
toTimeString()	Convertește porțiunea corespunzătoare timpului dintr-un obiect Date într-un șir de caractere
valueOf()	Returnează valoarea primară a unui obiect Date

Exemple:

Exemplul 1

Ilustrează utilizarea metodei **Date()** pentru a obține data curentă.

```

2   <html>
3   <body>
4   <h3>Obiectul Date. Obținerea datei curente cu Date().</h3> <hr/>
5   <script>
6
7   document.write("Astazi este: "+Date());
8
9   </script>
10  </body>
    </html>

```

```

1   <html>
2   <body>
3   <h3>Obiectul Date. Obținerea datei curente cu Date().</h3> <hr/>
4   <script>
5
6   document.write("Astazi este: "+Date());
7
8   </script>
9   </body>
10  </html>

```

Exemplul 2

Ilustrează utilizarea metodei **getTime()** pentru a calcula anii scurși din 1970 până în prezent.

```

1   <html>
2   <body>
3   <h3>Obiectul Date. Utilizarea metodei getTime().</h3>
4   <hr/>
5   <script>
6
7   var d=new Date();
8   document.write("Au trecut "+d.getTime() + " milisecunde din 01.01.1970 și pana acum.");
9   </script>
10  </body>
    </html>

```

```

1   <html>
2   <body>
3   <h3>Obiectul Date. Utilizarea metodei getTime().</h3>
4   <hr/>
5   <script>
6
7   var d=new Date();
8   document.write("Au trecut "+d.getTime() + " milisecunde din 01.01.1970 și pana acum.");
9   </script>
10  </body>
    </html>

```

Exemplul 3

Ilustrează utilizarea metodei **setFullYear()** pentru a seta o dată specifică.

```

1   <html>
2   <body>
3   <h3>Obiectul Date. Setarea datei cu setFullYear().</h3> <hr/>
4   <script>
5
6
7   var d = new Date();
8   d.setFullYear(2010,1,19);
9   document.write("Data a fost setata la "+d);
10
11  </script>
12  </body>
    </html>

```

```
1
2 <html>
3 <body>
4 <h3>Obiectul Date. Setarea datei cu setFullYear().</h3> <hr/>
5 <script>
6
7 var d = new Date();
8 d.setFullYear(2010,1,19);
9 document.write("Data a fost setata la "+d);
10
11 </script>
12 </body>
</html>
```

Exemplul 4

Ilustrează utilizarea metodei **toString()** pentru a converti data curentă într-un șir de caractere.

```
1
2 <html>
3 <body>
4 <script>
5
6 var d=new Date();
7 document.write(d.toString());
8
9 </script>
10 </body>
</html>
```

```
1
2 <html>
3 <body>
4 <script>
5
6 var d=new Date();
7 document.write(d.toString());
8
9 </script>
10 </body>
</html>
```

Exemplul 5

Ilustrează utilizarea metodei **getDay()** și a unui tablou pentru a scrie denumirea zilei din săptămâna curentă.

```
1
2 <html>
3 <body>
4 <h3>Obiectul Date. Utilizarea metodei getDay() pentru a determina ziua din saptamana.</h3>
5 <hr/>
6 <script>
7
8 var d=new Date();
9 var weekday=new Array(7);
10 weekday[0]="Duminica";
11 weekday[1]="Luni";
12 weekday[2]="Marti";
13 weekday[3]="Miercuri";
14 weekday[4]="Joi";
15 weekday[5]="Vineri";
16 weekday[6]="Sambata";
17 document.write("Astazi este " + weekday[d.getDay()]);
18
19 </script>
</body>
</html>
```

```

1  <html>
2  <body>
3  <h3>Obiectul Date. Utilizarea metodei getDay() pentru a determina ziua din saptamana.</h3>
4  <hr/>
5  <script>
6
7  var d=new Date();
8  var weekday=new Array(7);
9  weekday[0]="Duminica";
10 weekday[1]="Luni";
11 weekday[2]="Marti";
12 weekday[3]="Miercuri";
13 weekday[4]="Joi";
14 weekday[5]="Vineri";
15 weekday[6]="Sambata";
16 document.write("Astazi este " + weekday[d.getDay()]);
17
18 </script>
19 </body>
20 </html>

```

Exemplul 6

Ilustrează cum se poate afișa un ceas într-o pagină web.

```

1  <html>
2  <head>
3  <script>
4  function ceas()
5  {
6    var today=new Date();
7    var h=today.getHours();
8    var m=today.getMinutes();
9    var s=today.getSeconds();
10   //functia urmatoare adauga un zero in fata
11   //numerelor<10
12   m=verifica(m);
13   s=verifica(s);
14   document.getElementById('txt').innerHTML=h+":"+m+":"+s;
15   t=setTimeout('ceas()',500);
16 }
17
18 function verifica(i)
19 {
20   if (i<10)
21   {
22     i="0" + i;
23   }
24   return i;
25 }
26 </script>
27 </head>
28 <body onload="ceas()">
29 <h3>Obiectul String. Afisarea unui ceas.</h3> <hr/>
30 <div id="txt"></div>
31 </body>
32 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function ceas()
5  {
6    var today=new Date();
7    var h=today.getHours();
8    var m=today.getMinutes();
9    var s=today.getSeconds();
10   //functia urmatoare adauga un zero in fata

```

```

12 //numerelelor<10
13 m=verifica(m);
14 s=verifica(s);
15 document.getElementById('txt').innerHTML=h+" "+m+" "+s;
16 t=setTimeout('ceas()',500);
17 }
18
19 function verifica(i)
20 {
21   if (i<10)
22   {
23     i="0" + i;
24   }
25   return i;
26 }
27 </script>
28 </head>
29 <body onload="ceas()">
30 <h3>Obiectul String. Afisarea unui ceas.</h3> <hr/>
31 <div id="txt"></div>
32 </body>
   </html>

```

Obiectul Array

Un tablou este o variabilă specială care poate păstra la un moment dat mai multe valori de un anumit tip. Dacă aveți o listă de elemente, animale de companie de exemplu, ați putea păstra valorile în variabile simple, ca în exemplul următor:

```

1
2 pet1="Caine";
3 pet2="Pisica";
  pet3="Papagal";

1
2 pet1="Caine";
3 pet2="Pisica";
  pet3="Papagal";

```

Desigur, problema se complică dacă aveți de memorat zeci, sau sute de valori. Cea mai bună soluție este să folosiți tablouri. Un tablou poate reține toate valorile sub un singur nume și puteți accesa fiecare valoare stocată în tablou folosind numele tabloului și indexul valorii.

Crearea unui tablou

Un tablou poate fi definit în trei moduri:

1:

```

1
2 var pets=new Array();
3 //tablou obisnuit
4 pets[0]="Caine";
5 pets[1]="Pisica";
  pets[2]="Papagal";

1
2 var pets=new Array();
3 //tablou obisnuit
4 pets[0]="Caine";
5 pets[1]="Pisica";
  pets[2]="Papagal";

```


2:

```
1 |
2 | var pets=new Array("Caine","Pisica","Papagal");
   | //tablou condensat

1 |
2 | var pets=new Array("Caine","Pisica","Papagal");
   | //tablou condensat
```

3:

```
1 |
2 | var pets=["Caine","Pisica","Papagal"];
   | //tablou literal

1 |
2 | var pets=["Caine","Pisica","Papagal"];
   | //tablou literal
```

Obs: Dacă în tablou stocați valori numerice sau logice, tipul tabloului va fi **Number** sau **Boolean**, în loc de **String**.

Accesarea elementelor dintr-un tablou

Puteți accesa un element dintr-un tablou precizînd numele tabloului și indexul elementului. Primul element din tablou are indexul **0**.

Următoarea linie de cod

```
document.write(pets[0]);
```

va afișa șirul: **Caine**

Modificarea valorilor dintr-un tablou

Pentru a modifica o valoare dintr-un tablou, este suficient să atribuiți o nouă valoare elementului respectiv, ca în exemplul următor:

```
pets[0]="Iguana";
```

Proprietățile obiectului Array

Proprietatea	Descriere
constructor	Returnează funcția care a creat prototipul obiectului Array
length	Setează sau returnează numărul elementelor stocate în tablou
prototype	Permite adăugare de proprietăți și metode unui obiect

Metodele obiectului Array

Metodă	Descriere
concat()	Concatenează două sau mai multe tablouri și returnează tabloul obținut
join()	Concatenează toate elementele unui tablou într-un șir de caractere
pop()	Înlătură ultimul element dintr-un tablou și returnează respectivul element

push()	Adaugă noi elemente la sfârșitul unui tablou și returnează noua lungime a tabloului
reverse()	Răstoarnă ordinea elementelor dintr-un tablou
shift()	Înlătură primul element dintr-un tablou și returnează respectivul element
slice()	Selectează o parte dintr-un tablou și returnează elementele selectate
sort()	Returnează secunde (0-59)
getTime()	Sortează elementele unui tablou
splice()	Adaugă/Înlătură elemente dintr-un tablou
toString()	Convertește un tablou în șir de caractere și returnează rezultatul
unshift()	Adaugă noi elemente la începutul unui tablou și returnează noua lungime a tabloului
valueOf()	Returnează valoarea primară a unui tablou

Example:

Exemplul 1

Ilustrează crearea unui tablou, atribuirea de valori și afișarea elementelor tabloului.

```

1  <html>
2  <head>
3  <h3>Obiectul Array. Crearea unui tablou, initializarea și afisarea elementelor.</h3> <hr/>
4  <script>
5  var pets = new Array();
6  pets[0] = "Pisica";
7  pets[1] = "Caine";
8  pets[2] = "Perus";
9
10
11 document.write("Elementele memorate in tablou sunt:"+"<br/>");
12 for (i=0;i<pets.length;i++)
13 {
14 document.write(pets[i] + "<br />");
15 }
16 </script>
17 </body>
18 </html>

```

```

1  <html>
2  <head>
3  <h3>Obiectul Array. Crearea unui tablou, initializarea și afisarea elementelor.</h3> <hr/>
4  <script>
5  var pets = new Array();
6  pets[0] = "Pisica";
7  pets[1] = "Caine";
8  pets[2] = "Perus";
9
10
11 document.write("Elementele memorate in tablou sunt:"+"<br/>");
12 for (i=0;i<pets.length;i++)
13 {
14 document.write(pets[i] + "<br />");
15 }
16 </script>
17 </body>
18 </html>

```

Exemplul 2

Ilustrează utilizarea instrucțiunii **for...in** pentru a parcurge elementele unui tablou.

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Afisarea elementelor unui tablou cu instructiunea for..in.</h3> <hr/>
4  <script>
5  var x;
6  var pets = new Array();
7  pets[0] = "Pisica";
8  pets[1] = "Caine";
9  pets[2] = "Perus";
10
11  document.write("Elementele memorate in tablou sunt:"+"<br/>");
12  for (x in pets)
13  {
14  document.write(pets[x] + "<br />");
15  }
16  }
17  </script>
18  </body>
19  </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Afisarea elementelor unui tablou cu instructiunea for..in.</h3> <hr/>
4  <script>
5  var x;
6  var pets = new Array();
7  pets[0] = "Pisica";
8  pets[1] = "Caine";
9  pets[2] = "Perus";
10
11  document.write("Elementele memorate in tablou sunt:"+"<br/>");
12  for (x in pets)
13  {
14  document.write(pets[x] + "<br />");
15  }
16  }
17  </script>
18  </body>
19  </html>

```

Exemplul 3

Ilustrează utilizarea metodei **concat()** pentru a concatena trei tablouri.

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Concatenarea a trei tablouri cu concat().</h3> <hr/>
4  <script>
5
6  var parinti = ["Maria", "George"];
7  var copii = ["Elena", "Mihai"];
8  var frati = ["Paul", "Dan"];
9  var familie = parinti.concat(copii,frati);
10 document.write("Parinti: "+parinti+"<br/>");
11 document.write("Copii: "+copii+"<br/>");
12 document.write("Frati: "+frati+"<br/>");
13
14 document.write("Familia: "+familie);
15
16 </script>
17 <p><b>Obs.</b>Tablourile concatenate nu se modifica. Rezultatul concatenarii este un nou
18 tablou.</p>
19 </body>
20 </html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Concatenarea a trei tablouri cu concat().</h3> <hr/>
4  <script>
5
6  var parinti = ["Maria", "George"];
7  var copii = ["Elena", "Mihai"];
8  var frati = ["Paul", "Dan"];
9  var familie = parinti.concat(copii,frati);
10 document.write("Parinti: "+parinti+"<br/>");
11 document.write("Copii: "+copii+"<br/>");
12 document.write("Fratii: "+frati+"<br/>");
13
14 document.write("Familia: "+familie);
15
16 </script>
17 <p><b>Obs.</b>Tablourile concatenate nu se modifica. Rezultatul concatenarii este un nou
18 tablou.</p>
19 </body>
20 </html>

```

Exemplu 4

Ilustrează utilizarea metodei **join()** pentru a concatena toate elementele unui tablou într-un șir de caractere.

```

1  <html>
2  <body>
3  <h3>obiectul Array. Concatenarea elementelor unui tablou intr-un sir de caractere cu
4  join().</h3> <hr/>
5  <script>
6
7  var fructe = ["Mere", "Pere", "Banane", "Kiwi"];
8  document.write("Tabloul contine valorile: "+fructe+"<br/>");
9  document.write("Sirul concatenat cu join(\" \"): "+fructe.join(" ") + "<br />");
10 document.write("Sirul concatenat cu join(\"+\"): "+fructe.join("+") + "<br />");
11 document.write("Sirul concatenat cu join(\" și \"): "+fructe.join(" și ")+"<br/>");
12
13 </script>
14 <p><b>Obs.</b>Tabloul nu se modifica. Sirul concatenat poate fi memorat intr-o variabila.
15 </body>
16 </html>

```

```

1  <html>
2  <body>
3  <h3>obiectul Array. Concatenarea elementelor unui tablou intr-un sir de caractere cu
4  join().</h3> <hr/>
5  <script>
6
7  var fructe = ["Mere", "Pere", "Banane", "Kiwi"];
8  document.write("Tabloul contine valorile: "+fructe+"<br/>");
9  document.write("Sirul concatenat cu join(\" \"): "+fructe.join(" ") + "<br />");
10 document.write("Sirul concatenat cu join(\"+\"): "+fructe.join("+") + "<br />");
11 document.write("Sirul concatenat cu join(\" și \"): "+fructe.join(" și ")+"<br/>");
12
13 </script>
14 <p><b>Obs.</b>Tabloul nu se modifica. Sirul concatenat poate fi memorat intr-o variabila.
15 </body>
16 </html>

```

Exemplu 5

Ilustrează utilizarea metodei **pop()** pentru a înlătura ultimul element dintr-un tablou.

```

1  <html>
2  <body>
3

```

```

4  <h3>Obiectul Array. Eliminarea ultimului element din tablou cu pop().</h3> <hr/>
5  <script>
6
7  var pets = ["Pisica", "Caine", "Hamster", "Iguana"];
8
9  document.write("Tablou initial contine valorile: "+pets+"<br/>");
10 document.write("S-a eliminat valoarea: "+pets.pop() + "<br />");
11 document.write("Tabloul dupa eliminare: "+pets + "<br />");
12 document.write("S-a eliminat valoarea: "+pets.pop() + "<br />");
13 document.write("Tabloul dupa eliminare: "+pets);
14
15 </script>
16 <p><b>Obs.</b> Tabloul se modifica! Valoarea eliminata poate fi memorata intr-o
17 variabila.</p>
</body>
</html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Eliminarea ultimului element din tablou cu pop().</h3> <hr/>
4  <script>
5
6  var pets = ["Pisica", "Caine", "Hamster", "Iguana"];
7
8  document.write("Tablou initial contine valorile: "+pets+"<br/>");
9  document.write("S-a eliminat valoarea: "+pets.pop() + "<br />");
10 document.write("Tabloul dupa eliminare: "+pets + "<br />");
11 document.write("S-a eliminat valoarea: "+pets.pop() + "<br />");
12 document.write("Tabloul dupa eliminare: "+pets);
13
14 </script>
15 <p><b>Obs.</b> Tabloul se modifica! Valoarea eliminata poate fi memorata intr-o
16 variabila.</p>
17 </body>
</html>

```

Exemplul 6

Ilustrează utilizarea metodei **push()** pentru a adăuga noi elemente la sfârșitul unui tablou. Pot fi adăugate mai multe valori simultan. Valorile trebuie separate prin virgulă.

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Adaugarea de noi elemente la sfarsitul tabloului cu push().</h3> <hr/>
4  <script>
5
6  var pets = ["Pisica", "Caine", "Perus", "Hamster"];
7
8  document.write("Tabloul initial: "+pets+"<br/>");
9  document.write("Se adauga valorile: "+Iguana+"<br/>");
10 pets.push("Iguana");
11 document.write("Tabloul obtinut: "+pets+"<br/>");
12 document.write("Se adauga valorile: "+Pesti și Iepure+"<br/>");
13 k=pets.push("Pesti", "Iepure");
14 document.write("Tabloul obtinut: "+pets+"<br/>");
15 document.write("Tabloul final are "+k+" elemente");
16
17 </script>
18 <p><b>Obs.</b> Tabloul se modifica. Metoda returneaza noua lungime a tabloului și valoarea
19 poate fi memorata intr-o variabila.</p>
20 </body>
</html>

```

```

1  <html>
2  <body>
3  <h3>Obiectul Array. Adaugarea de noi elemente la sfarsitul tabloului cu push().</h3> <hr/>
4  <script>
5

```

```

6   var pets = ["Pisica", "Caine", "Perus", "Hamster"];
7
8   document.write("Tabloul initial: "+pets+"<br/>");
9   document.write("Se adauga valorile: "+"Iguana"+"<br/>");
10  pets.push("Iguana");
11  document.write("Tabloul obtinut: "+pets+"<br/>");
12  document.write("Se adauga valorile: "+"Pesti și Iepure"+"<br/>");
13  k=pets.push("Pesti","Iepure");
14  document.write("Tabloul obtinut: "+pets+"<br/>");
15  document.write("Tabloul final are "+k+" elemente");
16
17
18  </script>
19  <p><b>Obs.</b> Tabloul se modifica. Metoda returneaza noua lungime a tabloului și valoarea
20  poate fi memorata intr-o variabila.</p>
    </body>
    </html>

```

Exemplul 7

Ilustrează utilizarea metodei **reverse()** pentru a inversa ordinea elementelor dintr-un tablou.

```

1   <html>
2   <body>
3   <h3>Obiectul Array. Inversarea ordinii elementelor din tablou cu reverse().</h3> <hr/>
4   <script>
5   var friends = ["Mihai", "Elena", "Andra", "Dan"];
6   document.write("Tabloul initial: "+friends+"<br/>");
7   friends.reverse();
8   document.write("Tabloul dupa inversare: "+friends);
9
10  </script>
11  </body>
12  </html>

```

```

1   <html>
2   <body>
3   <h3>Obiectul Array. Inversarea ordinii elementelor din tablou cu reverse().</h3> <hr/>
4   <script>
5   var friends = ["Mihai", "Elena", "Andra", "Dan"];
6   document.write("Tabloul initial: "+friends+"<br/>");
7   friends.reverse();
8   document.write("Tabloul dupa inversare: "+friends);
9
10  </script>
11  </body>
12  </html>

```

Exemplul 8

Ilustrează utilizarea metodei **shift()** pentru a elimina primul element dintr-un tablou.

```

1   <html>
2   <body>
3   <h3>Obiectul Array. Eliminarea primului element din tablou cu shift().</h3> <hr/>
4   <script>
5
6   var friends = ["Mihai", "Elena", "Andra", "Dan"];
7   document.write("Tabloul initial: "+friends+"<br/>");
8
9   var x=friends.shift();
10  document.write("Elementul eliminat: "+x+"<br/>");
11  document.write("Tabloul dupa eliminare: "+friends);
12
13  </script>
14
15

```

```

16 <p><b>Obs.</b> Tabloul se modifica. Metoda returneaza elementul eliminat și rezultatul poate
17 fi memorat într-o variabila.</p>
   </body>
   </html>

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Eliminarea primului element din tablou cu shift().</h3> <hr/>
5 <script>
6
7 var friends = ["Mihai", "Elena", "Andra", "Dan"];
8 document.write("Tabloul initial: "+friends+"<br/>");
9
10 var x=friends.shift();
11 document.write("Elementul eliminat: "+x+"<br/>");
12 document.write("Tabloul dupa eliminare: "+friends);
13
14 </script>
15
16 <p><b>Obs.</b> Tabloul se modifica. Metoda returneaza elementul eliminat și rezultatul poate
17 fi memorat într-o variabila.</p>
   </body>
   </html>

```

Exemplul 9

Ilustrează cum se selectează elementele unui tablou cu metoda **slice()**. Metoda are două argumente: primul precizează poziția de început a secvenței selectate, iar al doilea poziția de sfârșit. Dacă al doilea argument lipsește, se vor selecta toate elementele pînă la sfârșitul tabloului. Dacă argumentul este negativ, se vor selecta elementele de la sfârșitul șirului către început.

```

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Selectare elementelor din tablou cu slice().</h3> <hr/>
5 <script>
6
7 var pets = ["Caine", "Pisica", "Papagal", "Hamster"];
8 document.write("Tabloul initial: "+pets+"<br/>");
9
10 var x=pets.slice(0,1);
11 document.write("Elementele selectate cu slice(0,1): "+x+"<br/>");
12 document.write("Elementele selectate cu slice(1): "+pets.slice(1) + "<br />");
13 document.write("Elementele selectate cu slice(-2): "+pets.slice(-2) + "<br />");
14
15 </script>
16 <p><b>Obs.</b> Tabloul nu se modifica. Elementele selectate pot fi memorate într-o
17 variabila.</p>
   </body>
   </html>

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Selectare elementelor din tablou cu slice().</h3> <hr/>
5 <script>
6
7 var pets = ["Caine", "Pisica", "Papagal", "Hamster"];
8 document.write("Tabloul initial: "+pets+"<br/>");
9
10 var x=pets.slice(0,1);
11 document.write("Elementele selectate cu slice(0,1): "+x+"<br/>");
12 document.write("Elementele selectate cu slice(1): "+pets.slice(1) + "<br />");
13 document.write("Elementele selectate cu slice(-2): "+pets.slice(-2) + "<br />");
14
15 </script>
16 <p><b>Obs.</b> Tabloul nu se modifica. Elementele selectate pot fi memorate într-o

```

```

17 | variabila.</p>
    | </body>
    | </html>

```

Exemplul 10

Ilustrează utilizarea metodei **sort()** pentru a sorta alfabetic crescător un tablou de șiruri de caractere. Metoda sortează implicit în ordine alfabetică crescătoare.

```

1 |
2 | <html>
3 | <body>
4 | <h3>Obiectul Array. Sortarea unui tablou cu sort().</h3> <hr/>
5 | <script>
6 |
7 | var friends = ["Mihai", "Elena", "Andra", "Dan"];
8 | document.write("Tabloul initial: "+friends+"<br/>");
9 | document.write("Tabloul sortat: "+friends.sort());
10|
11| </script>
12| <p><b>Obs.</b> Tabloul se modifica!</p>
13| </body>
    | </html>

```

```

1 |
2 | <html>
3 | <body>
4 | <h3>Obiectul Array. Sortarea unui tablou cu sort().</h3> <hr/>
5 | <script>
6 |
7 | var friends = ["Mihai", "Elena", "Andra", "Dan"];
8 | document.write("Tabloul initial: "+friends+"<br/>");
9 | document.write("Tabloul sortat: "+friends.sort());
10|
11| </script>
12| <p><b>Obs.</b> Tabloul se modifica!</p>
13| </body>
    | </html>

```

Exemplul 11

Ilustrează utilizarea metodei **sort()** pentru a sorta descendent un tablou de numere. Numerele nu vor fi sortate corect. Trebuie adăugată o funcție care să compare numerele.

```

1 |
2 | <html>
3 | <body>
4 | <h3>Obiectul Array. Sortarea unui tablou cu valori numerice.</h3> <hr/>
5 | <script>
6 |
7 | function sortDesc(a, b)
8 | {
9 |   return b - a;
10| }
11|
12| function sortCresc(a,b)
13| {
14|   return a-b;
15| }
16|
17| var n = new Array(10,5,40,25,100,1);
18| document.write("Tabloul initial: "+n+"<br/>");
19| document.write("Tabloul sortat crescator: "+n.sort(sortCresc)+"<br/>");
20| document.write("Tabloul sortat descrescator: "+n.sort(sortDesc));
21|
22| </script>
23| <p><b>Obs.</b> In urma sortarii tabloul se modifica!</p>

```



```

24 | </body>
    | </html>

1 |
2 | <html>
3 | <body>
4 | <h3>Obiectul Array. Sortarea unui tablou cu valori numerice.</h3> <hr/>
5 | <script>
6 |
7 | function sortDesc(a, b)
8 | {
9 |   return b - a;
10 | }
11 |
12 | function sortCresc(a,b)
13 | {
14 |   return a-b;
15 | }
16 |
17 | var n = new Array(10,5,40,25,100,1);
18 | document.write("Tabloul initial: "+n+"<br/>");
19 | document.write("Tabloul sortat crescator: "+n.sort(sortCresc)+"<br/>");
20 | document.write("Tabloul sortat descrescator: "+n.sort(sortDesc));
21 |
22 | </script>
23 | <p><b>Obs.</b> In urma sortarii tabloul se modifica!</p>
24 | </body>
    | </html>

```

Exemplul 12

Ilustrează utilizarea metodei **splice()** pentru a adăuga un element în poziția 3 din tablou. Metoda are trei argumente: primul, obligatoriu precizează poziția în care vor fi adăugate/șterse valori, al doilea, obligatoriu, reprezintă numărul de valori care vor fi șterse (dacă are valoarea 0, nu se vor șterge ci se vor insera valori) și, al treilea, opțional, care reprezintă noile valori adăugate în tablou.

```

1 |
2 | <html>
3 | <body>
4 | <h3>Obiectul Array. Inserarea/stergere elementelor dintr-o pozitie a tabloului cu
5 | splice().</h3> <hr/>
6 | <script>
7 |
8 | var friends = ["Ana", "Mircea", "Dan", "Maria"];
9 |
10 | document.write("Tabloul initial: "+friends+"<br/>");
11 | document.write("Se adauga valoarea \"Andra\" in pozitia 3 din tablou cu
12 | splice(3,0)+"<br/>");
13 | friends.splice(3,0,"Andra");
14 | document.write("Tabloul obtinut: "+friends+"<br/>");
15 | document.write("Se sterg din tablou primele doua valori cu splice(0,2)+"<br/>");
16 | document.write("Valorile sterse: "+friends.splice(0,2)+"<br/>");
17 | document.write("Tabloul obtinut: "+friends);
18 |
19 | </script>
    | <p><b>Obs.</b> Tabloul se modifica. Daca metoda elimina elemente din tablou, va returna
    | elementele eliminate.</p>
    | </body>
    | </html>

1 |
2 | <html>
3 | <body>
4 | <h3>Obiectul Array. Inserarea/stergere elementelor dintr-o pozitie a tabloului cu
5 | splice().</h3> <hr/>
6 | <script>
7 |
8 | var friends = ["Ana", "Mircea", "Dan", "Maria"];

```

```

9
10 document.write("Tabloul initial: "+friends+"<br/>");
11 document.write("Se adauga valoarea \"Andra\" in pozitia 3 din tablou cu
12 splice(3,0)"+ "<br/>");
13 friends.splice(3,0,"Andra");
14 document.write("Tabloul obtinut: "+friends+"<br/>");
15 document.write("Se sterg din tablou primele doua valori cu splice(0,2)"+ "<br/>");
16 document.write("Valorile sterse: "+friends.splice(0,2)+"<br/>");
17 document.write("Tabloul obtinut: "+friends);
18
19 </script>
<p><b>Obs.</b> Tabloul se modifica. Daca metoda elimina elemente din tablou, va returna
elementele eliminate.</p>
</body>
</html>

```

Exemplul 13

Ilustrează utilizarea metodei **toString()** pentru a converti un tablou într-un șir de caractere. Metoda returnează șirul de caractere, valorile fiind separate prin virgulă.

```

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Convertirea unui tablou in sir de caractere cu toString().</h3> <hr/>
5 <script>
6
7 var n = new Array(14,0,7,-4,25,13,7);
8 document.write("Tabloul contine valorile:"+ "<br/>");
9 for(x in n) document.write(n[x]+"<br/>");
10 document.write("Sirul de caractere obtinut:" +n.toString());
11
12 </script>
13 <p><b>Obs.</b> Tabloul nu se modifica. Metoda returneaza sirul de caractere obtinut.</p>
14 </body>
</html>

```

```

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Convertirea unui tablou in sir de caractere cu toString().</h3> <hr/>
5 <script>
6
7 var n = new Array(14,0,7,-4,25,13,7);
8 document.write("Tabloul contine valorile:"+ "<br/>");
9 for(x in n) document.write(n[x]+"<br/>");
10 document.write("Sirul de caractere obtinut:" +n.toString());
11
12 </script>
13 <p><b>Obs.</b> Tabloul nu se modifica. Metoda returneaza sirul de caractere obtinut.</p>
14 </body>
</html>

```

Exemplul 14

Ilustrează utilizarea metodei **unshift()** pentru a adăuga noi valori la începutul unui tablou

```

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Adaugarea de elemente la inceputul unui tablou cu unshift().</h3> <hr/>
5 <script>
6
7 var pets = ["Pisica", "Caine", "Iguana", "Pesti"];
8 document.write("Tabloul initial: "+pets+"<br/>");
9 document.write("Se adauga valoarea \"Papagal\" "+ "<br/>");
10 pets.unshift("Papagal");
11 document.write("Tabloul dupa adaugare: "+pets+"<br/>");

```

```

12 document.write("Se adauga valorile \"Perus\" și \"Broasca testoasa\" "+"<br/>");
13 k=pets.unshift("Perus","Broasca testoasa");
14 document.write("Tabloul dupa adaugare: "+pets+"<br/>");
15 document.write("Tabloul final are "+k+" elemente.");
16 </script>
17
18 <p><b>Nota:</b> Metoda unshift() nu lucreaza corect in Internet Explorer; returneaza
19 undefined!</p>
20 <p><b>Obs.</b> Tabloul se modifica. Metoda returneaza numarul de elemente din tablou dupa
21 adaugare.
22 <br>Valoarea poate fi memorata intr-o variabila.</p>

</body>
</html>

1
2 <html>
3 <body>
4 <h3>Obiectul Array. Adaugarea de elemente la inceputul unui tablou cu unshift().</h3> <hr/>
5 <script>
6
7 var pets = ["Pisica", "Caine", "Iguana", "Pesti"];
8 document.write("Tabloul initial: "+pets+"<br/>");
9 document.write("Se adauga valoarea \"Papagal\" "+"<br/>");
10 pets.unshift("Papagal");
11 document.write("Tabloul dupa adaugare: "+pets+"<br/>");
12 document.write("Se adauga valorile \"Perus\" și \"Broasca testoasa\" "+"<br/>");
13 k=pets.unshift("Perus","Broasca testoasa");
14 document.write("Tabloul dupa adaugare: "+pets+"<br/>");
15 document.write("Tabloul final are "+k+" elemente.");
16 </script>
17
18 <p><b>Nota:</b> Metoda unshift() nu lucreaza corect in Internet Explorer; returneaza
19 undefined!</p>
20 <p><b>Obs.</b> Tabloul se modifica. Metoda returneaza numarul de elemente din tablou dupa
21 adaugare.
22 <br>Valoarea poate fi memorata intr-o variabila.</p>

</body>
</html>

```

Obiectul Boolean

Obiectul **Boolean** este utilizat pentru a converti o valoare ne-booleană într-o valoare booleană (cu valoarea **true** sau **false**).

Crearea unui obiect boolean poate fi realizată ca în secvența de cod următoare:

```
var sem=new Boolean();
```

Obs: Dacă obiectul **Boolean** nu are valoare inițială sau are una din valorile **0**, **-0**, **null**, **""**, **false**, **undefined**, sau **NaN(Not a Number)**, obiectul este inițializat cu valoarea **false**. În caz contrar, valoarea obiectului va fi **true**.

Toate declarațiile din liniile următoare de cod creează un obiect boolean inițializat cu **false**:

```
var sem=new Boolean();
```

```
var sem=new Boolean(0);
```

```
var sem=new Boolean(null);
```

```
var sem=new Boolean("");
```

```
var sem=new Boolean(false);
```

```
var sem=new Boolean(NaN);
```

Toate declarațiile din liniile următoare de cod crează un obiect boolean inițializat cu **true**:

```
var sem=new Boolean(true);
```

```
var sem=new Boolean("true");
```

```
var sem=new Boolean("false");
var sem=new Boolean("home");
```

Proprietățile obiectului Boolean

Proprietate	Descriere
constructor	Returnează funcția care a creat prototipul obiectului
prototype	Permite adăugarea de proprietăți și metode unui obiect

Metodele obiectului Boolean

Metodă	Descriere
toString()	Convertește o valoare booleană în șir de caractere și returnează rezultatul
valueOf()	Returnează valoarea primară a unui obiect Boolean

Exemplu

Ilustrează cum se verifică valoarea unui obiect **Boolean**.

```
1  <html>
2  <body>
3  <script>
4  var b1=new Boolean( 0);
5  var b2=new Boolean(1);
6  var b3=new Boolean("");
7  var b4=new Boolean(null);
8  var b5=new Boolean(NaN);
9  var b6=new Boolean("false");
10
11 document.write("0 are valoarea "+ b1 + "<br />");
12 document.write("1 are valoarea "+ b2 + "<br />");
13 document.write("Un sir vid are valoarea "+ b3 + "<br />");
14 document.write("null are valoarea "+ b4+ "<br />");
15 document.write("NaN are valoarea "+ b5 + "<br />");
16 document.write("Sirul 'false' are valoarea "+ b6 + "<br />");
17 </script>
18 </body>
19 </html>
```

```
1  <html>
2  <body>
3  <script>
4  var b1=new Boolean( 0);
5  var b2=new Boolean(1);
6  var b3=new Boolean("");
7  var b4=new Boolean(null);
8  var b5=new Boolean(NaN);
9  var b6=new Boolean("false");
10
11 document.write("0 are valoarea "+ b1 + "<br />");
12 document.write("1 are valoarea "+ b2 + "<br />");
13 document.write("Un sir vid are valoarea "+ b3 + "<br />");
14 document.write("null are valoarea "+ b4+ "<br />");
15 document.write("NaN are valoarea "+ b5 + "<br />");
16 document.write("Sirul 'false' are valoarea "+ b6 + "<br />");
17 </script>
18 </body>
19 </html>
```

Obiectul Math

Obiectul **Math** permite realizarea prelucrărilor matematice. Obiectul include constante matematice și metode. Obiectul nu are constructor. Toate proprietățile și metodele pot fi utilizate fără a crea efectiv un obiect.

Sintaxa de utilizare:

```
var pi=Math.PI;
```

```
var x=Math.sqrt(16);
```

Constante matematice

În JavaScript se pot utiliza opt constante matematice accesibile prin obiectul **Math**. Ele pot fi utilizate cu următoarea sintaxă:

Math.E

Math.PI

Math.SQRT2

Math.SQRT1_2

Math.LN2

Math.LN10

Math.LOG2E

Math.LOG10E

Metode matematice

Exemplul următor ilustrează utilizarea metodei **round()** pentru a rotunji un număr la cel mai apropiat întreg:

```
1 document.write(Math.round(4.7));
```

```
1 document.write(Math.round(4.7));
```

Exemplul următor utilizează metoda **random()** pentru a genera un număr aleator cuprins între **0** și **1**:

```
1 document.write(Math.random());
```

```
1 document.write(Math.random());
```

Exemplul următor utilizează metodele **floor()** și **random()** pentru a genera un număr aleator cuprins între **0** și **10**:

```
1 document.write(Math.floor(Math.random()*11));
```

```
1 document.write(Math.floor(Math.random()*11));
```

Proprietățile obiectului Math

Proprietate	Descriere

E	Returnează constanta lui Euler (cca. 2.718)
LN2	Returnează logaritm natural din 2 (cca. 0.693)
LN10	Returnează logaritm natural din 10 (cca. 2.302)
LOG2E	Returnează logaritm în baza 2 din E (cca. 1.442)
LOG10E	Returnează logaritm în baza 10 din E (cca. 0.434)
PI	Returnează PI (cca. 3.14159)
SQRT1_2	Returnează rădăcina pătrată din 1/2 (cca. 0.707)
SQRT2	Returnează rădăcina pătrată din 2 (cca. 1.414)

Metodele obiectului Math

Metodă	Descriere
abs(x)	Valoarea absolută a lui x
acos(x)	Arccosinus de x, în radiani
asin(x)	Arcsinus de x, în radiani
atan(x)	Arctangentă de x ca valoare numerică între -PI/2 și PI/2 radiani
ceil(x)	Rotunjește x la întreg prin adaos
cos(x)	Cosinus de x (x în radiani)
exp(x)	Valoarea lui E ^x
floor(x)	Rotunjește x la întreg prin lipsă
log(x)	Logaritm natural din x
max(x,y,z,...,n)	Valoarea maximă din șir
min(x,y,z,...,n)	Valoarea minimă din șir
pow(x,y)	Valoarea lui x la puterea y
random()	Un număr aleator între 0 și 1
round(x)	Rotunjește x la cel mai apropiat întreg
sin(x)	Sinus de x (x în radiani)
sqrt(x)	Rădăcină pătrată din x
tan(x)	Tangenta unui unghi

Exemple:

Exemplul 1

Ilustrează utilizarea metodei **max()** pentru a determina maximul a două sau mai multe valori.

```

2  <html>
3  <body>
4  <script>
5  document.write(Math.max(5,10) + "<br />");
6  document.write(Math.max(0,150,30,20,38) + "<br />");
7  document.write(Math.max(-5,10) + "<br />");
8  document.write(Math.max(-5,-10) + "<br />");
9  document.write(Math.max(1.5,2.5));
10 </script>
11 </body>
    </html>

```

```

1  <html>
2  <body>
3  <script>
4  document.write(Math.max(5,10) + "<br />");
5  document.write(Math.max(0,150,30,20,38) + "<br />");
6  document.write(Math.max(-5,10) + "<br />");
7  document.write(Math.max(-5,-10) + "<br />");
8  document.write(Math.max(1.5,2.5));
9  </script>
10 </body>
11 </html>

```

Exemplul 2

Ilustrează utilizarea metodei **min()** pentru a determina minimul a două sau mai multe valori.

```

1  <html>
2  <body>
3  <script>
4  document.write(Math.min(5,10) + "<br />");
5  document.write(Math.min(0,150,30,20,38) + "<br />");
6  document.write(Math.min(-5,10) + "<br />");
7  document.write(Math.min(-5,-10) + "<br />");
8  document.write(Math.min(1.5,2.5));
9  </script>
10 </body>
11 </html>

```

```

1  <html>
2  <body>
3  <script>
4  document.write(Math.min(5,10) + "<br />");
5  document.write(Math.min(0,150,30,20,38) + "<br />");
6  document.write(Math.min(-5,10) + "<br />");
7  document.write(Math.min(-5,-10) + "<br />");
8  document.write(Math.min(1.5,2.5));
9  </script>
10 </body>
11 </html>

```

Exemplul 3

Convertirea gradelor din Celsius în Fahrenheit și reciproc:

```

1  <html>
2  <head>
3  <script>
4  function convert(degree)
5  {
6  if (degree=="C")
7  {
8  {
9  F=document.getElementById("c").value * 9 / 5 + 32;

```

```

10 document.getElementById("f").value=Math.round(F);
11 }
12 else
13 {
14 C=(document.getElementById("f").value -32) * 5 / 9;
15 document.getElementById("c").value=Math.round(C);
16 }
17 }
18 </script>
19 </head>
20 <body>
21 <p><b>Introduceti un numar in unul din cele doua campuri:</b></p>
22 <form>
23 <input id="c" name="c" onkeyup="convert('C')"> grade Celsius<br />
24 egal<br />
25 <input id="f" name="f" onkeyup="convert('F')"> grade Fahrenheit
26 </form>
27 </body>
</html>

```

```

1 <html>
2 <head>
3 <script>
4 function convert(degree)
5 {
6   if (degree=="C")
7   {
8     F=document.getElementById("c").value * 9 / 5 + 32;
9     document.getElementById("f").value=Math.round(F);
10  }
11  }
12  else
13  {
14    C=(document.getElementById("f").value -32) * 5 / 9;
15    document.getElementById("c").value=Math.round(C);
16  }
17  }
18 </script>
19 </head>
20 <body>
21 <p><b>Introduceti un numar in unul din cele doua campuri:</b></p>
22 <form>
23 <input id="c" name="c" onkeyup="convert('C')"> grade Celsius<br />
24 egal<br />
25 <input id="f" name="f" onkeyup="convert('F')"> grade Fahrenheit
26 </form>
27 </body>
</html>

```

Obiectul RegExp

Obiectul **RegExp** este folosit pentru a realiza căutări și înlocuiri într-un text. **RegExp** este prescurtarea pentru *expresie regulată*. Când realizați căutări într-un text, puteți defini modele de căutare cu ajutorul obiectului **RegExp**.

Un model simplu poate fi un singur caracter. Un model mai complicat conține mai multe caractere și poate fi utilizat pentru a analiza, verifica formatul, înlocui etc. O expresie regulată este un obiect care descrie modelul căutat în text.

Definire

Un obiect **RegExp** poate fi definit cu una din următoarele forme:

var txt=new RegExp(pattern,modifiers);

sau, mai simplu:

var txt=/pattern/modifiers;

pattern specifică modelul căutat
modifiers specifică dacă cătarea este globală, case-senzitivă etc.
Următoarea linie de cod definește un obiect **RegExp** numit **m1** cu modelul **"d"**:
var m1=new RegExp("d");
Cînd folosiți obiectul **m1** ca să căutați într-un șir, va fi găsită litera **"d"**.

Modificatorii

Modificator	Descriere
i	Caută potrivirea fără a face diferența între literele mici și mari
g	Realizează o căutare globală (găsește toate potrivirile, nu numai prima)
m	Caută potrivirea pe mai multe linii

Parantezele pătrate

Sunt utilizate pentru a defini un șir de caractere.

Expression	Descriere
[abc]	Găsește orice caracter precizat între paranteze
[^abc]	Găsește orice caracter diferit de cele precizate
[0-9]	Găsește o cifră între 0 și 9
[a-z]	Găsește orice literă mică
[A-Z]	Găsește orice literă mare
[a-Z]	Găsește orice literă, mare sau mică
[red blue green]	Găsește oricare dintre alternativele specificate

Metacaracterele

Sunt caracterele care au o semnificație specială:

Metacaracter	Descriere
.	Găsește un singur caracter (orice caracter diferit de linie nouă și sfîrșit de linie).
\w	Caută un caracter de cuvînt (litere mici sau mari, cifre și underscore)
\W	Găsește un caracter care nu este de cuvînt
\d	Găsește o cifră
\D	Găsește un caracter care nu este cifră
\s	Caută un spațiu alb
\S	Caută un caracter diferit de spațiu
\b	Caută o potrivire la începutul/sfîrșitul unui cuvînt

\B	Caută o potrivire care nu este la începutul/sfârșitul unui cuvânt
\0	Caută un caracter NUL
\n	Caută un caracter linie nouă
\f	Caută un caracter form feed
\r	Caută un caracter retur de car
\t	Caută un caracter tab
\v	Caută un tab caracter

Cuantificatori

Cuantificator	Descriere
n+	Caută orice șir care conține cel puțin un caracter n
n*	Caută orice șir care conține 0 sau mai multe apariții ale caracterului n
n?	Caută orice șir care conține 0 sau o apariție a caracterului n
n{X}	Caută orice șir care conține o secvență de X caractere n
n{X,Y}	Caută orice șir care conține o secvență de X sau Y caractere n
n{X,}	Caută orice șir care conține o secvență de cel puțin X caractere n
n\$	Caută orice șir care se incheie cu caracterul n
^n	Caută orice șir care începe cu caracterul n
?=n	Caută orice șir care este urmat de șirul n
?!n	Caută orice șir care nu este urmat de șirul n

Proprietățile obiectului RegExp

Proprietate	Descriere
global	Specifică dacă modificatorul "g" este setat
ignoreCase	Specifică dacă modificatorul "i" este setat
lastIndex	Specifică indexul de la care începe căutarea următoarei potriviri
multiline	Specifică dacă modificatorul "m" este setat
source	Textul din modelul RegExp

Metodele obiectului RegExp

Obiectul **RegExp** are trei metode: **test()**, **exec()** și **compile()**.

Metoda test()

Caută într-un șir un model și returnează **true** sau **false**.

Exemplu:

```

1  <html>
2  <body>
3  <script>
4  var m1=new RegExp("e");
5  document.write(m1.test("Cele mai frumoase carti le pastrez in amintire"));
6  </script>
7  </body>
8  </html>

```

```

1  <html>
2  <body>
3  <script>
4  var m1=new RegExp("e");
5  document.write(m1.test("Cele mai frumoase carti le pastrez in amintire"));
6  </script>
7  </body>
8  </html>

```

Deoarece modelul **"e"** apare în șir, metoda va returna valoarea **true** care va fi afișată.

Metoda exec()

Caută în text un model și returnează modelul, dacă acesta este găsit, sau valoarea **null**, dacă modelul nu apare în text.

Exemplu:

```

1  <html>
2  <body>
3  <script>
4  var m1=new RegExp("e");
5
6  document.write(m1.exec("Cele mai frumoase carti le pastrez in amintire"));
7  </script>
8  </body>
9  </html>

```

```

1  <html>
2  <body>
3  <script>
4  var m1=new RegExp("e");
5
6  document.write(m1.exec("Cele mai frumoase carti le pastrez in amintire"));
7  </script>
8  </body>
9  </html>

```

Deoarece modelul **"e"** apare în șir, metoda va returna valoarea **e** care va fi afișată.

Puteți adăuga al doilea parametru obiectului **RegExp**, pentru a specifica modul de căutare. Spre exemplu, dacă doriți să găsiți toate aparițiile unui caracter, puteți folosi parametrul **"g"** (**"global"**).

Cînd utilizați parametrul **"g"**, metoda **exec()** lucrează astfel:

Găsește prima apariție a modelului **"e"** și îi memorează poziția

Dacă executați din nou metoda **exec()**, căutarea va începe de la poziția memorată anterior ș.a.m.d.

Exemplu:

```

1  <html>
2

```

```

3   <body>
4   <script>
5   var m1=new RegExp("e","g");
6   do
7   {
8   result=m1.exec("Cele mai frumoase carti le pastrez in amintire");
9   document.write(result);
10  }
11  while (result!=null)
12  </script>
13  </body>
    </html>

```

```

1   <html>
2   <body>
3   <script>
4   var m1=new RegExp("e","g");
5   do
6   {
7   result=m1.exec("Cele mai frumoase carti le pastrez in amintire");
8   document.write(result);
9   }
10  while (result!=null)
11  </script>
12  </body>
13  </html>

```

Deoarece modelul "e" apare de șase ori în text, programul de mai sus va afișa secvența: **eeeeeenull**

Metoda compile()

Este utilizată pentru a modifica conținutul obiectului **RegExp**.

Metoda poate schimba modelul căutat și poate adăuga sau elimina al doilea parametru.

Exemplu:

```

1   <html>
2   <body>
3   <script>
4   var m1=new RegExp("e");
5   document.write(m1.test("Cele mai frumoase carti le pastrez in amintire"));
6   m1.compile("d");
7   document.write(m1.test("Cele mai frumoase carti le pastrez in amintire"));
8   </script>
9   </body>
10  </html>

```

```

1   <html>
2   <body>
3   <script>
4   var m1=new RegExp("e");
5   document.write(m1.test("Cele mai frumoase carti le pastrez in amintire"));
6   m1.compile("d");
7   document.write(m1.test("Cele mai frumoase carti le pastrez in amintire"));
8   </script>
9   </body>
10  </html>

```

Deoarece modelul "e" apare în șir, dar modelul "d" nu apare, programul anterior va afișa valorile: **truefalse**

Exemple:

Exemplul 1

Ilustrează utilizarea metodei **match()** a obiectului **String** pentru a găsi toate caracterele precizate în model cu ajutorul parantezelor pătrate.

```
1 <html>
2 <body>
3 <script>
4   var str="Ce mai faci?";
5   var m1=/[a-h]/g;
6   document.write(str.match(m1));
7 </script>
8 </body>
9 </html>
```

```
1 <html>
2 <body>
3 <script>
4   var str="Ce mai faci?";
5   var m1=/[a-h]/g;
6   document.write(str.match(m1));
7 </script>
8 </body>
9 </html>
```

Programul va afișa rezultatul: **e,a,f,a,c**

Exemplul 2

Ilustrează utilizarea metodei **match()** a obiectului **String** pentru a găsi toate caracterelor diferite de cele din model.

```
1 <html>
2 <body>
3 <script>
4   var str="Ce mai faci?";
5   var m1=/[^a-h]/g;
6   document.write(str.match(m1));
7 </script>
8 </body>
9 </html>
```

```
1 <html>
2 <body>
3 <script>
4   var str="Ce mai faci?";
5   var m1=/[^a-h]/g;
6   document.write(str.match(m1));
7 </script>
8 </body>
9 </html>
```

Programul anterior va afișa rezultatul: **C, ,m,i, ,i,?**

Exemplul 3

Ilustrează cum se poate construi un model care să găsească toate secvențele în care un caracter poate avea orice valoare.

```
1 <html>
2 <body>
3 <script>
4   var str="Pisica nu are blana tarcata";
5   var m1=/a.a/g;
6   document.write(str.match(m1));
7 </script>
```

```

9 | </body>
   | </html>

1 |
2 | <html>
3 | <body>
4 | <script>
5 | var str="Pisica nu are blana tarcata";
6 | var m1=/a.a/g;
7 | document.write(str.match(m1));
8 | </script>
9 | </body>
   | </html>

```

Programul va găsi toate ecvențele de trei caractere în care primul și ultimul caracter este „a”.

Rezultatul afișat pentru șirul de mai sus este: **ana,ata**

Exemplul 4

Ilustrează cum se poate construi un model care să găsească toate caracterele care nu fac parte dintr-un cuvânt.

```

1 |
2 | <html>
3 | <body>
4 | <script>
5 | var str="Ai obtinut 75%!";
6 | var m1=/\W/g;
7 | document.write(str.match(m1));
8 | </script>
9 | </body>
   | </html>

1 |
2 | <html>
3 | <body>
4 | <script>
5 | var str="Ai obtinut 75%!";
6 | var m1=/\W/g;
7 | document.write(str.match(m1));
8 | </script>
9 | </body>
   | </html>

```

Programul anterior va afișa rezultatul: , ,%,!

Exemplul 5

Ilustrează cum se poate construi un model cu care să înceapă sau să se sfirșească un cuvânt.

```

1 |
2 | <html>
3 | <body>
4 | <script>
5 | var str="Vizitati Google";
6 | var m1=/\bGo/g;
7 | document.write(str.match(m1));
8 | </script>
9 | </body>
   | </html>

1 |
2 | <html>
3 | <body>
4 | <script>
5 | var str="Vizitati Google";
6 | var m1=/\bGo/g;
7 | document.write(str.match(m1));

```

```

8 </script>
9 </body>
  </html>

```

Dacă nu este găsit nici-un cuvânt care începe sau se sfârșește cu modelul dat, metoda **match()** va returna valoarea **null**. Pentru exemplul considerat anterior, există în text un cuvânt care se potrivește și metoda returnează modelul.

Rezultatul afișat este: **Go**

Exemplul 6

Ilustrează cum se pot găsi toate secvențele dintr-un text, în care un anumit caracter apare cel puțin o dată.

```

1
2 <html>
3 <body>
4 <script>
5 var str="Tu creezi pagini web interesante!";
6 var m1=/e+/g;
7 do
8 {
9 result=m1.exec(str);
10 document.write(result); document.write(" ");
11 }
12 while(result!=null)
13 </script>
14 </body>
  </html>

```

```

1
2 <html>
3 <body>
4 <script>
5 var str="Tu creezi pagini web interesante!";
6 var m1=/e+/g;
7 do
8 {
9 result=m1.exec(str);
10 document.write(result); document.write(" ");
11 }
12 while(result!=null)
13 </script>
14 </body>
  </html>

```

Programul anterior va determina toate secvențele din text în care caracterul „e” apare cel puțin o dată (în poziții consecutive).

Rezultatul afișat de program este: **ee e e e e null**

Exemplul 7

Ilustrează cum se pot găsi secvențele de text în care un anumit caracter apare de minim x ori.

```

1
2 <html>
3 <body>
4 <script>
5 var str="Aveti 10, 100, 1000 sau 10000 de lei?";
6 var m1=/\d{3,}/g;
7 document.write(str.match(m1));
8 </script>
9 </body>
  </html>

```

```

1

```

```
2 <html>
3 <body>
4 <script>
5   var str="Aveti 10, 100, 1000 sau 10000 de lei?";
6   var m1=/d{3,}/g;
7   document.write(str.match(m1));
8 </script>
9 </body>
</html>
```

Programul anterior afișează rezultatul: **100,1000,10000**, adică secvențele care conțin cel puțin trei cifre zecimale.

Exemplul 8

Ilustrează cum se pot găsi toate subșirurile dintr-un text, care sunt urmate de un subșir dat.

```
1 <html>
2 <body>
3 <script>
4   var str="eu am o pisica, dar eu am și un papagal";
5   var m1=/eu(?= am)/g;
6   document.write(str.match(m1));
7 </script>
8 </body>
9 </html>
```

```
1 <html>
2 <body>
3 <script>
4   var str="eu am o pisica, dar eu am și un papagal";
5   var m1=/eu(?= am)/g;
6   document.write(str.match(m1));
7 </script>
8 </body>
9 </html>
```

Programul anterior determină toate șirurile „eu” care sunt urmate de șirul „am”.

Rezultatul afișat este: **eu,eu**

Obiectul Number

Obiectul **Number** este un container pentru valorile numerice de bază.

Obiectele **Number** sunt create cu următoarea sintaxă:

var nume = new Number(valoare);

Obs: Dacă parametrul **valoare** nu poate fi convertit într-un număr, va fi returnată valoarea **NaN (Not-a-Number)**.

Proprietățile obiectului Number

Proprietate	Descriere
constructor	Returnează funcția care a creat prototipul obiectului
MAX_VALUE	Returnează cel mai mare număr posibil în JavaScript
MIN_VALUE	Returnează cel mai mic număr posibil în JavaScript
NEGATIVE_INFINITY	Reprezintă infinitul negativ (returnat la depășire)
POSITIVE_INFINITY	Reprezintă infinitul pozitiv (returnat la depășire)

prototype	Permite adăugarea de proprietăți și metode
-----------	--

Metodele obiectului Number

Metodă	Descriere
toExponential(x)	Convertește numărul într-o notație exponențială
toFixed(x)	Formează un număr cu x cifre după virgulă
toPrecision(x)	Formează un număr cu lungimea x
toString()	Reprezintă infinitul negativ (returnat la depășire)
POSITIVE_INFINITY	Convertește un obiect Number în șir de caractere. Dacă metoda are parametru, acesta precizează baza în care este reprezentat numărul convertit în șir.
valueOf()	Returnează valoarea primară a obiectului

Exemple:

Exemplul 1

Ilustrează cum se afișează cel mai mare număr din JavaScript.

```

1  <html>
2  <body>
3  <script>
4  document.write(Number.MAX_VALUE);
5  </script>
6  </body>
7  </html>
    
```

```

1  <html>
2  <body>
3  <script>
4  document.write(Number.MAX_VALUE);
5  </script>
6  </body>
7  </html>
    
```

Exemplul 2

Ilustrează cum se stabilește numărul de zecimale afișate.

```

1  <html>
2  <body>
3  <script>
4  var num = new Number(27.2547);
5  document.write(num.toFixed()+"<br />");
6  document.write(num.toFixed(1)+"<br />");
7  document.write(num.toFixed(3)+"<br />");
8  document.write(num.toFixed(10));
9  </script>
10 </body>
11 </html>
    
```

1

```

2  <html>
3  <body>
4  <script>
5  var num = new Number(27.2547);
6  document.write(num.toFixed()+"<br />");
7  document.write(num.toFixed(1)+"<br />");
8  document.write(num.toFixed(3)+"<br />");
9  document.write(num.toFixed(10));
10 </script>
11 </body>
    </html>

```

Exemplul 3

Ilustrează cum se stabilește precizia numărului afișat (numărul total de cifre afișate).

```

1  <html>
2  <body>
3  <script>
4  var num = new Number(31.1593);
5  document.write(num.toPrecision()+"<br />");
6  document.write(num.toPrecision(2)+"<br />");
7  document.write(num.toPrecision(3)+"<br />");
8  document.write(num.toPrecision(10));
9  </script>
10 </body>
11 </html>

```

```

1  <html>
2  <body>
3  <script>
4  var num = new Number(31.1593);
5  document.write(num.toPrecision()+"<br />");
6  document.write(num.toPrecision(2)+"<br />");
7  document.write(num.toPrecision(3)+"<br />");
8  document.write(num.toPrecision(10));
9  </script>
10 </body>
11 </html>

```

Exemplul 4

Ilustrează cum se convertește în șir un număr, folosind diferite baze de numerație.

```

1  <html>
2  <body>
3  <script>
4  var num=new Number(31);
5
6  document.write(num.toString()+"<br />");    //numarul este reprezentat in baza 10 (implicit)
7
8  document.write(num.toString(2)+"<br />");    //numarul este reprezentat in baza 2
9
10 document.write(num.toString(8)+"<br />");    //numarul este reprezentat in baza 8
11
12 document.write(num.toString(16)+"<br />");    //numarul este reprezentat in baza 16
13 </script>
14 </body>
15 </html>

```

```

1  <html>
2  <body>
3  <script>
4  var num=new Number(31);

```

```

6
7 document.write(num.toString()+"<br />"); //numarul este reprezentat in baza 10 (implicit)
8
9 document.write(num.toString(2)+"<br />"); //numarul este reprezentat in baza 2
10
11 document.write(num.toString(8)+"<br />"); //numarul este reprezentat in baza 8
12
13 document.write(num.toString(16)+"<br />"); //numarul este reprezentat in baza 16
14 </script>
15 </body>
    </html>

```

Obiectul Navigator

Obiectul **Navigator** conține informații despre browserul vizitatorului. Aproape orice exemplu din acest curs funcționează în browserele care recunosc JavaScript. Totuși, unele exemple nu funcționează în anumite browsere, în special în cele vechi. De aceea, uneori este foarte util să determinați browserul utilizat de vizitator pentru a-i putea furniza informațiile potrivite. Cea mai bună metodă este să vă proiectați paginile web să se comporte diferit, în funcție de browserul vizitatorului. Obiectul **Navigator** poate fi utilizat în acest scop, deoarece conține informații despre numele și versiunea browserului vizitatorului.

Proprietățile obiectului Navigator

Proprietate	Descriere
appCodeName	Returnează codul browserului
appName	Returnează numele browserului
appVersion	Returnează informații despre versiunea browserului
cookieEnabled	Determină dacă are cookies activate
platform	Returnează pentru ce platformă este compilat browserul

Exemple:

Exemplul 1

Variabila "**browser**" din exemplul următor memorează numele browserului. Proprietatea **appVersion** returnează un șir de caractere care conține mult mai multe informații, nu numai numărul versiunii. Deoarece ne interesează numai versiunea, pentru a o extrage din șir, se utilizează o funcție numită **parseFloat()**, care extrage din șir și returnează prima secvență care arată ca un număr zecimal.

```

1
2 <html>
3 <body>
4 <script>
5 var browser=navigator.appName;
6 var b_ver=navigator.appVersion;
7 var versiune=parseFloat(b_ver);
8
9 document.write("Numele browserului: "+ browser);
10 document.write("<br />");
11 document.write("Versiunea browserului: "+ versiune);
12 </script>
13 </body>
    </html>

```

```

2  <html>
3  <body>
4  <script>
5  var browser=navigator.appName;
6  var b_ver=navigator.appVersion;
7  var versiune=parseFloat(b_ver);
8
9  document.write("Numele browserului: "+ browser);
10 document.write("<br />");
11 document.write("Versiunea browserului: "+ versiune);
12 </script>
13 </body>
   </html>

```

Exemplul 2

Ilustrează cum pot fi afișate diferite mesaje, în funcție de tipul și versiunea browserului.

```

1
2  <html>
3  <head>
4  <script>
5  function detectBrowser()
6  {
7  var browser=navigator.appName;
8  var b_ver=navigator.appVersion;
9  var versiune=parseFloat(b_ver);
10 if ((browser=="Netscape"||browser=="Microsoft Internet Explorer") && (versiune>=4))
11 {
12 alert("Browserul dvs. este destul de bun!");
13 }
14 else
15 {
16 alert("Este timpul sa va upgradati browserul!");
17 }
18 }
19 </script>
20 </head>
21 <body onload="detectBrowser()">
22 </body>
   </html>

```

```

1
2  <html>
3  <head>
4  <script>
5  function detectBrowser()
6  {
7  var browser=navigator.appName;
8  var b_ver=navigator.appVersion;
9  var versiune=parseFloat(b_ver);
10 if ((browser=="Netscape"||browser=="Microsoft Internet Explorer") && (versiune>=4))
11 {
12 alert("Browserul dvs. este destul de bun!");
13 }
14 else
15 {
16 alert("Este timpul sa va upgradati browserul!");
17 }
18 }
19 </script>
20 </head>
21 <body onload="detectBrowser()">
22 </body>
   </html>

```

Exemplul 3

Ilustrează cum pot fi afișate mai multe detalii despre browserul vizitatorului

```

1  <html>
2  <body>
3  <script>
4  document.write("<p>Browser: ");
5  document.write(navigator.appName + "</p>");
6  document.write("<p>Versiune: ");
7  document.write(navigator.appVersion + "</p>");
8  document.write("<p>Cod: ");
9  document.write(navigator.appCodeName + "</p>");
10 document.write("<p>Platforma: ");
11 document.write(navigator.platform + "</p>");
12 document.write("<p>Cookies activate: ");
13 document.write(navigator.cookieEnabled + "</p>");
14
15 </script>
16 </body>
17 </html>

```

```

1  <html>
2  <body>
3  <script>
4  document.write("<p>Browser: ");
5  document.write(navigator.appName + "</p>");
6  document.write("<p>Versiune: ");
7  document.write(navigator.appVersion + "</p>");
8  document.write("<p>Cod: ");
9  document.write(navigator.appCodeName + "</p>");
10 document.write("<p>Platforma: ");
11 document.write(navigator.platform + "</p>");
12 document.write("<p>Cookies activate: ");
13 document.write(navigator.cookieEnabled + "</p>");
14
15 </script>
16 </body>
17 </html>

```

Obiectele browserului

Obiectul window

Acest obiect reprezintă o fereastră deschisă în browser. Dacă conține cadre (tagurile <frame> sau <iframe>), browserul creează un obiect window pentru documentul HTML, și câte un obiect window pentru fiecare cadru.

Proprietățile obiectului window

Proprietate	Descriere
closed	Returnează o valoare booleană care indică dacă fereastra a fost închisă sau nu
defaultStatus	Setează sau returnează textul implicit din bara de stare a ferestrei
document	Returnează obiectul Document al ferestrei
frames	Returnează un tablou cu toate cadrele din fereastra curentă
history	Returnează obiectul History al ferestrei
innerHeight	Setează sau returnează înălțimea interioară a zonei de conținut a ferestrei
innerWidth	Setează sau returnează lățimea interioară a zonei de conținut a ferestrei

length	Returnează numărul de cadre (inclusiv cele inline) din fereastră
location	Returnează obiectul Location al ferestrei
name	Setează sau returnează numele ferestrei
opener	Returnează referința care a creat fereastra
outerHeight	Setează sau returnează înălțimea exterioară a ferestrei (inclusiv toolbars/scrollbars)
outerWidth	Setează sau returnează lățimea exterioară a ferestrei (inclusiv toolbars/scrollbars)
pageXOffset	Returnează numărul de pixeli cu care documentul curent a fost derulat orizontal, în raport cu colțul stnga sus al ferestrei
pageYOffset	Returnează numărul de pixeli cu care documentul curent a fost derulat vertical, în raport cu colțul stnga sus al ferestrei
parent	Returnează fereastra părinte a ferestrei curente
screenLeft	Returnează coordonata x a ferestrei, relativ la ecran
screenTop	Returnează coordonata y a ferestrei, relativ la ecran
screenX	Returnează coordonata x a ferestrei, relativ la ecran
screenY	Returnează coordonata y a ferestrei, relativ la ecran
self	Returnează fereastra curentă
status	Setează textul din bara de stare a ferestrei
top	Returnează cea mai din vîrf fereastră deschisă în browser

Metodele obiectului window

Metodă	Descriere
alert()	Afișează o casetă de alertare care conține un mesaj și un buton OK
blur()	Îndepărtează focus-ul de la fereastra curentă
clearInterval()	Resetează timer-ul setat cu setInterval()
clearTimeout()	Resetează timer-ul setat cu setTimeout()
close()	Închide fereastra curentă
confirm()	Afișează o casetă de dialog care conține un mesaj și butoanele OK și Cancel
createPopup()	Creează o fereastră Pop-up
focus()	Fixează focus-ul pe fereastra curentă
moveBy()	Mută fereastra, relativ la poziția ei curentă

moveTo()	Mută fereastra într-o nouă poziție
open()	Deschide o nouă fereastră în browse
print()	Tipărește conținutul ferestrei curente
prompt()	Afișează o casetă de dialog care cere utilizatorului să introducă anumite informații
resizeBy()	Redimensionează fereastra la dimensiunea specificată în pixeli
resizeTo()	Redimensionează fereastra la înălțimea și lățimea specificate
scrollBy()	Derulează conținutul ferestrei cu numărul specificat de pixeli
scrollTo()	Derulează conținutul ferestrei până la coordonatele specificate
setInterval()	Apelează o funcție sau evaluează o expresie la intervale specificate de timp (în milisecunde)
setTimeout()	Apelează o funcție sau evaluează o expresie după un număr specificat de milisecunde

Exemplul 1

Ilustrează utilizarea metodelor **open()** și **focus()**

```

1  <html>
2  <head>
3  <script>
4  function deschide()
5  {
6  myWindow=window.open('', '', 'width=200,height=100');
7  myWindow.document.write("<p>Aceasta este o fereastră creată cu metoda open()</p>");
8  myWindow.focus();
9  }
10 </script>
11 </head>
12 <body>
13 <input type="button" value="Deschide fereastră" onclick="deschide()" />
14 </body>
15 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function deschide()
5  {
6  myWindow=window.open('', '', 'width=200,height=100');
7  myWindow.document.write("<p>Aceasta este o fereastră creată cu metoda open()</p>");
8  myWindow.focus();
9  }
10 </script>
11 </head>
12 <body>
13 <input type="button" value="Deschide fereastră" onclick="deschide()" />
14 </body>
15 </html>

```

Exemplul 2

În acest exemplu, funcția **clock()** este apelată la fiecare 1000 milisecunde și actualizează ceasul afișat. Ceasul poate fi oprit prin apăsarea unui buton

```

1  <html>
2  <body>
3  <input type="text" id="clock" />
4  <script language=javascript>
5  var int=self.setInterval("clock()",1000);
6  function clock()
7  {
8  var d=new Date();
9  var t=d.toLocaleTimeString();
10 document.getElementById("clock").value=t;
11 }
12 </script>
13 </form>
14 <button onclick="int=window.clearInterval(int)">Stop</button>
15 </body>
16 </html>

```

```

1  <html>
2  <body>
3  <input type="text" id="clock" />
4  <script language=javascript>
5  var int=self.setInterval("clock()",1000);
6  function clock()
7  {
8  var d=new Date();
9  var t=d.toLocaleTimeString();
10 document.getElementById("clock").value=t;
11 }
12 </script>
13 </form>
14 <button onclick="int=window.clearInterval(int)">Stop</button>
15 </body>
16 </html>

```

Exemplul 3

Ilustrează mutarea ferestrei curente cu 250 pixeli relativ la poziția ei curentă

```

1  <html>
2  <head>
3  <script>
4  function deschide()
5  {
6  myWindow=window.open('', '', 'width=200,height=100');
7  myWindow.document.write("<p>Aceasta este o fereastră deschisa cu open()</p>");
8  }
9  function muta()
10 {
11 myWindow.moveBy(250,250);
12 myWindow.focus();
13 }
14 </script>
15 </head>
16 <body>
17 <input type="button" value="Deschide fereastră" onclick="deschide()" />
18 <br /><br />
19 <input type="button" value="Muta fereastră" onclick="muta()" />
20 </body>
21 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function deschide()
5  {
6

```



```
7 myWindow=window.open('', '', 'width=200,height=100');
8 myWindow.document.write("<p>Aceasta este o fereastră deschisa cu open()</p>");
9 }
10 function muta()
11 {
12 myWindow.moveBy(250,250);
13 myWindow.focus();
14 }
15 </script>
16 </head>
17 <body>
18 <input type="button" value="Deschide fereastră" onclick="deschide()" />
19 <br /><br />
20 <input type="button" value="Muta fereastră" onclick="muta()" />
21 </body>
22 </html>
```

Exemplul 4

Ilustrează redimensionarea ferestrei curente

```
1 <html>
2 <head>
3 <script>
4 function redimensioneaza()
5 {
6 top.resizeTo(500,300);
7 }
8 </script>
9 </head>
10 <body>
11 <form>
12 <input type="button" onclick="redimensioneaza()" value="Redimensioneaza fereastră" />
13 </form>
14 </body>
15 </html>
```

```
1 <html>
2 <head>
3 <script>
4 function redimensioneaza()
5 {
6 top.resizeTo(500,300);
7 }
8 </script>
9 </head>
10 <body>
11 <form>
12 <input type="button" onclick="redimensioneaza()" value="Redimensioneaza fereastră" />
13 </form>
14 </body>
15 </html>
```

Exemplul 5

Ilustrează utilizarea metodei **blur()** pentru a trimite o fereastră în background.

```
1 <html>
2 <head>
3 <script>
4 function deschide()
5 {
6 myWindow=window.open('', '', 'width=200,height=100');
7 myWindow.document.write("<p>Aceasta fereastră este deschisa cu open()</p>");
8 myWindow.blur();
9 }
10
```

```

11 </script>
12 </head>
13 <body>
14 <input type="button" value="Deschide fereastra" onclick="deschide()" />
15 </body>
</html>

1 <html>
2 <head>
3 <script>
4 function deschide()
5 {
6   myWindow=window.open('', '', 'width=200,height=100');
7   myWindow.document.write("<p>Aceasta fereastra este deschisa cu open()</p>");
8   myWindow.blur();
9 }
10 </script>
11 </head>
12 <body>
13 <input type="button" value="Deschide fereastra" onclick="deschide()" />
14 </body>
15 </html>

```

Obiectul screen

Acest obiect conține informații despre ecranul vizitatorului.

Proprietățile obiectului screen

Proprietate	Descriere
availHeight	Returnează înălțimea ecranului (fără Taskbar)
availWidth	Returnează lățimea ecranului (fără Taskbar)
colorDepth	Returnează numărul de biți din paleta de culori folosită pentru afișarea imaginilor
height	Returnează înălțimea totală a ecranului
pixelDepth	Returnează rezoluția culorii ecranului (în biți/pixel)
width	Returnează lățimea totală a ecranului

Exemplu următor ilustrează utilizarea tuturor proprietăților obiectului screen pentru a obține informații despre ecranul vizitatorului:

```

1 <html>
2 <body>
3 <h3>Ecranul dumneavoastra are proprietatile:</h3>
4 <script>
5 document.write("Latime/Inaltime totala: ");
6 document.write(screen.width + "*" + screen.height);
7 document.write("<br />");
8 document.write("Latime/Inaltime disponibila: ");
9 document.write(screen.availWidth + "*" +
10 screen.availHeight);
11 document.write("<br />");
12 document.write("Numarul de biti ai culorii: ");
13 document.write(screen.colorDepth);
14 document.write("<br />");
15 document.write("Rezoluția culorii: ");
16 document.write(screen.pixelDepth);
17

```

```

18 </script>
19 </body>
    </html>

1 <html>
2 <body>
3 <h3>Ecranul dumneavoastra are proprietatile:</h3>
4 <script>
5 document.write("Latime/Inaltime totala: ");
6 document.write(screen.width + "*" + screen.height);
7 document.write("<br />");
8 document.write("Latime/Inaltime disponibila: ");
9 document.write(screen.availWidth + "*" +
10 screen.availHeight);
11 document.write("<br />");
12 document.write("Numarul de biti ai culorii: ");
13 document.write(screen.colorDepth);
14 document.write("<br />");
15 document.write("Rezoluția culorii: ");
16 document.write(screen.pixelDepth);
17 </script>
18 </body>
19 </html>
    
```

Obiectul history

Acest obiect conține URL-urile vizitate de utilizator (într-o fereastră de browser). Obiectul **history** face parte din obiectul **window** și este accesat prin proprietatea **window.history**.

Proprietățile obiectului history

Proprietate	Descriere
length	Returnează numărul de URL-uri din lista history

Metodele obiectului history

Metodă	Descriere
back()	Încarcă URL-ul anterior din lista history
forward()	Încarcă URL-ul următor din lista history
go()	Încarcă un anumit URL din lista history

Obiectul location

Obiectul **location** conține informații despre URL-ul curent. Obiectul **location** este parte a obiectului **window** și este accesat prin intermediul proprietății **window.location**.

Proprietățile obiectului location

Proprietate	Descriere
hash	Returnează porțiunea de ancoră din URL
host	Returnează hostname-ul și port-ul URL-ului
hostname	Returnează hostname-ul URL-ului
href	Returnează întregul URL

pathname	Returnează numele căii URL-ului
port	Returnează numărul de port pe care serverul îl utilizează pentru URL
protocol	Returnează protocolul URL-ului
search	Returnează porțiunea query din URL

Metodele obiectului location

Metodă	Descriere
assign()	Încarcă un nou document
reload()	Reincarcă documentul curent
replace()	Înlocuiește documentul curent cu un alt document

Exemplu

Ilustrează utilizarea metodei **assign()**

```

1  <html>
2  <head>
3  <script>
4  function nou()
5  {
6  window.location.assign("http://www.google.com")
7  }
8  </script>
9  </head>
10 <body>
11 <input type="button" value="Incarca noul document" onclick="nou()" />
12 </body>
13 </html>

```

```

1  <html>
2  <head>
3  <script>
4  function nou()
5  {
6  window.location.assign("http://www.google.com")
7  }
8  </script>
9  </head>
10 <body>
11 <input type="button" value="Incarca noul document" onclick="nou()" />
12 </body>
13 </html>

```

Cookies

Un cookie este o variabilă păstrată în calculatorul vizitatorului. De fiecare dată când calculatorul respectiv cere browserului o pagină, el va trimite și cookie-ul respectiv. Cu JavaScript, puteți crea și extrage cookies.

Exemple de cookies:

1. **Numele utilizatorului** - Prima dată când utilizatorul vă vizitează pagina trebuie să-și completeze numele. Numele este stocat într-un cookie. Următoarea dată când vizitatorul ajunge la pagina dvs., puteți să-l întâmpinați cu un mesaj de genul "Bine ai venit.....!" Numele este recuperat dintr-un cookie.

2. Parolă - Prima dată utilizatorul vă vizitează pagina trebuie să completeze o parolă. Parola este memorată într-un cookie. Data viitoare când vizitatorul ajunge în pagină, parola poate fi recuperată dintr-un cookie.

3. Data calendaristică - Prima dată când utilizatorul vă vizitează pagina, data curentă este memorată într-un cookie. Data viitoare când utilizatorul vă vizitează pagina, puteți să afișați un mesaj de genul *"Ultima dvs. vizita a fost in data de"*. Această dată este recuperată dintr-un cookie.

Crearea și memorarea unui cookie

În acest exemplu vom crea un cookie care memorează numele vizitatorului, apoi vom folosi numele memorat în variabila cookie pentru a afișa un mesaj de bun venit. Prima dată vom crea o funcție care memorează numele vizitatorului într-o variabilă cookie:

```

1 function setCookie(c_name,value,expiredays)
2 {
3   var exdate=new Date();
4   exdate.setDate(exdate.getDate()+expiredays);
5   document.cookie=c_name+ "=" +escape(value)+ ((expiredays==null) ? "" :
6   ";expires="+exdate.toGMTString());
7 }

1 function setCookie(c_name,value,expiredays)
2 {
3   var exdate=new Date();
4   exdate.setDate(exdate.getDate()+expiredays);
5   document.cookie=c_name+ "=" +escape(value)+ ((expiredays==null) ? "" :
6   ";expires="+exdate.toGMTString());
7 }

```

Parametrii funcției reprezintă numele și valoarea cookie-ului și numărul de zile pînă când acesta expiră. Funcția convertește numărul de zile într-o dată validă și apoi adaugă numărul de zile după care va expira cookie-ul. Apoi, numele și valoarea cookie-ului și data expirării sunt memorate într-un obiect **document.cookie**.

În continuare, vom crea o funcție care verifică dacă cookie-ul a fost setat:

```

1 function getCookie(c_name)
2 {
3   if (document.cookie.length>0)
4   {
5     c_start=document.cookie.indexOf(c_name + "=");
6     if (c_start!=-1)
7     {
8       c_start=c_start + c_name.length+1;
9       c_end=document.cookie.indexOf(";",c_start);
10      if (c_end==-1) c_end=document.cookie.length;
11      return unescape(document.cookie.substring(c_start,c_end));
12    }
13  }
14  return "";
15 }

1 function getCookie(c_name)
2 {
3   if (document.cookie.length>0)
4   {
5     c_start=document.cookie.indexOf(c_name + "=");
6     if (c_start!=-1)
7     {
8       c_start=c_start + c_name.length+1;
9       c_end=document.cookie.indexOf(";",c_start);
10      if (c_end==-1) c_end=document.cookie.length;
11    }

```

```

12 | return unescape(document.cookie.substring(c_start,c_end));
13 | }
14 | }
15 | return "";
    | }

```

Funcția verifică mai întâi dacă în obiectul **document.cookie** este memorat vre-un cookie. În caz afirmativ, verificăm dacă este memorat cookie-ul dorit. Dacă cookie-ul este găsit, îi returnăm valoarea, în caz contrar returnăm un șir vid. În cele din urmă, vom crea funcția care afișează un mesaj de bun venit dacă cookie-ul este setat și o casetă prompt care cere numele vizitatorului, în caz contrar:

```

1 |
2 | function checkCookie()
3 | {
4 |   username=getCookie('username');
5 |   if (username!=null && username!="")
6 |   {
7 |     alert('Welcome again '+username+'!');
8 |   }
9 |   else
10 |  {
11 |    username=prompt('Please enter your name:', "");
12 |    if (username!=null && username!="")
13 |    {
14 |      setCookie('username',username,365);
15 |    }
16 |  }
    | }

```

```

1 |
2 | function checkCookie()
3 | {
4 |   username=getCookie('username');
5 |   if (username!=null && username!="")
6 |   {
7 |     alert('Welcome again '+username+'!');
8 |   }
9 |   else
10 |  {
11 |    username=prompt('Please enter your name:', "");
12 |    if (username!=null && username!="")
13 |    {
14 |      setCookie('username',username,365);
15 |    }
16 |  }
    | }

```

Programul complet este:

```

1 |
2 | <html>
3 | <head>
4 | <script>
5 | function getCookie(c_name)
6 | {
7 |   if (document.cookie.length>0)
8 |   {
9 |     c_start=document.cookie.indexOf(c_name + "=");
10 |    if (c_start!=-1)
11 |    {
12 |      c_start=c_start + c_name.length+1 ;
13 |      c_end=document.cookie.indexOf(";",c_start);
14 |      if (c_end==-1) c_end=document.cookie.length return
15 |      unescape(document.cookie.substring(c_start,c_end));
16 |    }
17 |  }
    | }

```

```

18  return ""
19  }
20
21  function setCookie(c_name,value,expiredays)
22  {
23  var exdate=new Date();
24  exdate.setDate(exdate.getDate()+expiredays);
25  document.cookie=c_name+ "=" +escape(value)+((expiredays==null) ? "" : ";
26  expires="+exdate.toGMTString());
27  }
28
29  function checkCookie()
30  {
31  username=getCookie('username');
32  if (username!=null && username!="")
33  {
34  alert('Bine ai revenit '+username+'!');
35  }
36  else
37  {
38  username=prompt('Va rog sa va introduceti numele:', "");
39  if (username!=null && username!="")
40  {
41  setCookie('username',username,365);
42  }
43  }
44  }
45  </script>
46  </head>
47  <body onLoad="checkCookie()">
  </body>
</html>

```

```

1
2  <html>
3  <head>
4  <script>
5  function getCookie(c_name)
6  {
7  if (document.cookie.length>0)
8  {
9  c_start=document.cookie.indexOf(c_name + "=");
10  if (c_start!=-1)
11  {
12  c_start=c_start + c_name.length+1 ;
13  c_end=document.cookie.indexOf(";",c_start);
14  if (c_end==-1) c_end=document.cookie.length return
15  unescape(document.cookie.substring(c_start,c_end));
16  }
17  }
18  return ""
19  }
20
21  function setCookie(c_name,value,expiredays)
22  {
23  var exdate=new Date();
24  exdate.setDate(exdate.getDate()+expiredays);
25  document.cookie=c_name+ "=" +escape(value)+((expiredays==null) ? "" : ";
26  expires="+exdate.toGMTString());
27  }
28
29  function checkCookie()
30  {
31  username=getCookie('username');
32  if (username!=null && username!="")
33  {
34  alert('Bine ai revenit '+username+'!');
35  }
36  else
37  {
38  username=prompt('Va rog sa va introduceti numele:', "");
39  if (username!=null && username!="")
40  {
41  setCookie('username',username,365);

```

```

41 }
42 }
43 }
44 }
45 </script>
46 </head>
47 <body onLoad="checkCookie()">
  <body>
</body>
</html>

```

Exemplul prezentat execută funcția **checkCookie()** cînd pagina se încarcă.

Obiectul Form

Formularele sunt elemente HTML care dau viața unor pagini statice deoarece asigură o interfață prin care vizitatorii pot interacționa, cu ajutorul controalelor (elemente ale formularului).

Obiectul Form este mijlocul prin care se poate interacționa cu acest element HTML în scripturile JavaScript.

Metode și proprietăți ale obiectului Form

Proprietățile obiectului Form

Proprietate	Descriere
action	Contine atributul "action" din eticheta <form>, specifică adresa URL unde este trimis formularul
elements	Matricea care conține toate elementele din <form>
encoding	Contine atributul "enctype" din <form>, specifică o codificare MIME a formularului
length	Numărul de elemente continute în formular method - conține atributul "method" din <form>, definește felul în care formularul este trimis serverului (get sau post)
name	Contine atributul "name" al unei etichetei <form>
target	Contine atributul "target" al etichetei <form>, specifică fereastra în care serverul ar trebui să returneze informațiile. Dacă nu este specificată, serverul afișează Rezultatele în fereastra care a expediat formularul

Metodele obiectului Form

Proprietate	Descriere
handleEvent()	Apelează handlerul de eveniment specificat
reset()	Reduce elementele formularului la valorile prestabilite
submit()	Declanșează un eveniment "submit" care trimite datele spre programul specificat în atributul "action" al etichetei <form>

Trimiterea formularelor spre server

Limbaul JavaScript permite prelucrarea datelor introduse în formular înainte ca acestea să fie trimise spre procesare la programul (scriptul) de pe server. Puteți expedia datele unui formular utilizând una din cele două

modalitati:

- Prin apelarea metodei "submit()" a obiectului "Form".
- Prin click pe un buton Submit care trimite automat datele din formularul de care apartine.

Multe din proprietatile obiectului "Form" se ocupa de informatiile suplimentare care sunt trimise la server prin intermediul formularului. Aceste proprietati sunt:

- **action** - specifica adresa URL a programului catre care vor fi trimise datele din formular

Exemplu:

```
<form name="f1" action="http://www.web.cich.md/script.php" method="post">
```

Pentru formularele care folosesc metoda "post" puteti preciza in atributul "action" ca datele sa fie trimise prin e-mail, folosind urmatoarea sintaxa:

```
<form name="f2" action="mailto:adresa_de_email" method="post">
```

Cînd expediatu formularul, se va deschide un client de e-mail ce contine, printre altele, și perechile de "nume-valoare" ale elementelor (casetelor) cuprinse in formular.

- **enctype** - specifica o codificare MIME a formularului, valoarea prestabilita este "application/x-www-form-urlencoded".

O metoda de verificare a acestei codificari este urmatoarea:

```
if (f1.enctype == "application/x-www-form-urlencoded") {  
    alert("Tipul de codificare e normal");  
}
```

Un alt tip de codificare des folosit este "text/plain".

Iata un exemplu:

```
1  
2 <form method="post" name="f3" action="mailto:adresa_de_mail" enctype="text/plain">  
3   <br>  
4   Va place site-ul? <br><br>  
5   <input name="alege" type="radio" value="1" checked> Da  
6   <br>  
7   <input name="alege" type="radio" value="2"> Nu  
8   <br>  
9   <input name="alege" type="submit" value="Trimite">  
   </form>
```

```
1  
2 <form method="post" name="f3" action="mailto:adresa_de_mail" enctype="text/plain">  
3   <br>  
4   Va place site-ul? <br><br>  
5   <input name="alege" type="radio" value="1" checked> Da  
6   <br>  
7   <input name="alege" type="radio" value="2"> Nu  
8   <br>  
9   <input name="alege" type="submit" value="Trimite">  
   </form>
```

Proprietatea enctype="text/plain" este folosita pentru a trimite text neformatat. Prin aceasta, mailul va fi mai usor de citit, in cadrul mesajului va apare optiunea aleasa. Fara specificarea acestei proprietati mailul va avea ca atasament un fisier cu optiunea aleasa.

- **method** - defineste modul in care datele formularului sunt expediate. Se poate folosi valoarea "get" sau "post".

Pentru formularele care folosesc metoda "get", in majoritatea site-urilor este disponibil un program CGI de testare, numit "test-cgi" furnizat impreuna cu serverul (locatia fisierului "test-cgi" poate fi diferita, intrebati administratorul serverului). Folositi urmatoarea sintaxa:

```
<form action="http://nume_site/cgi-bin/test-cgi" method="get">
```

Veti primi o pagina care va contine, printre altele, și valorile elementelor din formular. Pentru incepatori, mai puțin cunoscatori, și pentru folosirea cu scripturi PHP, este recomandata metoda "post". Pentru formularele care folosesc metoda "post", in majoritatea site-urilor este disponibil un program CGI de testare, numit "post-query" furnizat impreuna cu serverul (locatia acestuia poate fi diferita, intrebati administratorul serverului). Daca doriti sa verificati ce date trimite formularul, folositi urmatoarea eticheta:

```
<form action="http://nume_site/cgi-bin/post-query" method="post">
```

Cind trimiteti formularul veti primi, printre altele, și perechile de "nume-valoare" corespunzatoare elementelor din formular. Puteti verifica tipul metodei de trimitere a datelor și cu un script JavaScript, ca in exemplul urmator:

```
<script>
```

```
<!--
```

```
var tip_metoda;
```

```
tip_metoda = formular.method; alert("Tipul metodei pt. acest formular e :"+tip_metoda);
```

```
//-->
```

```
</script>
```

Daca doriti sa faceti o verificare a datelor din formular inainte ca acestea sa fi expediate la server, puteti folosi evenimentul "onSubmit".

onSubmit - este folosita pentru a valida formularul inainte de a fi trimis.

Metoda "onSubmit" se introduce in interiorul etichetei <form>, ca in exemplul urmator:

```
1  <script>
2      <!--
3      // Verifica daca input OK
4      function validare() {
5          if (inputOK)
6              return true;
7          else
8              return false;
9      }
10     //-->
11     </script>
12     ...
13     <form method="post" name="nume_form" action="script.php" onSubmit="return validare()">
14         ...
```

```
1  <script>
2      <!--
3      // Verifica daca input OK
4      function validare() {
5          if (inputOK)
6              return true;
7          else
8              return false;
9      }
10     //-->
11     </script>
12     ...
13     <form method="post" name="nume_form" action="script.php" onSubmit="return validare()">
14         ...
```

Validarea formularelor

JavaScript poate fi utilizat pentru a valida formularele HTML înainte de a fi trimise către server.

Datele verificate uzual cu JavaScript pot fi:

au rămas câmpuri obligatorii necompletate?

adresa de e-mail este validă?

data calendaristică este validă?

s-a introdus text într-un cîmp numeric?

ș.a.

Cîmpuri obligatorii

Exemplul următor utilizează o funcție care verifică dacă un cîmp obligatoriu a rămas necompletat. În caz afirmativ, o casetă de alertare afișează un mesaj și funcția returnează valoarea **false**. Dacă cîmpul a fost completat, funcția returnează valoarea **true** și data este considerată validă:

```

1  <html>
2  <head>
3  <script>
4  function valideaza_obligatoriu(camp,txt)
5  {
6  with (camp)
7  {
8  if (value==null||value=="")
9  {
10 alert(txt);return false;
11 }
12 else
13 {
14 return true;
15 }
16 }
17 }
18 }
19 function valideaza_formular(formular)
20 {
21 with (formular)
22 {
23 if (valideaza_obligatoriu(email,"Campul Email este obligatoriu!")==false)
24 {
25 email.focus();return false;
26 }
27 }
28 }
29 </script>
30 </head>
31 <body>
32 <form action="submit.htm" onsubmit="return valideaza_formular(this)" method="post">
33 Email: <input type="text" name="email" size="30">
34 <input type="submit" value="Trimite">
35 </form>
36 </body>
   </html>

```

```

1  <html>
2  <head>
3  <script>
4  function valideaza_obligatoriu(camp,txt)
5  {
6  with (camp)
7  {
8  if (value==null||value=="")
9  {
10 alert(txt);return false;
11 }
12 else
13 {
14 return true;
15 }
16 }
17 }
18 }
19 function valideaza_formular(formular)

```

```

20 {
21   with (formular)
22   {
23     if (valideaza_obligatoriu(email,"Campul Email este obligatoriu!")==false)
24     {
25       email.focus();return false;
26     }
27   }
28 }
29 </script>
30 </head>
31 <body>
32 <form action="submit.htm" onsubmit="return valideaza_formular(this)" method="post">
33 Email: <input type="text" name="email" size="30">
34 <input type="submit" value="Trimite">
35 </form>
36 </body>
</html>

```

Validarea adresei de e-mail

Exemplul următor utilizează o funcție care verifică dacă câmpul respectă sintaxa generală a unei adrese de e-mail. Asta înseamnă că date respectivă trebuie să conțină cel puțin caracterul @ și un punct. De asemenea, @ nu poate fi primul caracter din șir, iar ultimul punct trebuie să fie cel puțin la un caracter distanță de @:

```

1
2 <html>
3 <head>
4 <script>
5 function valideaza_email(camp,txt)
6 {
7   with (camp)
8   {
9     apos=value.indexOf("@");
10    dotpos=value.lastIndexOf(".");
11    if (apos<1||dotpos-apos<2)
12    {alert(txt);return false;}
13    else {return true;}
14  }
15 }
16
17 function valideaza_formular(formular)
18 {
19   with (formular)
20   {
21     if (valideaza_email(email,"Adresa e-mail nu este valida!")==false)
22     {email.focus();return false;}
23   }
24 }
25 </script>
26 </head>
27 <body>
28 <form action="submit.htm" onsubmit="return valideaza_formular(this);" method="post">
29 Email: <input type="text" name="email" size="30">
30 <input type="submit" value="Trimite">
31 </form>
32 </body>
</html>

```

```

1
2 <html>
3 <head>
4 <script>
5 function valideaza_email(camp,txt)
6 {
7   with (camp)
8   {
9     apos=value.indexOf("@");
10    dotpos=value.lastIndexOf(".");

```

```

11  if (apos<1||dotpos-apos<2)
12  {alert(txt);return false;}
13  else {return true;}
14  }
15  }
16
17  function valideaza_formular(formular)
18  {
19  with (formular)
20  {
21  if (valideaza_email(email,"Adresa e-mail nu este valida!")==false)
22  {email.focus();return false;}
23  }
24  }
25  </script>
26  </head>
27  <body>
28  <form action="submit.htm" onsubmit="return valideaza_formular(this);" method="post">
29  Email: <input type="text" name="email" size="30">
30  <input type="submit" value="Trimite">
31  </form>
32  </body>
</html>

```

Validarea Butoanelor Radio și Checkbox

În caz că sunt mai multe variante de răspuns la o întrebare și doriți să se poată alege doar una este suficient să dați nume identice butoanelor radio care sunt cuprinse în acea întrebare. Vom folosi JavaScript pentru a ne asigura că formularul nu va fi trimis fără să fie bifat un răspuns la fiecare întrebare.

Întroducem funcția *check_buttons* care va indica spre fișierul care să preia datele din formular numai în cazul în care s-a răspuns la fiecare rubrică. Vom verifica asta cu IF, parcurgînd înainte fiecare buton ca pe un array cu FOR.

```

1  <html>
2  <head>
3  </head>
4  <body>
5  <script>
6  function check_buttons(){
7  for(i=0; i<4; i++)
8  for (j=0; j<4; j++){
9  if(document.form1.intr1[i].checked==true) {
10  if(document.form1.intr2[j].checked==true) {
11  document.form1.action="rezultat.php"; }
12  }
13  }
14  return
15  }
16  }
17  </script>
18  <form name="form1" method="post" onsubmit="javascript:check_buttons();" action="">
19  1. Cîte limbaje de programare cunoști?
20  <p>
21  <input type="radio" name="intr1" value="a">a. Nici unul<br>
22  <input type="radio" name="intr1" value="b">b. 1<br>
23  <input type="radio" name="intr1" value="c">c. 2<br>
24  <input type="radio" name="intr1" value="d">d. 3<br>
25  </p>
26  2. De cît timp ești interesat de Web Design?
27  <p>
28  <input type="radio" name="intr2" value="a">a. De cînd m-am născut<br>
29  <input type="radio" name="intr2" value="b">b. De mai mult de 10 ani<br>
30  <input type="radio" name="intr2" value="c">c. De mai puțin de 10 ani<br>
31  <input type="radio" name="intr2" value="d">d. De Ieri<br>
32  </p>
33  <br>
34  <input type="submit" value="Trimite formularul">
35  </form>

```

```

36 | </body>
    | </html>

1 |
2 | <html>
3 | <head>
4 | </head>
5 | <body>
6 | <script>
7 |     function    check_buttons(){
8 |         for(i=0; i<4; i++)
9 |             for (j=0; j<4; j++){
10 |                 if(document.form1.intr1[i].checked==true) {
11 |                     if(document.form1.intr2[j].checked==true) {
12 |                         document.form1.action="rezultat.php"; }
13 |                     }
14 |             }
15 |         return
16 |     }
17 | </script>
18 | <form name="form1" method="post" onSubmit="javascript:check_buttons();" action="">
19 |     1. Cite    limbaje de programare cunoști?
20 |     <p>
21 |         <input type="radio" name="intr1" value="a">a. Nici unul<br>
22 |         <input type="radio" name="intr1" value="b">b. 1<br>
23 |         <input type="radio" name="intr1" value="c">c. 2<br>
24 |         <input type="radio" name="intr1" value="d">d. 3<br>
25 |     </p>
26 |     2. De cît timp ești    interesat de Web Design?
27 |     <p>
28 |         <input type="radio" name="intr2" value="a">a. De cînd m-am născut<br>
29 |         <input type="radio" name="intr2" value="b">b. De mai mult de 10 ani<br>
30 |         <input type="radio" name="intr2" value="c">c. De mai puțin de 10 ani<br>
31 |         <input type="radio" name="intr2" value="d">d. De Ieri<br>
32 |     </p>
33 |     <br>
34 |     <input type="submit" value="Trimite formularul">
35 | </form>
36 | </body>
    | </html>

```

Animație

Puteți folosi JavaScript pentru a crea imagini animate. Secretul este să lăsați scriptul să afișeze imagini diferite pentru evenimente diferite.

În exemplul următor vom adăuga o imagine care se va comporta ca un link în pagina web. Vom adăuga apoi un eveniment **onMouseOver** și un eveniment **onMouseOut** care vor apela două funcții JavaScript ce vor comuta între două imagini.

Codul HTML arată astfel:

```

1 |
2 | <a href="http://www.google.com" target="_blank">
3 | 
  | </a>

1 |
2 | <a href="http://www.google.com" target="_blank">
3 | 
  | </a>

```

Observați că imaginea a primit un **id**, pentru ca JavaScript să poată referi imaginea în diferite puncte din script.

Evenimentul **onMouseOver** va spune browserului c ,  n momentul  n care mouse-ul trece peste imagine, trebuie apelat  o func ie care s  schimbe imaginea. Evenimentul **onMouseOut** va spune browserului c , atunci c nd mouse-ul se  ndep rteaz  de imagine, trebuie apelat  o func ie care va afi a din nou imaginea ini ial . Codul celor dou  func ii este:

```

1  <script>
2  function mouseOver()
3  {
4  document.getElementById("m1").src ="img1.gif";
5  }
6  function mouseOut()
7  {
8  document.getElementById("m1").src ="img2.gif";
9  }
10 </script>

1  <script>
2  function mouseOver()
3  {
4  document.getElementById("m1").src ="img1.gif";
5  }
6  function mouseOut()
7  {
8  document.getElementById("m1").src ="img2.gif";
9  }
10 </script>

```

Func ia **mouseOver()** va determina afi area imaginii **"img1.gif"**, iar func ia **mouseOut()** va determina afi area imaginii **"img2.gif"**. Efectul de anima ie este mai vizibil dac  cele dou  imagini sunt foarte asem n toare, diferind spre exemplu prin culoare. Codul  ntregului program este:

```

1  <html>
2  <head>
3  <script>
4  function mouseOver()
5  {
6  document.getElementById("m1").src ="img1.gif";
7  }
8  function mouseOut()
9  {
10 document.getElementById("m1").src ="img2.gif";
11 }
12 </script>
13 </head>
14 <body>
15 <a href="http://www.google.com" target="_blank">
16 
18 </a>
19 </body>
20 </html>

1  <html>
2  <head>
3  <script>
4  function mouseOver()
5  {
6  document.getElementById("m1").src ="img1.gif";
7  }
8  function mouseOut()
9  {

```

```

10 {
11 document.getElementById("m1").src ="img2.gif";
12 }
13 </script>
14 </head>
15 <body>
16 <a href="http://www.google.com" target="_blank">
17 
19 </a>
20 </body>
21 </html>

```

Imagini mapate

O imagine mapată (image-map) este o imagine care are zone ce pot fi acționate cu mouse-ul. În mod normal, fiecare zonă are un hiperlink asociat. În tagurile **<area>** din imaginea mapată pot fi adăugate evenimente care apelează funcții JavaScript. Tagul **<area>** suportă evenimentele **onClick**, **onDbIClick**, **onMouseDown**, **onMouseUp**, **onMouseOver**, **onMouseMove**, **onMouseOut**, **onKeyPress**, **onKeyDown**, **onKeyUp**, **onFocus** și **onBlur**.

Exemplul următor ilustrează utilizarea unei imagini mapate într-un program HTML:

```

1 <html>
2 <head>
3 <script>
4 function writeText(txt)
5 {
6   document.getElementById("desc").innerHTML=txt;
7 }
8 </script>
9 </head>
10 <body>
11 
12 <map name="planetmap"> </p>
13 <area shape="rect" coords="0,0,82,126" onMouseOver="writeText('Soarele și planetele gigante
14 gazoase, ca Jupiter, sunt cu certitudine cele mai mari corpuri din sistemul nostru solar.')"
15 href="sun.htm" target="_blank" alt="Sun" />
16
17 <area shape="circle" coords="90,58,3" onMouseOver="writeText('Planeta Mercur este foarte greu
18 de studiat de pe Pamant datorita apropierii ei de Soare.')" href="mercur.htm" target
19 ="_blank" alt="Mercur" />
20
21 <area shape="circle" coords="124,58,8" onMouseOver="writeText('Pana in anii 60, Venus a fost
22 adesea considerata sora geamana a Pamantului pentru ca este cea mai apropiata de noi, și cele
23 doua planete au multe caracteristici comune.')" href="venus.htm" target="_blank" alt="Venus"
24 />
25 </map>
26 <p id="desc"></p>
27 </body>
28 </html>

```

```

1 <html>
2 <head>
3 <script>
4 function writeText(txt)
5 {
6   document.getElementById("desc").innerHTML=txt;
7 }
8 </script>
9 </head>
10 <body>
11 
12 <map name="planetmap"> </p>
13 <area shape="rect" coords="0,0,82,126" onMouseOver="writeText('Soarele și planetele gigante
14 gazoase, ca Jupiter, sunt cu certitudine cele mai mari corpuri din sistemul nostru solar.')"
15 href="sun.htm" target="_blank" alt="Sun" />
16

```



```

17 <area shape="circle" coords="90,58,3" onMouseOver="writeText('Planeta Mercur este foarte greu
18 de studiat de pe Pamant datorita apropierei ei de Soare.')" href="mercur.htm" target
19 ="_blank" alt="Mercur" />
20
21 <area shape="circle" coords="124,58,8" onMouseOver="writeText('Pana in anii 60, Venus a fost
    adesea considerata sora geamana a Pamantului pentru ca este cea mai apropiata de noi, si cele
    doua planete au multe caracteristici comune.')" href="venus.htm" target="_blank" alt="Venus"
    />
    </map>
    <p id="desc"></p>
    </body>
    </html>

```

Programarea evenimentelor

Codurile JavaScript pot fi executate la intervale de timp programate. Programarea evenimentelor JavaScript se realizează ușor cu ajutorul următoarelor două metode:

setTimeout() - execută un cod cândva în viitor

clearTimeout() - anulează programările realizate cu **setTimeout()**

Obs: Ambele metode aparțin obiectului **Window** din HTML DOM.

Metoda setTimeout()

Sintaxă:

var t=setTimeout("declarație javascript", milliseconds);

Metoda **setTimeout()** returnează o valoare care este memorată în variabila **t** declarată mai sus. Dacă doriți să anulați programarea, o puteți face folosind variabila asociată. Primul argument al metodei este un șir de caractere care conține o declarație JavaScript care poate fi, de exemplu, un apel de funcție sau instrucțiunea de afișare a unei casete de mesaj. Al doilea parametru precizează numărul de milisecunde (începând de acum) după care va fi executat primul parametru.

Obs: O secundă are 1000 de milisecunde.

Exemplul 1

În exemplul următor, când butonul este apăsat, o casetă de alertare va fi afișată după 7 secunde.

```

1 <html>
2 <head>
3 <script>
4 function mesaj()
5 {
6   var t=setTimeout("alert('Casetă afisată după 7 secunde!')",7000);
7 }
8 </script>
9 </head>
10 <body>
11 <form>
12 <input type="button" value="Afisează mesajul!" onClick="mesaj()" />
13 </form>
14 </body>
15 </html>

```

```

1 <html>
2 <head>
3 <script>
4 function mesaj()
5 {
6   var t=setTimeout("alert('Casetă afisată după 7 secunde!')",7000);
7 }
8 </script>
9

```

```

10 </head>
11 <body>
12 <form>
13 <input type="button" value="Afiseaza mesajul!" onClick="mesaj()" />
14 </form>
15 </body>
   </html>

```

Exemplul 2

Pentru a repeta la infinit o secvență de cod, trebuie să scriem o funcție care se autoapelează. În exemplul următor, când butonul este apăsat, un câmp de intrare dintr-un formular va începe să numere, plecând de la zero, secunde scurse, fără să se oprească. A fost inclusă și o funcție care verifică dacă numărătorul funcționează deja, pentru a nu crea un alt numărător dacă butonul este apăsat de mai multe ori.

```

1  <html>
2  <head>
3  <script>
4    var c=0;
5    var t;
6    var pornit=0;
7
8    function numara()
9    {
10     document.getElementById('txt').value=c;
11     c=c+1;
12     t=setTimeout("numara()",1000);
13   }
14
15   function verifica()
16   {
17     if (!pornit)
18     {
19       pornit=1;
20       numara();
21     }
22   }
23
24 </script>
25 </head>
26 <body>
27 <form>
28 <input type="button" value="Incepe numararea!" onClick="verifica()">
29 <input type="text" id="txt" />
30 </form>
31 </body>
   </html>

```

```

1  <html>
2  <head>
3  <script>
4    var c=0;
5    var t;
6    var pornit=0;
7
8    function numara()
9    {
10     document.getElementById('txt').value=c;
11     c=c+1;
12     t=setTimeout("numara()",1000);
13   }
14
15   function verifica()
16   {
17     if (!pornit)
18     {
19       pornit=1;
20       numara();
21     }
22   }
23
24 </script>
25 </head>
26 <body>
27 <form>
28 <input type="button" value="Incepe numararea!" onClick="verifica()">
29 <input type="text" id="txt" />
30 </form>
31 </body>
   </html>

```

```

22     }
23     }
24 </script>
25 </head>
26 <body>
27 <form>
28 <input type="button" value="Incepe numararea!" onClick="verifica()">
29 <input type="text" id="txt" />
30 </form>
31 </body>
</html>

```

Metoda clearTimeout()

Sintaxă:

clearTimeout(variabila_setTimeout)

În exemplul următor utilizăm numărătorul infinit din exemplul următor și adăugăm o funcție care va opri numărătorul la apăsarea unui buton:

```

1
2 <html>
3 <head>
4 <script>
5 var c=0;
6 var t;
7 var pornit=0;
8
9 function numara()
10 {
11 document.getElementById('txt').value=c;
12 c=c+1;
13 t=setTimeout("numara()",1000);
14 }
15
16 function verifica()
17 {
18 if (!pornit)
19 {
20 pornit=1;
21 numara();
22 }
23 }
24
25 function stop()
26 {
27 clearTimeout(t);
28 pornit=0;
29 }
30 </script>
31 </head>
32 <body>
33 <form>
34 <input type="button" value="Incepe numararea!" onClick="verifica()">
35 <input type="text" id="txt">
36 <input type="button" value="Opreste numararea!" onClick="stop()">
37 </form>
38 </body>
</html>

```

```

1
2 <html>
3 <head>
4 <script>
5 var c=0;
6 var t;
7 var pornit=0;
8
9 function numara()
10 {

```

```

10
11 document.getElementById('txt').value=c;
12 c=c+1;
13 t=setTimeout("numara()",1000);
14 }
15
16 function verifica()
17 {
18 if (!pornit)
19 {
20 pornit=1;
21 numara();
22 }
23 }
24
25 function stop()
26 {
27 clearTimeout(t);
28 pornit=0;
29 }
30 </script>
31 </head>
32 <body>
33 <form>
34 <input type="button" value="Incepe numararea!" onClick="verifica()">
35 <input type="text" id="txt">
36 <input type="button" value="Opreste numararea!" onClick="stop()">
37 </form>
38 </body>
</html>

```

Exemplu

Acest exemplu ilustrează crearea unui ceas cu ajutorul evenimentelor programate.

```

1
2 <html>
3 <head>
4 <script>
5 function numara()
6 {
7   var azi=new Date();
8   var h=azi.getHours();
9   var m=azi.getMinutes();
10  var s=azi.getSeconds();
11  // adauga un zero in fata numerelor <10
12  m=verifica(m);
13  s=verifica(s);
14  document.getElementById('txt').innerHTML=h+":"+m+":"+s;
15  t=setTimeout('numara()',500);
16 }
17
18 function verifica(i)
19 {
20 if (i<10)
21 {
22 i="0" + i;
23 }
24 return i;
25 }
26 </script>
27 </head>
28 <body onload="numara()">
29 <div id="txt"></div>
30 </body>
</html>

```

```

1
2 <html>
3 <head>
4 <script>

```

```

5   function numara()
6   {
7   var azi=new Date();
8   var h=azi.getHours();
9   var m=azi.getMinutes();
10  var s=azi.getSeconds();
11  // adauga un zero in fata numerelor <10
12  m=verifica(m);
13  s=verifica(s);
14  document.getElementById('txt').innerHTML=h+":"+m+":"+s;
15  t=setTimeout('numara()',500);
16  }
17
18  function verifica(i)
19  {
20  if (i<10)
21  {
22  i="0" + i;
23  }
24  return i;
25  }
26  </script>
27  </head>
28  <body onload="numara()">
29  <div id="txt"></div>
30  </body>
    </html>

```

Depanarea aplicațiilor JavaScript

Instrucțiunea try...catch

Cînd navigăm prin paginile web de pe internet, pot să apară mesaje de eroare la încărcarea unei pagini. În acest caz, uzual, apare o casetă de alertare JavaScript care ne anunță că s-a detectat o eroare de execuție (runtime error) și ne întreabă dacă dorim să depanăm codul paginii. Aceste mesaje sunt utile pentru proiectanții paginilor web, nu și pentru vizitatori care, de obicei, părăsesc pagina respectivă.

În acest capitol veți învăța cum să gestionați mesajele de eroare JavaScript, astfel încît să nu vă pierdeți audiența.

Instrucțiunea **try...catch** vă permite să testați blocurile de cod pentru a depista erorile. Blocul **try** conține codul ce trebuie executat, iar blocul **catch** conține codul ce va fi executat dacă apare o eroare.

Sintaxă:

```

try {
    codul ce trebuie executat
}
catch(err) {
    gestionarea erorilor
}

```

Exemple

În exemplul următor ar trebui afișată o casetă de alertare cu mesajul **"Bine ati venit!"** cînd butonul este acționat. Totuși, în corpul funcției **mesaj()** există o eroare, cuvîntul rezervat **alert** este scris greșit. Această eroare va fi detectată de JS. Blocul **catch** sesizează eroarea și execută un cod special pentru a o rezolva. Acest cod afișează un mesaj de eroare pentru a informa utilizatorul ce se întîmplă. Dacă utilizatorul apasă butonul **OK**, încărcarea paginii va continua fără probleme:

```

1   <html>
2

```

```

3   <head>
4   <script>
5   var txt="";
6   function mesaj()
7   {
8   try
9   {
10  adddler("Bine ati venit!");
11  }
12  catch(err)
13  {
14  text="In aceasta pagina este o eroare.\n\n";
15  text+="Descrierea erorii: " + err.description + "\n\n";
16  text+="Pentru a continua apasati OK.\n\n";
17  alert(text);
18  }
19  }
20 </script>
21 </head>
22 <body>
23 <h3>Utilizarea instructiunii try..catch pentru sesizarea erorilor</h3> <hr/>
24 <input type="button" value="Vedeti mesajul" onclick="mesaj()" />
25 </body>
    </html>

```

```

1   <html>
2   <head>
3   <script>
4   var txt="";
5   function mesaj()
6   {
7   try
8   {
9   {
10  adddler("Bine ati venit!");
11  }
12  catch(err)
13  {
14  text="In aceasta pagina este o eroare.\n\n";
15  text+="Descrierea erorii: " + err.description + "\n\n";
16  text+="Pentru a continua apasati OK.\n\n";
17  alert(text);
18  }
19  }
20 </script>
21 </head>
22 <body>
23 <h3>Utilizarea instructiunii try..catch pentru sesizarea erorilor</h3> <hr/>
24 <input type="button" value="Vedeti mesajul" onclick="mesaj()" />
25 </body>
    </html>

```

În exemplul următor **alert** este de asemenea scris greșit. Blocul **catch** utilizează o casetă de confirmare pentru a afișa un mesaj care informează utilizatorii că pot apăsa **OK** pentru a continua să viziteze pagina în care a fost depistată eroarea sau pot apăsa **Cancel** dacă doresc să se întoarcă la pagina principală (homepage). Dacă metoda **confirm** returnează **false** (utilizatorul a acționat butonul **Cancel**), atunci utilizatorul este redirectat. Dacă **confirm** returnează **true**, codul din blocul **catch** nu are nici-un efect:

```

1   <html>
2   <head>
3   <script>
4   var txt="";
5   function mesaj()
6   {
7   try
8   {
9   {
10  adddler("Bine ati venit!");

```

```

11 }
12 catch(err)
13 {
14   text="In aceasta pagina este o eroare.\n\n";
15   text+="Apasati OK daca doriti sa continuati vizualizarea paginii,\n";
16   text+="sau Cancel pentru a va intoarce la pagina principala.\n\n";
17   if(!confirm(text))
18   {
19     document.location.href="http://cich.md";
20   }
21 }
22 }
23 </script>
24 </head>
25 <body>
26 <h3>Un alt exemplu de utilizare a instructiunii try..catch</h3> <hr/>
27 <input type="button" value="Vedeti mesajul" onclick="mesaj()" />
28 </body>
</html>

```

```

1 <html>
2 <head>
3 <script>
4   var txt="";
5   function mesaj()
6   {
7     try
8     {
9       adddler("Bine ati venit!");
10    }
11    catch(err)
12    {
13      text="In aceasta pagina este o eroare.\n\n";
14      text+="Apasati OK daca doriti sa continuati vizualizarea paginii,\n";
15      text+="sau Cancel pentru a va intoarce la pagina principala.\n\n";
16      if(!confirm(text))
17      {
18        document.location.href="http://cich.md";
19      }
20    }
21  }
22 }
23 </script>
24 </head>
25 <body>
26 <h3>Un alt exemplu de utilizare a instructiunii try..catch</h3> <hr/>
27 <input type="button" value="Vedeti mesajul" onclick="mesaj()" />
28 </body>
</html>

```

Instrucțiunea throw

Această instrucțiune vă permite să creați o excepție. Dacă o utilizați împreună cu instrucțiunea **try...catch**, puteți controla execuția programului și afișa mesaje de eroare adecvate.

Sintaxă:

throw(exceptie)

Argumentul *exceptie* poate fi un șir de caractere, un număr întreg, o valoare booleană sau un obiect.

Exemplu:

Exemplul următor testează valoarea variabilei **x**. Dacă valoarea este mai mare decît **0**, mai mică decît **10** sau nu este un număr, blocul **throw** aruncă o eroare. Această eroare este prinsă de blocul **catch** care afișează un mesaj corespunzător:

```

1 <html>
2 <body>
3

```

```

4  <h3>Utilizarea instructiunii throw pentru tratarea corespunzatoare a erorilor</h3> <hr/>
5  <script>
6  var x=prompt("Introduceti un numar cuprins între 0 și 10:", "");
7  try
8  {
9    if(x>10)
10   {
11     throw "Err1";
12   }
13   else if(x<0)
14   {
15     throw "Err2";
16   }
17   else if(isNaN(x))
18   {
19     throw "Err3";
20   }
21   }
22   catch(er)
23   {
24     if(er=="Err1")
25     {
26       alert("Eroare! Valoarea este prea mare");
27     }
28     if(er=="Err2")
29     {
30       alert("Eroare! Valoarea este prea mică");
31     }
32     if(er=="Err3")
33     {
34       alert("Eroare! Valoarea nu este un numar");
35     }
36   }
37 </script>
38 </body>
</html>

```

```

1  <html>
2  <body>
3  <h3>Utilizarea instructiunii throw pentru tratarea corespunzatoare a erorilor</h3> <hr/>
4  <script>
5  var x=prompt("Introduceti un numar cuprins între 0 și 10:", "");
6  try
7  {
8    if(x>10)
9    {
10     throw "Err1";
11   }
12   else if(x<0)
13   {
14     throw "Err2";
15   }
16   else if(isNaN(x))
17   {
18     throw "Err3";
19   }
20   }
21   catch(er)
22   {
23     if(er=="Err1")
24     {
25       alert("Eroare! Valoarea este prea mare");
26     }
27     if(er=="Err2")
28     {
29       alert("Eroare! Valoarea este prea mică");
30     }
31     if(er=="Err3")
32     {
33       alert("Eroare! Valoarea nu este un numar");
34     }
35   }
36 </script>

```



```

37
38 | </body>
    </html>

```

Exemplu:

Exemplul următor ilustrează utilizarea evenimentului **onerror**

```

1 |
2 | <html>
3 | <head>
4 | <script>
5 | onerror=handleErr;
6 | var txt="";
7 | function handleErr(msg,url,l)
8 | {
9 |   txt="In aceasta pagina este o eroare.\n\n";
10 |   txt+="Eroare: " + msg + "\n";
11 |   txt+="URL: " + url + "\n";
12 |   txt+="Linie: " + l + "\n\n";
13 |   txt+="Pentru a continua apasati OK.\n\n";
14 |   alert(txt);
15 |   return true;
16 | }
17 | function message()
18 | {
19 |   adddler("Bine ai venit!");
20 | }
21 | </script>
22 | </head>
23 | <body>
24 | <h3>Exemplu de utilizare a evenimentului onerror</h3>
25 | <hr/>
26 | <input type="button" value="Afiseaza mesajul" onclick="message()" />
27 | </body>
    </html>

```

```

1 |
2 | <html>
3 | <head>
4 | <script>
5 | onerror=handleErr;
6 | var txt="";
7 | function handleErr(msg,url,l)
8 | {
9 |   txt="In aceasta pagina este o eroare.\n\n";
10 |   txt+="Eroare: " + msg + "\n";
11 |   txt+="URL: " + url + "\n";
12 |   txt+="Linie: " + l + "\n\n";
13 |   txt+="Pentru a continua apasati OK.\n\n";
14 |   alert(txt);
15 |   return true;
16 | }
17 | function message()
18 | {
19 |   adddler("Bine ai venit!");
20 | }
21 | </script>
22 | </head>
23 | <body>
24 | <h3>Exemplu de utilizare a evenimentului onerror</h3>
25 | <hr/>
26 | <input type="button" value="Afiseaza mesajul" onclick="message()" />
27 | </body>
    </html>

```

Crearea obiectelor proprii

Obiectele sunt utile pentru a organiza informația. În completarea obiectelor predefinite în JavaScript, cum ar fi **String, Date, Array, etc.**, vă puteți crea propriile obiecte.

Un obiect este doar un tip particular de date, cu o colecție de proprietăți și metode. Spre exemplu, o persoană este un obiect înzestrat cu proprietăți cum ar fi: nume, vîrstă, înălțime, greutate, culoarea ochilor etc. Proprietățile au anumite valori, care diferă de la o persoană la alta. Metodele sunt acțiuni care pot fi realizate cu un obiect. Pentru obiectul persoana, acestea ar putea fi: mananca(), munceste(), doarme(), scrie(), citeste() etc.

Proprietățile

Pentru a accesa o proprietate a unui obiect sintaxa este:

nume_obiect.nume_proprietate

Puteți adăuga proprietăți unui obiect prin simpla atribuire de valori. Dacă presupunem că obiectul persoana există deja, putem să-i adăugăm proprietăți prin atribuire, ca în exemplul următor:

```

1
2 persoana.nume="Popescu";
3 persoana.prenume="Andrei";
4 persoana.varsta=25;
5 persoana.ochi="verzi";
6
document.write(persoana.nume);

1
2 persoana.nume="Popescu";
3 persoana.prenume="Andrei";
4 persoana.varsta=25;
5 persoana.ochi="verzi";
6
document.write(persoana.nume);

```

Codul de mai sus va afișa rezultatul: **Popescu**

Metodele

Pentru a accesa o metodă a unui obiect sintaxa este:

Nume_obiect.nume_metoda()

Obs: Dacă metoda are parametri, ei vor fi scriși între paranteze.

Un obiect poate fi creat în trei moduri:

1. Crearea directă a unui obiect

Secvența următoare de cod creează un obiect și îi adaugă proprietăți:

```

persoana=new Object();
persoana.nume="Popescu";
persoana.prenume="Andrei";
persoana.varsta=25;
persoana.ochi="verzi";

```

Adăugare unei metode se face simplu, ca în exemplul următor:

```
persoana.scrie=scrie;
```

2. Crearea șablonului unui obiect

Șablonul definește structura unui obiect:

```

function persoana(nume,prenume,varsta,ochi)
{
this.nume=nume;
this.prenume=prenume;

```

```

this.varsta=varsta;
this.ochi=ochi;
}

```

Observați că șablonul este doar o funcție. În interiorul funcției trebuie să faceți atribuiri pentru **this.nume_proprietate**. Construcția **"this"** se referă la instanța curentă a obiectului.

După ce ați construit șablonul obiectului, puteți crea noi instanțe după modelul următor:

```

tata=new persoana("Marcu","Ion",40,"verzi");
mama=new persoana("Marcu","Maria",38,"negri");

```

Adăugarea de metode obiectului persoana se realizează tot în interiorul șablonului:

```

function persoana(nume,prenume,varsta,ochi)
{
  this.nume=nume;
  this.prenume=prenume;
  this.varsta=varsta;
  this.ochi=ochi;

```

```

  this.numenou=numenou;
}

```

Observați că metodele sunt funcții atașate obiectului. Acum va trebui scrisă

```

funcția numenou():
function numenou(str)
{
  this.nume=str;
}

```

Puteți folosi metoda astfel: **mama.numenou("Georgescu");**

Proprietăți și metode globale

Aceste proprietăți și metode pot fi folosite pentru orice variabile, din acest motiv se numesc globale.

Proprietățile

Proprietate	Descriere
Infinity	O valoare numerică care reprezintă infinitiv pozitiv/negativ
NaN	O valoare "Not-a-Number"
undefined	Indică o variabilă căreia nu i-a fost atribuită o valoare

Exemplul 1

Ilustrează utilizarea proprietății **NaN**:

```

1  <html>
2  <body>
3  <script>
4  var luna=13;
5  if (luna < 1 || luna > 12)
6  {
7    luna = luna.NaN;
8  }

```

```
9      }
10     document.write(luna);
11     </script>
12     </body>
      </html>

1     <html>
2     <body>
3     <script>
4     var luna=13;
5     if (luna < 1 || luna > 12)
6     {
7     {
8     luna = luna.NaN;
9     }
10    document.write(luna);
11    </script>
12    </body>
      </html>
```

Exemplul 2

Ilustrează utilizarea proprietății **undefined**:

```
1     <html>
2     <body>
3     <script>
4     var t1="";
5     var t2;
6     if (t1==undefined)
7     {
8     {
9     document.write("Variabila t1 nu este definita");
10    }
11    if (t2==undefined)
12    {
13    document.write("Variabila t2 nu este definita");
14    }
15    </script>
16    </body>
      </html>

1     <html>
2     <body>
3     <script>
4     var t1="";
5     var t2;
6     if (t1==undefined)
7     {
8     {
9     document.write("Variabila t1 nu este definita");
10    }
11    if (t2==undefined)
12    {
13    document.write("Variabila t2 nu este definita");
14    }
15    </script>
16    </body>
      </html>
```

Metodele

Funcție	Descriere
escape()	Codează caracterele speciale dintr-un șir de caractere astfel incit șirul devine portabil în rețea către orice calculator care suportă codurile ASCII

eval()	Evaluează un șir de caractere și, dacă șirul conține o secvență de cod JavaScript, execută secvența
isFinite()	Determină dacă valoarea este un număr valid, finit
isNaN()	Determină dacă valoarea este un număr invalid
Number()	Convertește valoarea unui obiect în număr
parseFloat()	Convertește un șir într-un număr zecimal
parseInt()	Convertește un șir într-un număr întreg
String()	Convertește valoarea unui obiect într-un șir
unescape()	Decodează un șir codat

Exemplul 1

Ilustrează utilizarea metodei **eval()**:

```

1  <html>
2  <body>
3  <script>
4  eval("x=10;y=20;document.write(x*y)");
5  document.write("<br />" + eval("2+2"));
6  document.write("<br />" + eval(x+17));
7  </script>
8  </body>
9  </html>

1  <html>
2  <body>
3  <script>
4  eval("x=10;y=20;document.write(x*y)");
5  document.write("<br />" + eval("2+2"));
6  document.write("<br />" + eval(x+17));
7  </script>
8  </body>
9  </html>

```

Exemplul 2

Ilustrează utilizarea metodei **Number()**:

```

1  <html>
2  <body>
3  <script>
4  var t1= new Boolean(true);
5  var t2= new Boolean(false);
6  var t3= new Date();
7  var t4= new String("921");
8  var t5= new String("193 469");
9
10
11 document.write(Number(t1)+ "<br />");
12 document.write(Number(t2)+ "<br />");
13 document.write(Number(t3)+ "<br />");
14 document.write(Number(t4)+ "<br />");
15 document.write(Number(t5)+ "<br />");
16 </script>
17 </body>
18 </html>

```

```

1  <html>
2  <body>
3  <script>
4  var t1= new Boolean(true);
5  var t2= new Boolean(false);
6  var t3= new Date();
7  var t4= new String("921");
8  var t5= new String("193 469");
9
10 document.write(Number(t1)+ "<br />");
11 document.write(Number(t2)+ "<br />");
12 document.write(Number(t3)+ "<br />");
13 document.write(Number(t4)+ "<br />");
14 document.write(Number(t5)+ "<br />");
15 </script>
16 </body>
17 </html>

```

Exemplul 3

Ilustrează utilizarea metodei **parseFloat()** pentru a extrage valoarea dintr-un șir ca număr zecimal:

```

1  <html>
2  <body>
3  <script>
4  document.write(parseFloat("10") + "<br />");
5  document.write(parseFloat("10.00") + "<br />");
6  document.write(parseFloat("10.33") + "<br />");
7  document.write(parseFloat("34 45 66") + "<br />");
8  document.write(parseFloat(" 60 ") + "<br />");
9  document.write(parseFloat("40 de ani") + "<br />");
10 document.write(parseFloat("Ea are 40 de ani") + "<br />");
11 </script>
12 </body>
13 </html>

```

```

1  <html>
2  <body>
3  <script>
4  document.write(parseFloat("10") + "<br />");
5  document.write(parseFloat("10.00") + "<br />");
6  document.write(parseFloat("10.33") + "<br />");
7  document.write(parseFloat("34 45 66") + "<br />");
8  document.write(parseFloat(" 60 ") + "<br />");
9  document.write(parseFloat("40 de ani") + "<br />");
10 document.write(parseFloat("Ea are 40 de ani") + "<br />");
11 </script>
12 </body>
13 </html>

```

Obs. Metoda verifică dacă primul caracter din șir poate apare în scrierea unei valori zecimale și, în caz afirmativ continuă construirea acestui număr până la întâlnirea primului caracter care nu poate apare în scrierea unui număr.

Exemplul 4

Ilustrează utilizarea metodei **parseInt()** pentru a extrage valoarea dintr-un șir ca număr întreg:

```

1  <html>
2  <body>
3  <script>
4  document.write(parseInt("10") + "<br />");
5  document.write(parseInt("10.33") + "<br />");
6  document.write(parseInt("34 45 66") + "<br />");
7  document.write(parseInt(" 60 ") + "<br />");
8

```

```

9 document.write(parseInt("40 de ani") + "<br />");
10 document.write(parseInt("Ea are 40 de ani") + "<br />");
11 document.write("<br />");
12 document.write(parseInt("10",10)+ "<br />");
13 document.write(parseInt("010")+ "<br />");
14 document.write(parseInt("10",8)+ "<br />");
15 document.write(parseInt("0x10")+ "<br />");
16 document.write(parseInt("10",16)+ "<br />");
17 </script>
18 </body>
    </html>

```

```

1 <html>
2 <body>
3 <script>
4 document.write(parseInt("10") + "<br />") ;
5 document.write(parseInt("10.33") + "<br />");
6 document.write(parseInt("34 45 66") + "<br />");
7 document.write(parseInt(" 60 ") + "<br />");
8 document.write(parseInt("40 de ani") + "<br />");
9 document.write(parseInt("Ea are 40 de ani") + "<br />");
10 document.write("<br />");
11 document.write(parseInt("10",10)+ "<br />");
12 document.write(parseInt("010")+ "<br />");
13 document.write(parseInt("10",8)+ "<br />");
14 document.write(parseInt("0x10")+ "<br />");
15 document.write(parseInt("10",16)+ "<br />");
16 </script>
17 </body>
18 </html>

```

Obs. Dacă numărul începe cu **0** va fi interpretat ca fiind scris în baza **8**, iar dacă începe cu **0x** ca fiind scris în baza **16**. Baza poate fi specificată și prin adăugarea celui de-al doilea parametru în metodă. Conversia se încheie la întâlnirea primului caracter din șir care nu poate apărea în scrierea unui număr întreg.